



Antmicro

Renode - documentation

2026-06-24

INTRODUCTION

INSTALLING RENODE

The installation process of the Renode framework depends on the operating system. Please use the packages available on [Renode's Github](#).

The complete installation instructions are described in the [Renode project README](#).

You can also run Renode in a Docker container - see [the README](#) for details how to do that.

For instructions how to install Renode from sources (for advanced users) see *Building Renode from source*.

USING RENODE

2.1 Running and quitting Renode

To start Renode, use the `renode` command in your operating system's terminal.

The current terminal will now contain the Renode log window, and a new window will pop up for “the Monitor” - the CLI for Renode.

To quit Renode gracefully and close all the associated windows, use the `quit` command in the Monitor at any time.

2.2 Monitor (Renode CLI)

The Monitor is used to interact with Renode and control the emulation. It exposes a set of basic commands and allows the user to access emulation objects. Using the Monitor, the user can execute actions provided by those objects as well as examine and modify their state.

The Monitor comes with several built-in features to make the user experience similar to a regular terminal application.

2.2.1 Using built-in commands

The `help` command provides a list of available built-in commands with a short description:

```
(monitor) help
Available commands:
Name                | Description
=====
alias                : sets an alias.
allowPrivates        : allow private fields and properties manipulation.
analyzers            : shows available analyzers for peripheral.
commandFromHistory   : executes command from history.
createPlatform        : creates a platform.
currentTime          : prints out and logs the current emulation virtual and real_
↳time
...
```

You can get more detailed information about a selected command by using the `help` command with another built-in command as an argument:

```
(monitor) help analyzers
Usage:
-----
analyzers [peripheral]
  lists ids of available analyzer for [peripheral]
analyzers default [peripheral]
  writes id of default analyzer for [peripheral]
```

Typing any command with wrong or incomplete arguments will also print a help string.

For ease of use, a partial autocompletion feature is available. Just press Tab once to complete the current command or twice to see all available suggestions.

For commands with file arguments, the @ sign represents a path to a file; for convenience, Renode also provides autocompletion for filenames.

Note

After a @ sign, the Monitor will suggest files both in the current working directory from which Renode was run and in the Renode installation directory as a fallback - the former taking precedence in case of ambiguity. For non existing files the Renode directory takes precedence. If you want to create a file in your local directory, use the \$CWD variable or provide a full path.

The most common commands (e.g., start or quit) provide short, usually single-letter, aliases (so s and q, respectively).

The CLI provides a command history (arrows ↑/↓) with interactive search Ctrl+R to easily re-execute previous commands.

Pasting with Ctrl+Shift+V, as well as via the context menu on right click, is also available. To erase the current command and return to a clean prompt, use: Ctrl+C.

2.3 Basic interactive workflow

When running Renode interactively, the user would normally start by *creating the emulation* through a sequence of commands building up, configuring, and connecting the relevant emulated (guest) platform or platforms (called “machines”).

This is normally done using nested *.resc scripts* which help encapsulate some of the repeatable elements in this activity (normally, the user will want to create the same platform over and over again in between runs, or even script the execution entirely).

When the emulation is created and all the necessary elements (including e.g. binaries to be executed) are loaded, the emulation itself can be started - to do this, use the start command in the Monitor.

At this point, you will be able to see lots of information about the operation of the emulated environment in the *logger window*, extract additional information and manipulate the running emulation using the Monitor (or plugins such as *Wireshark* - as well as interact with the external interfaces of the emulated machines like UARTs or *Ethernet controllers*).

For some typical commands useful in creating and manipulating machines from the Monitor, you can refer to the *Working with machines* section.

Some more commands and info on interacting with the emulation can be found in the *Basic execution control* section.

2.4 .resc scripts

Renode scripts (.resc) enable you to encapsulate repeatable elements of your project (like creating a machine and loading a binary) to conveniently execute them multiple times. The syntax used in .resc files is the same as that of the Monitor.

Renode has many built-in .resc files, like this [Intel Quark C1000 script](#).

To load it in Renode, use the include command with a path:

```
include @scripts/single-node/quark_c1000.resc
```

If in the above command you use start (or just s) instead of include, the emulation will start immediately after loading the script.


Note

Remember about path autocompletion using the Tab key after @, as described in the [previous section](#).

Scripts can include further scripts, which is useful e.g. to create complex multinode setups like in the [nRF52840 BLE demo](#).

Built-in Renode demo scripts are a great entry point - to run your first demo, proceed to the *Running your first demo* chapter.

2.5 Configuring the user interface

The appearance of the user interface can be customized via the user configuration file config. It is located in the directory ~/.config/renode on Unix-like systems and in AppData\Roaming\renode on Windows.

In the section [termsharp], the following settings are available:

tabulartyabulary

	Name	
	append-CR-to-LF	
	font-face	
	font-size	
	window-width	
	window-height	

RUNNING YOUR FIRST DEMO

Your Renode installation contains a number of example scripts, located in the [scripts/ directory](#) (for example, if you installed from Linux packages, this will be in `/opt/renode/scripts` on your PC).

You can run any of those demos using the `include` or `start` command (`i` and `s` for short) with the script's path (by default relative to the Renode installation directory and your current working directory) as a parameter. For example, run a single node STM32F4 Discovery demo as follows:

```
s @scripts/single-node/stm32f4_discovery.resc
```

Remember about Tab autocompletion, which will hint you what demos are available.

The binaries for the demos are hosted on our servers, and can be replaced with your own by setting the `$bin` variable before loading the script (or changing its value inside the script).

You are free to copy any of the provided demo scripts to your preferred directory and modify them as necessary to match your needs, they should work even from within a different path as they typically only use paths relative to the Renode installation directory.

You can also run a script by passing its path to the `renode` command, this will be interpreted as running Renode and using `include @/path/to/script.resc` (note that you will have to start the emulation manually).

SUPPORTED BOARDS

Renode supports a wide array of hardware platforms, covering multiple architectures, CPU families and providing various I/O capabilities.

You can explore IoT development boards supported as part of the [Zephyr Dashboard](#) and learn more about them in the [Interactive System Designer](#).

In the Interactive System Designer, you can see pre-executed runs of different embedded software binaries as well as use the available artifacts to run the demos yourself.

On top of that, this chapter contains an (incomplete) list of selected supported hardware targets with dedicated demos - all of these include sample software binaries that run both on real hardware and in Renode.

To run example software on any of the below boards, simply run Renode and use:

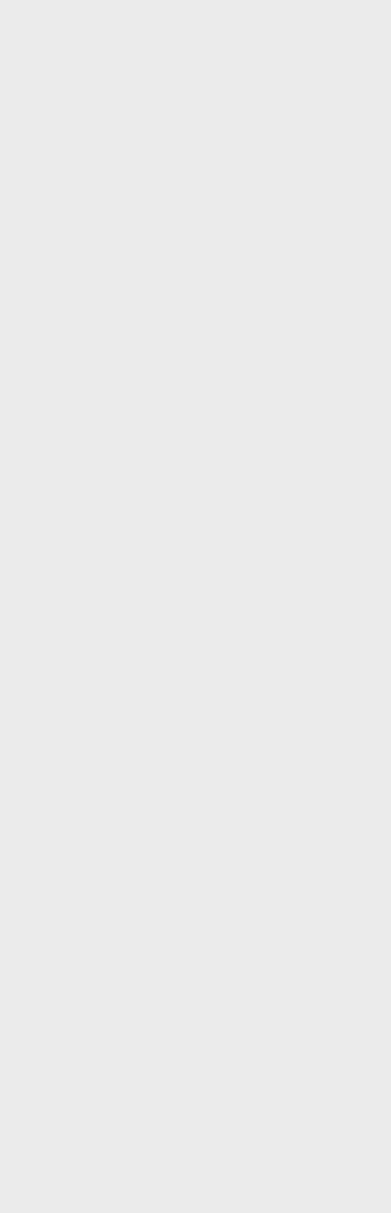
```
s @scripts/PATH/TO/SCRIPT-NAME.resc
```

Tab completion is available also for filenames, so be sure to explore the available demos.

The ultimate goal of Renode is to run any binary-compatible software targeted for any of those hardware platforms without modification, although of course your specific use case may require extending the provided hardware description / models.

Boards supported in this way include:

tabularytabulary



ST Micro STM32 Nucleo-64

There are of course more, and new ones are being added quickly - Renode makes it easy to create your own platform which reuses the same peripherals / CPUs that exist in other platforms.

We provide commercial services to add new platforms - if you need help in this regard, please write to support@renode.io.

SUPPORTED PERIPHERALS

TROUBLESHOOTING

If you encounter a problem, please check if it has not been reported previously in [our GitHub issue tracker](#). Note that we also have our internal issue tracking system and we might already be aware of your problem, but just did not get around to fixing it. Still, it is worthwhile to bring your use case to our attention so that we know it is affecting you / your use case.

The Renode team and community may also be aware of some workarounds or alternative ways to accomplish what you want to do and suggest those.

To speed up the process of getting help, please:

1. reduce your example to the minimal example that still does not work for you (e.g. simplify the software that is crashing to its most basic equivalent exhibiting identical symptoms, only execute the Monitor command that does not work for you etc.)
2. state the operating system you are working on, Renode version used, and if possible, what you are trying to achieve (context helps!)
3. provide all the files and commands needed to reproduce the problem 1:1 (.repl, .resc files), precompiled binary in the ELF format
4. provide logs - the Renode log and UART terminal output where relevant
5. it helps if you can provide sources for your example, along with compilation instructions

Following those recommendations will help us confirm the bug and act on it.

Please note that if for some reason you are not able to share your setup publicly, or you need priority support, Antmicro provides commercial services for Renode. For more information, please contact support@renode.io.

TESTING WITH RENODE

7.1 Renode's testing capabilities

Renode is very well suited to be a part of an automated tests scenario, e.g. run in the background on a CI server.

Renode is integrated with the **Robot Framework** testing suite and provides user-friendly scripts for running tests.

It comes with a variety of prepared test scripts, but it also allows you to extend them or add new ones.

7.2 Running the robot test script

Running a robot test script in Renode is as simple as executing a single command:

```
$ renode-test my_test.robot
```

The above command will:

- start a Renode instance in the background,
- enable Renode's built-in Robot Framework server (providing an interface between Robot Framework and Renode) on port 9999 (the port number can be changed by the user),
- start the Robot Framework test engine and connect to Renode,
- run the provided `my_test.robot` test case,
- print the progress status on the console,
- generate the log and the summary after finishing the test.

Below, you can see an example output:

```
Preparing suites
Started Renode instance on port 9999; pid 2293056
Starting suites
Running tests/platforms/LiteX-VexRiscv.robot
+++++ Starting test 'LiteX-VexRiscv.Timer Test'
+++++ Finished test 'LiteX-VexRiscv.Timer Test' in 46.79 seconds with status OK
+++++ Starting test 'LiteX-VexRiscv.I2C Test'
+++++ Finished test 'LiteX-VexRiscv.I2C Test' in 8.10 seconds with status OK
Cleaning up suites
```

(continues on next page)

(continued from previous page)

```
Closing Renode pid 2293056
Aggregating all robot results
Output: /home/antmicro/renode/output/tests/robot_output.xml
Log:    /home/antmicro/renode/output/tests/log.html
Report: /home/antmicro/renode/output/tests/report.html
Tests finished successfully :)
```

Note

Two entries in the output are associated with a single test case appear - one when the test starts and another when it finishes (useful when running tests in parallel). The second message contains information about the test's duration and status.

The details of the run can be found in:

- the `robot_output.xml` report (suitable for automatic parsing),
- the `log.html` and `report.html` documents (suitable for an interactive inspection).

7.3 Creating the test file

Robot Framework uses text files based on a custom syntax to express the test cases. The details of the grammar can be found in [the official documentation](#). In order for the robot files to work with Renode, proper configuration (explained below) is needed.

Here is an example of a simple robot test file that works with Renode:

```
*** Settings ***
Suite Setup      Setup
Suite Teardown   Teardown
Test Teardown    Test Teardown
Resource         ${RENODEKEYWORDS}

*** Test Cases ***
Should Print Help
    ${x}=    Execute Command    help
            Should Contain     ${x}    Available commands:
```

The `Should Print Help` test case executes the `help` command in Renode's monitor and verifies the result.

Integration with Renode is achieved by adding entries to the settings section. The `RENODEKEYWORDS` variable (initialized by the `renode-test` script) contains the path to the `renode-keywords.robot` script responsible for setting up the connection with Renode. Other settings configure the suite/test setup and teardowns.

It is recommended to copy the above Settings section to each new robot test file.

7.3.1 Adding new test cases

Each robot test file might contain many test cases. For general instructions on how to define tests cases, please refer to [the Robot Framework documentation](#). In this section we will focus on using the Robot Framework-Renode integration.

The Robot Framework-Renode integration layer provides keywords allowing the user to control and inspect the state of the simulation directly from the Robot Framework test file in a similar manner to the built-in keywords. The [basic keywords](#) allow the user to:

- start the emulation (`Start Emulation`),
- clear the emulation (`Reset Emulation`),
- execute a command in the Monitor (`Execute Command`),
- allocate a file in the Renode temporary folder (`Allocate Temporary File`),
- download a file to the Renode temporary folder (`Download File`),

and more (see the source code for details).

Additionally, Renode provides [a set of keywords for interacting with UART devices](#). They allow:

- writing a text to UART (`Send Key To Uart`, `Write Char On Uart`, `Write Line To Uart`),
- waiting for a specific line to appear on UART (`Wait For Line On Uart`, `Wait For Prompt On Uart`),
- waiting for any output on UART (`Wait For Next Line On Uart`),
- waiting for the lack of output on UART (`Test If Uart Is Idle`).

There is also [a set of keywords for interacting with network devices](#) that allow:

- waiting for the next outgoing network packet (`Wait For Outgoing Packet`),
- waiting for a specific outgoing network packet (`Wait For Outgoing Packet With Bytes At Index`).

It is possible to extend the Renode-Robot Framework interface by implementing more keywords in C# if necessary.

For reference on how to use the keywords mentioned in this section, see the robot test files that Renode comes with.

7.4 Advanced usage

7.4.1 Test-case dependencies

Normally, test cases in a single `.robot` file should be independent of each other. In fact, the default test teardown keyword calls `Reset Emulation` to ensure that the state of one test does not affect the next.

However, there are situations where it may be desirable to split a long-running test scenario into multiple test cases. This allows for better reporting of execution progress and improves overall performance (compared to re-running a common part of the test).

Renode provides custom Robot keywords to annotate situations where a test should continue execution from a state provided by another test:

- `Provides` - creates a named snapshot of the simulation state (see *State saving and loading*),

- Requires - loads a named snapshot and resumes execution of the test from that state.

An example of usage:

```
*** Test Cases ***
Boot Linux
    [...]
    Provides          booted-linux

Write to flash
    Requires          booted-linux
    Write Line To Uart ...
    [...]

Ping another node
    Requires          booted-linux
    Write Line To Uart ...
    [...]
```

See [Zedboard.robot](#) for a real-life example.

Note

There is also an alternative implementation of the mechanism available, where instead of state snapshots we use keywords recording. In this case when the Requires keyword is used, all the keywords executed before the corresponding Provides are re-executed. To use this mode, pass Reexecution as an additional parameter when using the Provides keyword:

```
*** Test Cases ***
Boot Linux
    [...]
    Provides          booted-linux          Reexecution
```

7.4.2 Running many test files with a single command

The example in the previous section showed how to run a single test file (which may still contain many test cases). It is possible to run many test files and aggregate the results into a single report. To do this, you need to pass many test files as arguments to the renode-test command:

```
renode-test my_tests.robot additional_tests.robot extra_tests.robot
```

The tests will be executed in the order in which the arguments were provided.

An alternative is to prepare a yaml file with the list of tests to run, e.g:

```
- my_tests.robot
- additional_tests.robot
- extra_tests.robot
```

and call renode-test with a special switch:

```
$ renode-test -t my_tests.yaml
```

Note

The `.yaml` notation allows the user to include other `.yaml` files and to group items that should not be executed in parallel (see the next section).

7.4.3 Running tests in parallel

Test cases from a single file are always executed serially (in the order defined in the file), but it's possible to run tests from different files in parallel. To do this, run the `renode-test` command with a special switch:

```
$ renode-test -j12 -t my_tests.yaml
```

This will allow you to run up to 12 Renode instances, each running test cases from a different file. Using the `.yaml` file allows you to group items that should not be run in parallel (e.g. because they share a resource such as a port number):

```
- my_tests.robot
- my_group:
  - my_test2.robot
  - my_test3.robot
```

In the above example, `my_test2.robot` is executed before `my_test3.robot`, but in parallel with `my_tests.robot`.

You can also pass many test files as arguments (i.e. without the `.yaml` file), but you won't be able to do the grouping:

```
$ renode-test -j3 my_tests.robot my_tests2.robot my_tests3.robot
```

7.4.4 Stopping on error

By default, `renode-test` will run all the provided test cases. However, it is possible to stop execution on the first error encountered. To do this, run the `renode-test` script with

```
renode-test --stop-on-error my_tests.robot
```

7.4.5 Running multiple instances of `renode-test` at once

Renode communicates with the Robot Framework executor via a network socket. This means that running two test-renode instances at the same time will result in a network port conflict.

To avoid this, you can explicitly specify the port number to be used for communication between the Robot Framework and Renode:

```
$ renode-test -P 9997 my_test.robot &
$ renode-test -P 9998 my_test2.robot &
```

7.4.6 Repeating tests

It is possible to run the specified tests multiple times using

```
$ renode-test -n 10 my_test.robot
```

This will re-run all test cases from `my_tests.robot` 10 times.

7.4.7 Running selected fixtures

It is possible to run only selected test cases from the file using:

```
$ renode-test -f "*GDB*" my_tests.robot
```

In the above example, only test cases with GDB in their name will be executed.

7.4.8 Running tests interactively

By default, the `renode-test` command runs tests in the background and only reports results to the console. However, it is possible to enable printing of log messages to the console in the same way as when running the `renode` command:

```
$ renode-test --show-log my_tests.robot
```

Note

This will cause the test progress messages to be mixed in with the log messages.

You can also show and interact with the monitor and analyzer windows:

```
$ renode-test --enable-xwt my_tests.robot
```

Note

Interacting with the running test may affect the results.

7.4.9 Saving state of failed tests

Renode's testing framework allows the automatic creation of snapshots of failed tests in order to load them later to inspect the state of the simulation and/or run them further. This feature is especially helpful in non-interactive CI environments.

To enable the automatic creation of failed test snapshots, set the `RENODE_CI_MODE` environment variable before running the `renode-test` command:

```
$ RENODE_CI_MODE=YES renode-test my_test.robot
```

Each time the snapshot is created, it will be given a name corresponding to the failed test and you will see the message in the console informing you about the path to it. All snapshots are stored in the `output/tests/snapshots` directory.

Note

Enabling the CI mode will also influence the way external resources are handled - the binaries cache will be disabled, so each external file will be downloaded every time it's referenced.

7.4.10 Inspecting failed tests interactively

With Renode, it is possible to stop the execution of the test suite in order to interactively debug a failed test case using the standard Renode interface (monitor, UART analyzers, etc).

To enable this feature, run the `renode-test` command with the following switch:

```
$ renode-test --debug-on-error my_test.robot
```

This will cause the execution of the test suite to stop on error, display the Renode Monitor and peripheral analyzers, and allow the user to inspect the state of the simulation. Once the interactive session is complete, it's possible to resume test execution by pressing a button in a command prompt window.

Note

This feature is not currently available in headless environments.

DEVELOPING RENODE

Renode has many built-in features to enable debugging of embedded software, e.g. *debugging with GDB*, but sometimes you might be interested in debugging Renode itself, especially if you are involved in its development

8.1 VS Code configurations

Visual Studio Code's extensive plugin ecosystem provides a good developer experience for working with the Renode codebase. You may decide to use either the official **VS Code** app from Microsoft or use one of the OSS binary releases **VSCodium** or **code-server**. The main difference between the official and OSS releases is that they use different extensions gallery, so some extensions may be unavailable for each version.

When launching Renode in VS Code, you can use several ready-to-use configurations (as defined in the **launch.json** file):

- Launch (console) - launches a previously built Renode under the .NET debugger.
- Attach - attaches the .NET debugger to a running Renode instance.
- (gdb) Tlib Attach - required to connect to a previously launched instance of Renode via GDB to debug translation libraries (implementation of emulated cores).

The workspace also provides build tasks (as defined in the **tasks.json** file):

- Build - Debug - equivalent to running `./build.sh -d` from the console.
- Build - Release - equivalent to running `./build.sh` from the console.

These configurations require the following VS Code extensions:

- **ms-dotnettools.csdevkit**
- **ms-vscode.cpptools**

8.1.1 Debugging C# code

By following the steps below, you will be able to add regular breakpoints in Renode's C# code.

1. Build Renode in debug configuration (or use the Build - Debug task in VS Code).

```
./build.sh -d
```

2. Use the Launch (console) configuration.
3. When Renode starts, you can set breakpoints and debug C# code as usual.

8.1.2 Debugging C code (translation libraries)

To debug the C implementation of emulated cores, follow the steps below.

1. Build Renode in debug configuration (or use the Build - Debug task in VS Code).

```
./build.sh -d
```

2. Launch Renode with the debug flag enabled.

```
./renode -d
```

3. When Renode starts, load the platform with a CPU whose code you wish to debug.
4. Use the (gdb) Tlib Attach configuration to connect with GDB.
5. Once started, a popup will appear. In the popup, type in Renode.dll.
6. From the drop-down list, select the option ending with Renode.dll, as highlighted in the screenshot below.

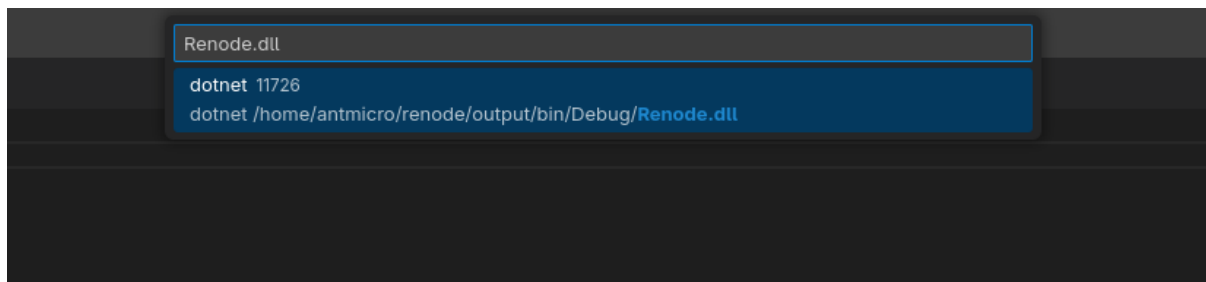


Figure 8.1: VSCode .NET drop-down list

8.2 Debugging with GDB

To start debugging Renode and its components using **GDB**, follow these steps:

1. Build Renode in debug configuration.

```
./build.sh -d
```

2. Launch Renode with the debug flag enabled.

```
./renode -d
```

3. Attach GDB to a running Renode process to debug implementation of cores (in C via GDB).

8.2.1 Useful VS Code extensions

Below, you can find a list of VS Code extensions that may prove useful when working with Renode:

- Description
 - [Open VSX Registry](#)
 - [Visual Studio Marketplace](#)

- C# support (code completion and debugging)
 - [muhammad-sammy.csharp](#)
 - [ms-dotnettools.csdevkit](#)
- C support (code completion and debugging)
 - [llvm-vs-code-extensions.vscode-clangd](#) and [vadimcn.vscode-lldb](#)
 - [ms-vscode.cpptools](#)
- CMake support (for standalone Tlib building)
 - [ms-vscode.cmake-tools](#) on Open VSX
 - [ms-vscode.cmake-tools](#) on VS Marketplace
- Python support
 - [ms-python.python](#) on Open VSX
 - [ms-python.python](#) on VS Marketplace
- Robot Framework language server
 - [robocorp.robotframework-lsp](#) on Open VSX
 - [robocorp.robotframework-lsp](#) on VS Marketplace
- Debugging embedded targets with Renode's GDB server
 - [marus25.cortex-debug](#) on Open VSX and [webfreak.debug](#) on Open VSX
 - [marus25.cortex-debug](#) on VS Marketplace and [webfreak.debug](#) on VS Marketplace

Note

Some of these extensions may require additional configuration depending on your machine's setup. **Tlib** should be first built with CMake to generate the `compile_commands.json` file for clangd language services.

WORKING WITH MACHINES

Renode allows to easily handle emulations spanning multiple machines.

9.1 Creating machines

In the beginning, the emulation is empty as there is no machine to run. In order to add an empty one, execute:

```
(monitor) mach create  
(machine-0)
```

This creates the first machine, which, if you don't give it a custom name, will be indexed from 0 (and thus called machine-0). This command also switches the Monitor's context to this new machine.

Executing the same command again will create another machine, called machine-1:

```
(machine-0) mach create  
(machine-1)
```

You can create a machine with a custom name by providing it as the argument:

```
(monitor) mach create "my-machine"  
(my-machine)
```

To list all created machines together with their names and indices, type:

```
(my-machine) help mach
```

9.2 Switching between machines

When you want to switch the Monitor's context to another machine type:

```
(machine-1) mach set "machine-0"  
(machine-0)
```

Instead of the machine's name, you can use its index as well:

```
(machine-1) mach set 0  
(machine-0)
```

9.3 Loading platforms

Once a machine is created, it contains only one peripheral - the *system bus* called simply *sysbus*. There is no memory or CPU, so the machine is not yet ready to execute any code.

To list all peripherals, execute:

```
(machine-0) peripherals
Available peripherals:

sysbus (SystemBus)
```

To load a predefined platform (in this example, *Microsemi MiV*), type:

```
(machine-0) machine LoadPlatformDescription @platforms/cpus/miv.repl
(machine-0) peripherals
Available peripherals:
sysbus (SystemBus)

├── clint (CoreLevelInterruptor)
│   └── <0x44000000, 0x4400FFFF>
├── cpu (RiscV)
│   └── Slot: 0
├── ddr (MappedMemory)
│   └── <0x80000000, 0x83FFFFFF>
├── flash (MappedMemory)
│   └── <0x60000000, 0x6003FFFF>
├── gpioInputs (MiV_CoreGPIO)
│   └── <0x70002000, 0x700020A3>
├── gpioOutputs (MiV_CoreGPIO)
│   └── <0x70005000, 0x700050A3>
├── plic (PlatformLevelInterruptController)
│   └── <0x40000000, 0x43FFFFFF>
├── timer0 (MiV_CoreTimer)
│   └── <0x70003000, 0x7000301B>
├── timer1 (MiV_CoreTimer)
│   └── <0x70004000, 0x7000401B>
└── uart (MiV_CoreUART)
    └── <0x70001000, 0x70001017>
```

Output of the peripherals command can be filtered by peripheral name, peripheral model name, or address range.

Filtering by name:

```
(machine-0) peripherals uart
Filtered peripherals:
sysbus (SystemBus)
├── uart (MiV_CoreUART)
│   <0x70001000, 0x70001017>
```

Filtering by model name:

```
(machine-0) peripherals CoreTimer
Filtered peripherals:
sysbus (SystemBus)
├── timer0 (MiV_CoreTimer)
│   <0x70003000, 0x7000301B>
├── timer1 (MiV_CoreTimer)
│   <0x70004000, 0x7000401B>
```

In case of filtering by name and model name the search string can be partial, like with CoreTimer.

Filtering by address range:

```
(machine-0) peripherals <0x70000000, 0x7FFFFFFF>
Filtered peripherals:
sysbus (SystemBus)
├── gpioInputs (MiV_CoreGPIO)
│   <0x70002000, 0x700020A3>
├── gpioOutputs (MiV_CoreGPIO)
│   <0x70005000, 0x700050A3>
├── timer0 (MiV_CoreTimer)
│   <0x70003000, 0x7000301B>
├── timer1 (MiV_CoreTimer)
│   <0x70004000, 0x7000401B>
├── uart (MiV_CoreUART)
│   <0x70001000, 0x70001017>
```

9.4 Accessing and manipulating peripherals

When you are in the context of a machine in the Monitor you can reference peripherals by name. You can read and write a peripheral's properties as well as execute some actions on them. The set of available properties and operations depends on the type of peripheral.

For example, to check memory size, execute:

```
(machine-0) sysbus.ddd Size
```

To call an action on the peripheral, use the same syntax but replace Size with the action name, e.g., ZeroAll:

```
(machine-0) sysbus.ddd ZeroAll
```

To get the complete list of available properties or actions, just enter the peripheral's name:

```
(machine-0) sysbus.ddd
The following methods are available:
- Void DebugLog (String message)
- Void Dispose ()
[...]
- Void WriteWordUsingDwordBigEndian (Int64 address, UInt16 value)
- Void ZeroAll ()
Usage:
sysbus.ddd MethodName param1 param2 ...
The following properties are available:
- Int32 SegmentCount
    available for 'get'
- Int32 SegmentSize
    available for 'get'
- Int64 Size
    available for 'get'
Usage:
- get: sysbus.ddd PropertyName
- set: sysbus.ddd PropertyName Value
```

The Usage sections describe the proper syntax for accessing the peripheral's features.

9.5 Loading binaries

Once the platform is created and configured, you can upload the software onto it. Renode allows you to run exactly the same executable as on the real hardware, which means there is no need to alter the binary or recompile the source.

In Renode, you can use local binaries or load them via HTTPS. If you do not have a binary at your disposal, you can use this [sample Zephyr Shell sample for MiV](#).

To load a local .elf file into memory, execute:

```
(machine-0) sysbus LoadELF @my-project.elf
```

To load a binary via HTTPS:

```
(machine-0) sysbus LoadELF @https://remote-server.com/my-project.elf
```

Note

If you load multiple ELF files, Renode will look for the lowest loaded section of the last ELF

file to locate the initial PC (or vector table offset on Cortex-M machines). That means the last ELF you load will set up your CPU starting point.

To override this, you can set the initial values manually, using:

```
sysbus.cpu PC 0xYourValue
```

or, on Cortex-M:

```
sysbus.cpu VectorTableOffset 0xYourValue
```

Renode supports other executable formats like raw binary, UIImage and HEX as well. To load them, use LoadBinary, LoadUIImage or LoadHEX accordingly.

9.6 Clearing the emulation

If you want to switch to another project, you can drop the whole emulation::

```
(machine-0) Clear
```

All machines, peripherals, and loaded binaries will be removed, and Renode will return to its initial state.

DESCRIBING PLATFORMS

Renode uses a text-based format to describe platforms. Platform description files typically have the `.repl` extension, but this is not a requirement.

The broad description of the format and its grammar is available in the *platform-description-format* section. Here we present the basic usage and most common scenarios.

10.1 Defining peripherals

To add a peripheral, you need to know its type, choose its name and the registration point. Most peripherals will be registered on the sysbus - a peripheral that is always available and does not have to be explicitly defined.

The type name has to indicate the class of the peripheral model. This has to be a full name with a namespace, but the default namespace, `Antmicro.Renode.Peripherals`, can be omitted.

For example, to create a UART object of type `Antmicro.Renode.Peripherals.UART.MiV_CoreUART`, connected to the system bus at `0x80000000`, use:

```
uart0: UART.MiV_CoreUART @ sysbus 0x80000000
```

Some peripherals, like the mentioned UART, need parameters to be constructed. The REPL format allows you to set the constructor parameters and properties of the peripheral model. They are placed below the declaration, with four spaces of indentation:

```
uart0: UART.MiV_CoreUART @ sysbus 0x80000000
      clockFrequency: 66000000
```

Constructor parameters begin with a lower case letter, and properties with an upper case letter.

10.2 Connecting peripherals

In the example above the `uart0` peripheral was connected to the system bus at a specific address. It is possible, however, to connect peripherals to other buses as well, like I2C or SPI, to a GPIO controller, etc.

For example, to connect a temperature sensor to an I2C controller called `i2c0` at `0x80`, type:

```
sensor: Sensors.SI70xx @ i2c0 0x80
```

Peripherals can also be connected via GPIOs or interrupts. Renode treats these signals similarly, and allows you to create a connection with the `->` operator.

To connect a timer to the 31-st interrupt on the plic interrupt controller, run:

```
timer: Timers.MiV_CoreTimer @ sysbus 0x1000000  
-> plic @ 31
```

10.3 Including files

You can include an existing REPL file in your platform with the using keyword.

The path will be looked up in each of the directories on the Monitor's internal PATH, which is editable with the path command. By default, the PATH contains the Renode installation directory, so the platform files distributed with Renode can be included like so:

```
using "platforms/cpus/miv.repl"
```

You can also provide an absolute path:

```
using "/tmp/platform.repl"
```

Or a path relative to the REPL file that contains the using statement:

```
using "../platform.repl"  
using "../other/platform.repl"
```

Note

On Windows, / can be used as a path separator in all of these cases. If you want to use Windows-style \ separators, you will need to escape them:

```
using "D:\\platforms\\platform1.repl"
```

MONITOR AND SCRIPT SYNTAX

The Monitor is Renode's command line interface (CLI). It lets you talk to Renode and control the emulation using a wide range of built-in functions. They allow you to access emulation objects like peripherals, machines, and external connectors.

Note

If you want to learn more about the Monitor and its basic usage, visit the *Using Renode* chapter.

Generally, commands in Renode are executed line-by-line, and the arguments need to be separated by a space character. You can further extend the range of available functions with Python (see [monitor.py](#) for examples and *Using Python in Renode* for instructions how to use Python to extend Renode).

11.1 Using built-in commands

Renode comes with dozens of built-in commands. You can access the full list of commands with their descriptions using the help command in the Monitor. To get more detailed information about a certain command, use `help <command>`, e.g.:

```
help start
```

11.2 Accessing emulation objects

To get access to commands available to your machine's emulation objects (e.g., peripherals), you can type the object's name and click Tab twice to activate tab completion.

Emulation objects are in a hierarchy, with sysbus being a machine's root for all peripherals. If you want to access a UART peripheral of your machine called uart, you need to use sysbus.uart. You can use using command to set a default prefix, e.g. using sysbus allows you to refer to the UART directly:

```
uart
```

You will find that the using sysbus command is used in most of the scripts bundled with Renode.

Passing the name of the emulation object as a command returns the full list of available methods, properties, and value fields for this object. In this list, you can find information on whether

the method returns a value, what data types it accepts, and how to use it:

```
(machine-0) uart
The following methods are available:
- Void AddLineHook (String contains, String pythonScript)
- Void CloseFileBackend (String path)
- Void CreateFileBackend (String path, Boolean immediateFlush = False)
- Void DebugLog (String message)
- String DumpHistoryBuffer (Int32 limit = 0)
- Endianness GetEndianness (Endianness? defaultEndianness = null)
- IEnumerable<tuple<string,< span="">IGPIO>> GetGPIOs ()</tuple<string,<
...
Usage:
sysbus.uart MethodName param1 param2 ...
The following properties are available:
- UInt32 BaudRate
    available for 'get'
- Parity ParityBit
    available for 'get'
- Int64 Size
    available for 'get'
- Bits StopBits
    available for 'get'
Usage:
- get: sysbus.uart PropertyName
- set: sysbus.uart PropertyName Value
```

To inspect the parameters of a specific method available to an object, simply provide the name of the object and the method:

```
(machine-0) uart WriteWordUsingByte
The following methods are available:
- Void WriteWordUsingByte (Int64 address, UInt16 value)
Usage:
sysbus.uart MethodName param1 param2 ...
```

Keep in mind that if a method does not require any parameters, providing its name will invoke it:

```
(machine-0) machine GetTimeSourceInfo
Elapsed Virtual Time: 00:00:00.000000
Elapsed Host Time: 00:00:00.000000
Current load: NaN
...
```

Many commands require you to specify a parameter after the command when using them:

```
using sysbus
```

11.3 Accessing attributes of an object

Objects in Renode have access to different methods, properties, fields, and indexers depending on their type. The methods are required to access the parameters of a peripheral, and to do it successfully, you need to follow this syntax:

```
(machine) registrationPoint.peripheral MethodName param1 param2
```

To read the value of a byte at offset 0 from the uart peripheral registered on the sysbus you would type:

```
(machine-0) sysbus.uart ReadByte 0
```

Renode enables you to both get and set values for the object's attributes. If you do not specify a value at the end of your command, the current value will be returned. If you want to set a value, you need to use the proper command and value at the end.

For example, to set the CyclesPerInstruction property to 0x000002, you would use:

```
(machine-0) cpu CyclesPerInstruction 0x000002
```

To get the value of this newly set property, you have to use:

```
(Mi-V) cpu CyclesPerInstruction  
0x00000002
```

Setting indexers is very similar to setting properties, with the main difference being that you have to put your parameters in the square brackets:

```
(machine-0) machine IndexerName [param1 param2 ...] Value
```

To get the value of the indexer, use:

```
(machine-0) machine IndexerName [param1 param2 ...]
```

The last type of emulation object attribute is a field. Their values can be accessed the same as you access properties:

```
(machine-0) machine SystemBusName  
sysbus
```

11.4 Command substitution with backticks

The Monitor allows you to use the output of one command as an argument to another. This can be achieved by enclosing a command in backticks (`), similarly to how it's done in Unix-like shells.

The command inside the backticks is executed first, and its output is then used as an argument in the outer command. For example, you can find the symbol at the current program counter (PC) of the CPU by running:

```
(machine-0) sysbus FindSymbolAt `cpu PC`
```

In this case, `cpu PC` is executed first, and its output (the current value of the program counter) is then passed to the `sysbus FindSymbolAt` command.

11.5 Operating on enumerable properties

Some peripherals and properties are enumerable, which means they can contain multiple elements. A good example of this is a CPU cluster, which consists of several cores. You can operate on all elements of such a collection at once using the `ForEach` and `Select` commands.

The `ForEach` command allows you to execute an action on each element of a collection. For example, to halt all cores in `cluster0`, you can run:

```
(machine-0) cluster0 ForEach IsHalted true
```

The `Select` command, on the other hand, allows you to transform each element of a collection. For example, to get the value of the `X0` register on each core in a cluster:

```
(machine-0) cluster0 Select GetRegister "X0"
[
0xaaac4db0770, 0xffff795b42148000, 0x5, 0x5,
]
```

Or, to get the `SFDPSignature` of a flash memory as a list of decimal strings, you can use:

```
(machine-0) qspi.spiFlash SFDPSignature Select ToString "d"
[
83, 70, 68, 80, 6, 1, 0, 255, 255, 255,
255, 255, 255, 255, 255, 255,
]
```

11.6 Supported data types

The Monitor supports a wide variety of data types:

Table 11.1: Data types supported by the Monitor

	Name of the data type	
	comment	
	integer	
	float	
	hexadecimal	
	strings	
	range	
	relative range	
	index	
	path	
	logical values	
	variables and macros	
	enum	

Note

You can pass the path parameter to a function that accepts strings and it will be converted to a string. Keep in mind that the additional filename autocompletion is only available after an @ sign.

11.7 Monitor variable types

There are three ways you can create a single line variable in the Monitor:

- `$var="hello",`
- `$var?="hello",`
- `set var "hello".`

The difference between `$var=` and `$var?=` is that the latter indicates a default value if the variable is not set. The Monitor also enables you to create multiline variables using the following syntax:

```
set var ""
"hello"
""
```

Note

When loading a script, you can use slightly different syntax for multiline variables, with the `""` mark put below the first line:

```
set var
""
"hello"
""
```

Variables in Renode are contextual, which means that for each machine you can have different variables with the same name. You can access them by their full path. To create a variable within the machine context of `machine-0`, you would use:

```
(machine-0) $machine_0.var
```

You can also create global variables if you use the global prefix:

```
(monitor) $global.CWD
```

Most of the time, a short name (without the context prefix) should be enough to access variables.

The Monitor gives you access to two special variables: `$ORIGIN` and `$CWD`.

Other types of variables available in Renode are macros, which enable you to encapsulate fragments of your script and execute them easily. You can set a new macro using the `macro` command with the name of the new variable and the commands you want it to execute:

```
macro newMacro  
> sysbus LoadELF $bin
```

You can create multiline macros using the Monitor multiline syntax. You can then execute your newly created macro using the `runMacro` command:

```
runMacro $newMacro
```

You can notice that many Renode sample scripts define the `$reset` macro. This macro is used whenever the `machine Reset` method is called by the user or via the simulation logic.

11.8 File paths

Each Renode project will require you to supply the file path to the components necessary for your project. The vast majority of them use the `@` sign, which activates autocompletion suggestions and represents a path to a file:

```
include @/path/to/platform.repl
```

When interpreting a path, Renode looks in several places based on the configured internal path. By default:

- it first checks in the Renode root directory,
- if the file was not found in the root directory, it checks the current working directory.

You can check and modify the path configuration using the `path` command in Monitor.

You can also pass paths as `"string"`, but completion suggestions will not work in that case.

Note

In Renode, you can use paths that are absolute or relative to the current directory.

11.8.1 Relative paths

If you want to express a path that is relative to the currently executed Renode script (`.resc`) you can use the `$ORIGIN` variable:

```
include $ORIGIN/my_subscript.resc
```

An example of usage can be found in [the fomu script](#).

Note

Do not use `@` at the beginning of an `$ORIGIN`-based path.

Keep in mind that the `$ORIGIN` variable is only available inside a script - it won't work interactively in Monitor.

In Monitor, you can use a special `$CWD` variable to provide a path that is relative to the current working directory:

```
(machine-0) include $CWD/my_script.resc
```

Note

There is no @ at the beginning of the \$CWD-based path.

Note

In a Robot file, you can also use another variable: \${CURDIR}. It is handled and resolved on the Robot Framework level and has nothing to do with Renode. Paths starting with \${CURDIR} are relative to the Robot file location.

An example of usage can be found in [the LSM9DS1 test](#).

A \${CURDIR}-based paths need to be prepended with @. Since it is resolved at the Robot Framework level, for Renode, it looks like any other path provided by a user.

If you want to learn more about testing with Renode, visit *chapter devoted to this topic*.

11.9 Renode Script syntax

Many of your projects in Renode will involve using the same platforms and commands multiple times. This process can be accelerated significantly using .resc files, which are Renode scripts. The syntax of .resc files is the same as that of the Monitor.

To load a Renode script, use the include or i command:

```
include @/path/to/script.resc
```

Note

You can use the start command instead of include to start your emulation immediately.

The syntax of a .resc file can be exemplified using one of many scripts built into Renode, like [Nordic Semiconductor NRF52840 script](#), which we will cover line by line:

```
:name: nRF52840
:description: This script runs Zephyr Shell demo on NRF52840.

using sysbus

mach create
machine LoadPlatformDescription @platforms/cpus/nrf52840.repl

$bin?=@https://dl.antmicro.com/projects/renode/renode-nrf52840-zephyr_shell_
↪module.elf-gf8d05cf-s_1310072-c00fbffd6b65c6238877c4fe52e8228c2a38bf1f

showAnalyzer uart0
```

(continues on next page)

(continued from previous page)

```
macro reset
"""
    sysbus LoadELF $bin
"""
runMacro $reset
```

The first two lines are comments, and they are not interpreted by the Monitor:

```
:name: nRF52840
:description: This script runs Zephyr Shell demo on NRF52840.
```

Next, we utilize the `using` command to omit a prefix when referring to a peripheral. The command using `sysbus` enables you to refer to the CPU device with `cpu` instead of `sysbus.cpu`:

```
using sysbus
```

Then, we create a new machine

```
mach create
```

We load a platform description from the `.repl` file:

```
machine LoadPlatformDescription @platforms/cpus/nrf52840.repl
```

Now, we load a sample shell binary:

```
$bin?=@https://dl.antmicro.com/projects/renode/renode-nrf52840-zephyr_shell_
↪module.elf-gf8d05cf-s_1310072-c00fbffd6b65c6238877c4fe52e8228c2a38bf1f
```

Next, we set up an UART analyzer for `uart0`:

```
showAnalyzer uart0
```

The next element of our script is a macro, which is executed every time the machine is reset.

To create a macro in your `.resc` you need to surround your macro's code with `"""` (3 double quotation marks). The contents of your macro do not need to be indented with four spaces, but we often do it for readability.

The macro below loads the previously specified ELF file:

```
sysbus LoadELF $bin
```

Lastly, we call the macro to run it, as it is only executed after it is invoked:

```
runMacro $reset
```

RENODE TESTING API

For an introduction to testing with Renode, visit the [Testing with Renode](#) section of our documentation.

12.1 General methods

We have many methods of accessing a simulation state you can use for testing. Some of them are Renode commands that you can execute in the robot framework as follows:

```
Execute Command      <renode-command>
```

You can use the following template to extract data out of a simulation:

```
${value}= Execute Command      <renode-command>
```

To check against conditions with robot keywords, run the commands below:

```
Should Be Equal      ${value}  <expected-value>
Should Be Equal As Numbers  ${value}  <expected-value>
Should Be Equal As Integers  ${value}  <expected-value>
```

12.1.1 Accessing memory

To access memory at a given address, you can use Renode Read<Width> commands as shown below. They enable you to get both memory and peripheral registers content.

```
${value}= Execute Command      sysbus ReadByte      <address>
${value}= Execute Command      sysbus ReadWord      <address>
${value}= Execute Command      sysbus ReadDoubleWord  <address>
${value}= Execute Command      sysbus ReadQuadWord   <address>
```

12.1.2 Accessing CPU registers

To access a CPU register, use the following command:

```
${val}= Execute Command      cpu GetRegister <register-name>
```

Alternatively, if you only want to compare a register against an expected value, use:

```
Register Should Be Equal      <register-name> <expected-value>
```

See the [example test case](#) for usage.

12.1.3 Accessing simulation logs

To access simulation logs, first create a log tester:

```
Create Log Tester      <timeout>
```

Then, to verify if an expected log entry appears in the logs, run:

```
Wait For Log Entry      <expected-log>
```

For more details, see the [Log Tester Keywords implementation](#) and the [example test case](#).

12.1.4 Hooks at symbols

Renode enables you to create a hook on a symbol or an address in the form of a Python-scripted breakpoint.

A simple example of a hook is an addition of a log through the following command:

```
Execute Command      cpu AddHook `sysbus GetSymbolAddress "timerIRQ"`  
↪ "cpu.ErrorLog('timer interrupt @ {0}', monitor.Machine.ElapsedVirtualTime.  
↪ TimeElapsed)"
```

You can then assert the hook execution, and therefore symbol execution, as follows:

```
Wait For Log Entry      timer interrupt
```

See the [example test case](#) for usage.

12.2 UART

To verify if a UART outputs correct data, first create a terminal tester and attach it to a selected peripheral:

```
Create Terminal Tester      sysbus.uart
```

Then, use one of the following commands to wait for the expected output:

- Wait until a given line is sent to the UART:

```
Wait For Line On Uart      Expected Line
```

- Wait until a given prompt (a line without a newline character) is sent to the UART:

```
Wait For Prompt On Uart      Expected Prompt
```

- Wait until a given sequence of bytes is sent to the UART:

```
Wait For Bytes On Uart      59 52 20 4f 53
```

You can also interfere with the simulation by sending data to the UART with one of the following commands:

- Send a single character to the UART:

Send Key To Uart 0xD

- Send a line to the UART:

Write Line To Uart Example Line

For a full list of commands and parameters, see the [Uart Tester Keywords implementation](#).

The UART testing can also be scripted in Python with [Virtual Console](#) or automated with file-based feeding using the [UART RESD Feeder](#).

12.3 GPIO

To verify a GPIO interface, use LEDs and buttons, both of which need to be created first. The easiest way to do it is by using REPL strings. Use the snippet below to connect a LED to the 100th pin of the gpio peripheral:

```

${LED_REPL}          SEPARATOR=\n
...                  """
...                  led: Miscellaneous.LED @ gpio 0
...
...                  gpio:
...                  ${SPACE*4}100 -> led@0
...                  """
...

```

The snippet below connects a button to the 7th pin of the gpio peripheral:

```

${BUTTON_REPL}       SEPARATOR=\n
...                  """
...                  button: Miscellaneous.Button @ gpio 1
...                  ${SPACE*4}-> gpio@7
...                  """
...

```

Then, load those snippets in a test case through the following commands:

```

Execute Command      machine LoadPlatformDescriptionFromString ${LED_
↪REPL}
Execute Command      machine LoadPlatformDescriptionFromString $
↪${BUTTON_REPL}

```

To check the GPIO state on the pin on which you created the LED, first create a LED tester:

Create Led Tester sysbus.gpio.led

Then, execute one of the following commands:

- Check if the GPIO pin matches the expected state at some point during the entire <timeout> period. If <timeout> equals 0, the check is performed immediately and fails if the pin state is incorrect.

Assert Led State <False/True> timeout=<timeout>

- Check if the GPIO pin matches the expected state during <timeout>, and if the states lasted for a given period.

```
Assert And Hold LED State true timeoutAssert=0.1 timeoutHold=2
```

- Check if a LED is blinking in a pattern determined by given parameters.

```
Assert Led Is Blinking testDuration=0.05 onDuration=0.005 offDuration=0.
↪005 tolerance=0.2
```

To set a GPIO input through a button, use one of the following options:

- Use the Press and Release button commands to set the GPIO pin to 1 and 0, respectively. Alternatively, you can use the Toggle command to change the current state to an opposite one.

```
Execute Command gpio.button Press
Execute Command gpio.button Release
Execute Command gpio.button Toggle
```

- Use the PressAndRelease button command to change the button state to 1 and then immediately to 0.

```
Execute Command gpio.button PressAndRelease
```

For more details, see the [Led Tester Keywords implementation](#), [Button implementation](#) and the [example test case](#).

12.4 Network interfaces

To test a network interface (either a radio or Ethernet), first create a network tester:

```
Create Network Interface Tester sysbus.ethernet
```

Then, you can verify network traffic with the command below. Here, the parameters are responsible for an expected bytes pattern (with the _ wildcard), for an index of the first byte in a packet that needs to be checked, and for the number of packets that need to be checked before failing.

```
Wait For Outgoing Packet With Bytes At Index 0800_____11____
↪C0000201E0000181013F013F_____0012 12 5 10
```

Use the command below to run a simulation until a packet is received or a timeout is triggered:

```
Wait For Outgoing Packet <timeout>
```

To send a packet to an interface, use the following command:

```
Send Frame <bytes>
```

For more details, see the [Network Interface Keywords implementation](#) and the [example test case](#).

12.5 Verifying other interfaces

To test communication through the I2C or SPI without a target device, we use dummy target devices with some enqueued data to respond with.

A verification is typically based on software behavior, e.g. matching a line on a UART.

For more complex scenarios, you can create an ad-hoc compiled I2C or SPI target device with verification implemented in C#.

12.5.1 I2C

First, create a DummyI2CSlave:

```
Execute Command      machine LoadPlatformDescriptionFromString "dummy_i2c:~
↳Mocks.DummyI2CSlave @ lpi2c0 42"
```

The DummyI2CSlave peripheral is designed to be interfaced via commands but can also be scripted through IronPython integration.

You can find an example of a Python-scripted I2C target device in [echo-i2c-peripheral.py](#).

More information on the Python-enabling features is available in the [Dummy I2C Slave](#) section of Renode documentation.

You can queue response data with the following command:

```
Execute Command      dummy_si2c EnqueueResponseByte 0x2a
```

See the [example test case](#) for usage.

12.5.2 SPI

First, create a DummySPISlave:

```
Execute Command      machine LoadPlatformDescriptionFromString "dummy_spi:~
↳Mocks.DummySPISlave @ lpspi0 42"
```

The dummy SPI supports only queuing response data. To do so use the following command:

```
Execute Command      dummy_spi EnqueueValue 0x2a
```

In addition to the verification based on software behavior, you can check the data sent against a log entry:

```
Wait For Log Entry    dummy_spi: Data received: 0xA
```

See the [example test case](#) for usage.

BASIC EXECUTION CONTROL

Renode allows you to precisely control the execution of the emulation.

13.1 Starting and pausing the execution

In the beginning the emulation is in a *paused* state, which means that no machine is running and *virtual time* is not progressing.

To start the emulation, execute:

```
(machine-0) start
Starting emulation...
```

To pause it, type:

```
(machine-0) pause
Pausing emulation...
```

13.2 Executing instruction-by-instruction

Note

Although using Renode's native stepping is a viable solution, we recommend using [GDB](#) to perform step-by-step execution. Using GDB with Renode emulated machines is described in detail in the *documentation chapter devoted to this issue*.

More information on how step execution in GDB can be found in the [GDB documentation](#).

When you need to analyze in detail how the execution of your binary influences the state of the hardware, you can switch to the *stepping* execution mode:

```
(machine-0) sysbus.cpu ExecutionMode SingleStepBlocking
```

This will stop the emulation after the execution of each instruction. In order to move to the next one, type:

```
(machine-0) sysbus.cpu Step
```

When you want to return to the *normal* execution mode, type:

```
(machine-0) sysbus.cpu ExecutionMode Continuous
```

More advanced control can be obtained by connecting an external GDB.

13.3 Blocking and non-blocking stepping

When you use `SingleStepBlocking` mode, the emulation won't progress between steps. This can cause problems with blocking the emulation when executing multiple cores instruction-by-instruction, in which case `SingleStepNonBlocking` mode is the preferred option. The drawback of the non-blocking mode is that the virtual time will progress between steps, which might introduce desynchronization and timeout-related issues.

The `Step` command will preserve the current mode when in instruction-by-instruction flow and use the default value (`SingleStepBlocking`) otherwise. This behavior can be overridden by selecting the non-blocking mode explicitly:

```
(machine-0) sysbus.cpu Step false
```

13.4 Inspecting the current location

Renode allows you to easily examine the current state of your application:

```
(machine-0) sysbus.cpu PC  
0xC01890A8
```

As a result, you will get the hexadecimal address of the instruction currently being executed.

If you want to know the name of the function that is currently being executed (assuming your binary has been compiled with the symbols inside) type:

```
(machine-0) sysbus FindSymbolAt `sysbus.cpu PC` # equivalent of 0xC01890A8  
uart_console_write
```

This will print the name of the symbol.

USING THE LOGGER

The first window that appears after starting Renode is dedicated to the logger.

There are many logging options you can use to improve your experience with the presented information.

14.1 Logging level

There are five available logging levels:

- NOISY (-1),
- DEBUG (0),
- INFO (1),
- WARNING (2),
- ERROR (3).

You can select which messages are logged using the `logLevel` command. Each emulation object can be configured separately. Also each logger backend (e.g. *log file*) can have its own configuration.

By default, messages from all levels but the NOISY level are logged.

To set the global log level to NOISY, type:

```
(machine-0) logLevel -1
```

To change the log level only for a selected peripheral (in this case - the UART device), type:

```
(machine-0) logLevel -1 sysbus.uart
```

Note

Increasing the number of logged messages may affect the performance of the emulation.

The current log level can be verified by running `logLevel` without parameters.

This is the output of this command after some configuration:

```
(machine-0) logLevel
Currently set levels:
Backend          | Emulation element          | Level
=====
console          :                               : DEBUG
                  : machine-0:sysbus.plic      : ERROR
                  : machine-0:sysbus.uart      : NOISY
-----
```

14.2 Logging to file

To analyze the output from a long-running emulation, it is often a good idea to redirect the log to a file.

To achieve that, use the `logFile` command:

```
(machine-0) logFile @some_file_name
```

This will not disable the console logger but will add a new sink, to be configured separately. From the performance point of view, depending on the scenario, it can be beneficial to increase the minimal console log level and keep the more detailed data in the log file.

To set the ERROR log level for a file backend, type:

```
(machine-0) logLevel 2 file
```

Peripherals can also have different log levels on different backends:

```
(machine-0) logLevel 1 file sysbus.uart
```

14.3 Logging access to peripherals

Apart from the standard logger configuration, you can enable logging of accesses to specific peripherals. This feature is enabled only for peripherals registered on a system bus.

To enable it, run:

```
(machine-0) sysbus LogPeripheralAccess sysbus.uart
```

Now, whenever the CPU tries to read or write to this peripheral, you will see a message similar to this one:

```
14:32:28.6083 [INFO] uart: ReadByte from 0x0 (TransmitData), returned 0x0.
```

To enable logging access to all peripherals, run:

```
(machine-0) sysbus LogAllPeripheralsAccess true
```

14.3.1 Creating a trace of the execution

It is possible to create a trace of every function executed by the binary:

```
(machine-0) sysbus.cpu LogFunctionNames true
```

As a result, the names of the functions will be printed to the log at INFO level:

```
17:05:23.8834 [INFO] cpu: Entering function kobject_uevent_env at 0xC014CD9C
17:05:23.8834 [INFO] cpu: Entering function dev_uevent_name (entry) at 0xC018FA5C
17:05:23.8834 [INFO] cpu: Entering function dev_uevent_name at 0xC018FA70
17:05:23.8834 [INFO] cpu: Entering function kobject_uevent_env at 0xC014CDA8
17:05:23.8835 [INFO] cpu: Entering function kobject_uevent_env at 0xC014CDB8
17:05:23.8835 [INFO] cpu: Entering function kmem_cache_alloc (entry) at 0xC0085610
17:05:23.8835 [INFO] cpu: Entering function kmem_cache_alloc at 0xC0085630
```

If you are interested only in a subset of functions, you can limit the results by providing space-separated names prefixes:

```
(machine-0) sysbus.cpu LogFunctionNames true "dev kobject"
```

You can also avoid logging subsequent duplicate function names by adding another true argument either instead of or after the optional function name prefixes:

```
(machine-0) sysbus.cpu LogFunctionNames true ["dev kobject"] true
```

Only these three lines would remain printed in the aforementioned example with both function name filtering and duplicate removal applied:

```
17:05:23.8834 [INFO] cpu: Entering function kobject_uevent_env at 0xC014CD9C
17:05:23.8834 [INFO] cpu: Entering function dev_uevent_name (entry) at 0xC018FA5C
17:05:23.8834 [INFO] cpu: Entering function kobject_uevent_env at 0xC014CDA8
```

14.4 Hushing excessive unhandled access logs

Renode, by default, informs you about unhandled accesses to memory ranges that are not covered by any model. You may see logs like this:

```
09:21:8.1960 [WARNING] sysbus: [cpu: 0x08001200] WriteDoubleWord to non_
↪existing peripheral at 0x400D0114, value 0xFFFFFFFF.
09:21:9.4538 [WARNING] sysbus: [cpu: 0x080012E6] ReadDoubleWord from non_
↪existing peripheral at 0x400D0118, returning 0x0.
```

These logs are there to inform you that your platform's description is not complete and if you observe issues with your simulation it might be one of the possible cases.

Very often these unhandled regions will not affect any important aspects of the execution and you might want to silence these logs.

While changing the *log level* to ERROR to hide warnings might be one option, it could be a too radical solution as you might want to continue seeing other warnings.

The best way to achieve fine-grained logging control in this case is with the `SilenceRange` feature. E.g. if you want to disable logging for addresses in the range between `0x80000` and

0x801000, run:

```
sysbus SilenceRange <0x80000 0x1000>
```

You can do it from the REPL level in the sysbus init section as well:

```
sysbus:
  init:
    SilenceRange <0x80000 0x1000>
```

STATE SAVING AND LOADING

Renode allows you to save the state of the emulation to a file.

Such a file can be transferred to another user and then loaded to fully recreate the original setup. No additional binaries or configuration files are required.

To save the emulation state to a file called `statefile.save`, run:

```
(monitor) Save @statefile.save
```

This file can be used with the Load command:

```
(monitor) Load @statefile.save
```

Or you can load it when starting Renode directly from CLI:

```
$ renode statefile.save
```

It is important to remember that a state file created on one version of Renode may not be compatible with another one.

Please note that loading the state file clears the current emulation, and is equivalent to:

```
(monitor) Clear  
(monitor) Load @statefile.save
```

Note

After the state is loaded, you must manually set the Monitor's context and reopen the UART windows:

```
(monitor) mach set 0  
(machine-0) showAnalyzer sysbus.uart
```

15.1 Autosaving

Renode has an autosaving system, which will automatically take a snapshot of the emulation state. You can enable autosaving system with specific virtual time interval by using `autoSave` command:

```
(monitor) autoSave true "0.2"
```

15.2 State saving in tests

It's possible to use the state saving and loading mechanism when defining complex Robot tests. For details see the *Test cases dependencies* section.

15.3 Loading gzip compressed save files

Renode also supports loading snapshots compressed with gzip.

You can load them the same way as regular save files:

```
(monitor) Load @statefile.save.gz
```

Or directly from CLI:

```
$ renode statefile.save.gz
```

RENODE SENSOR DATA FORMAT (RESN)

For an introduction and description of simple operations on your simulated sensors, such as assigning a constant value as a sensor readout, see the *Sensors and virtual environment* chapter.

16.1 What is the Renode Sensor Data format?

Renode Sensor Data (RESN) is a unified and portable way to provide sample data for sensor models implemented in Renode. Sensor data described in `.resn` files use standard units and fixed formats for all data types, so they can be used by any existing or future model.

The data stored in a `.resd` file is divided into independent channels, each of a given type (e.g. temperature or acceleration). This allows a single input file to be used for multiple sensors such as IMU. It is also possible to include multiple channels of the same type (each identified by a unique channel ID), e.g. two channels for temperature readings.

Each .resd file consists of a single file header followed by a variable number of data blocks:

00000000	52	45	53	44	01	00	00	00	02	01	00	00	00	a8	01	00	RESD.....
00000010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	70	c9	p.
00000020	b2	8b	00	00	00	00	00	00	00	00	00	00	00	cc	9b	03	
00000030	00	c4	95	03	00	79	9b	03	00	18	9a	03	00	75	a3	03	y.....u..
00000040	00	3b	a1	03	00	b1	b4	03	00	8d	9d	03	00	41	a5	03	.;.....A..
00000050	00	4d	9b	03	00	21	af	03	00	6f	96	03	00	20	ba	03	.M...!...o... ..

You can easily create a binary file like the one shown above from a CSV file using the *CSV-RES* parser.

The file header consists of a 4-byte magic string value RESD encoded in ASCII, followed by a single byte defining the version of the format the file uses (the code above uses version 0x1), followed by 3 bytes of reserved padding (currently defined as all zeros).

16.2 RESD Data Blocks

Each block header is structured in the following way:

Table 16.1: Block header structure

Bytes	0	1 - 2	3 - 4
Name	Block type	Sample type	Channel id

16.2.1 RESD Block Types

Currently, you can use the following block types in your `.resd` files:

Table 16.2: Block types

ID	Block type
0x0	Reserved
0x1	<i>Arbitrary Timestamp Sample Blocks</i>
0x2	<i>Constant Frequency Sample Blocks</i>

Arbitrary timestamp sample blocks

Arbitrary timestamp sample blocks contain a series of samples, each with its own *timestamp*. In this type of sample block, you do not specify a period between samples, but instead provide a specific timestamp for each sample to simulate irregular sensor readings.

Note

The timestamp of subsequent samples must be strictly increasing.

Constant frequency sample blocks

In constant-frequency blocks, the block header is 16 bytes longer than the arbitrary timestamp block and includes an additional sub-header that provides information about the timestamp of the first sample in a series and a period between consecutive samples.

Note

If your block contains more than 1 sample, the period value of θ is invalid.

Timestamps

Timestamps in `.resd` are encoded as unsigned 8-byte values expressed in virtual nanoseconds counted from the beginning of the file.

Note

The `.resd` parser can optionally support a global timestamp addend applied to all samples in the file (e.g. to allow loading the same input file twice at different moments in virtual time).

16.2.2 RESD Sample Types

Your .resd files can contain the following sample types:

tabulartabulary

Table 16.3: Sample types

ID	Sample Type
0x0000	Reserved
0x0001	Temperature
0x0002	Acceleration
0x0003	Angular rate
0x0004	Voltage
0x0005	ECG
0x0006	Humidity
0x0007	Pressure
0x0008	Magnetic Flux Density
0x0009	Binary Data
0xF000 - 0xFFFF	Custom

Keep in mind that sample types other than Custom do not utilize a *metadata* dictionary (metadata size set to 0).

Note

Using a custom sensor data type makes the input data tightly coupled to the particular sensor implementation, and therefore, it might not be compatible with other sensors.

Metadata

Metadata is a binary-encoded dictionary, where each entry consists of a key name, key type, and value. The first 8 bytes indicate the size of the metadata section and can be set to 0 to indicate that the given block contains no additional metadata.

The key name is a null-terminated string consisting of [a-z0-9_] characters. It's followed by a byte that describes the type of the value for the given key:

tabulartabulary

Table 16.4: Metadata Types

ID	Metadata type
0x00	Reserved
0x01	int8
0x02	uint8
0x03	int16
0x04	uint16
0x05	int32
0x06	uint32
0x07	int64
0x08	uint64
0x09	float
0x0A	double
0x0B	string (null-terminated)
0x0C	blob

Note

In blob type metadata, the first 4 bytes encode the blob content length reset as an unsigned integer.

For example, the encoding of a metadata section consisting of two entries: a description of type string and data of type blob would look like this:

Table 16.5: Example metadata string and blob

tabulartabulary

Bytes	Name
0 - 7	Size
8 - 19	Key #1 (string)
20	Type #1 (string)
21 - 59	Value #1
60 - 64	Key #3 (string)
65	Type #3 (blob)
66 - 69	Blob size
70 - 74	Blob content

16.2.3 Custom Sample Data Example - MAX86171 AFE

The following instructions show how to use RESD to create a custom sample data example based on a real sensor - **MAX86171 AFE**.

The measurements taken by the MAX86171 AFE sensor depend on the channel configuration, e.g. the LED exposure drive current value directly affects the photodiode output. Therefore, each MAX86171 AFE sample data block begins with a metadata section containing a dictionary of the following information:

Table 16.6: Metadata string and blob example

tabulartabulary

Key name	Type
led_a_exposure	uint16
led_a_source	uint8
led_b_exposure	uint16
led_b_source	uint8
led_c_exposure	uint16
led_c_source	uint8
pd_a_source_flags	uint8
pd_a_adc_range	uint32
pd_a_dac_offset	int16
pd_b_source_flags	uint8
pd_b_adc_range	uint32
pd_b_dac_offset	int16

MAX86171 AFE Sample Data

A single MAX86171 AFE sample is described as a one-byte unsigned value containing a number of active channels followed by a list of measurement frames, each encoded as a four-byte unsigned value:

Table 16.7: MAX86171 AFE sample data structure

tabulartabular

Bytes	0
Values [raw AFE]	Number of frames

An AFE sample in RESD corresponds to a single frame generated by the MAX86171 AFE: two tagged samples are required for each active measurement, as described in the FIFO Description section of the MAX86171 data sheet:

Table 16.8: MAX86171 AFE sample details

tabulartabular

Bits	31..24	23..20	19..0
Values	Reserved	Tag	Value

The tag section follows the specification from the FIFO Description section of the sensor's data sheet:

Table 16.9: MAX86171 AFE sample details

tabulartabular

Tag	Description
0x00	Reserved
0x01	Measurement 1 Data
0x02	Measurement 2 Data
0x03	Measurement 3 Data
0x04	Measurement 4 Data
0x05	Measurement 5 Data
0x06	Measurement 6 Data
0x07	Measurement 7 Data
0x08	Measurement 8 Data
0x09	Measurement 9 Data
0x0A	Dark Data
0x0B	ALC Overflow Event
0x0C	Exposure Overflow Event
0x0D	Picket Fence Event
0x0E	Invalid Data
0x0F	Reserved

A sample with data two measurement OC channel 1 would then look like this:

Table 16.10: MAX86171 AFE sample details

tabulartabular

Bytes	0	1
Names	No. frames	Frame 1 (Measurement 1)
Values	0x2	0x00 0x01 0x12 0

The frames can be decoded as:

Table 16.11: MAX86171 AFE decoded frames

tabulartabulary

Bits	31..24	23..20
Names	Reserved	Measurement 1 tag
Values	0x000	0x1

16.3 RESD tooling

16.3.1 CSV - RESD parser usage

The **CSV - RESD parser** is a tool in the Renode repository that allows you to convert CSV files to the RESD file format.

To use the tool, follow this syntax:

```
./csv2resd.py [GROUP]... OUTPUT
GROUP ::= -i <csv-file> [-m <type>:<field(s)>:<target(s)>*<channel>*]
        [-s <start-time>] [-f <frequency>] [-t <timestamp>] [-o <offset>] [-c
        ↪<count>]
```

The syntax allows multiple group specifications, where `--input` is a separator between groups. You can specify multiple mappings (`--map`) for each `--input`.

The `*` in `--map` indicates that the given property is optional. For your `--map` argument to be correct, it must be structured in one of the following ways:

- `--map <type>:<field(s)>`
- `--map <type>:<field(s)>:<target(s)>`
- `--map <type>:<field(s)>:<target(s)>:<channel>`
- `--map <type>:<field>::<channel>`

See `--help` for more information.

If you wanted to extract the columns `temp1` and `temp2` from the file `first.csv` and first 3 samples `temp` from the file `second.csv` and then map them to the temperature channels 0, 1 and 2 in RESD, respectively, you would run the script with the following parameters:

```
./csv2resd.py \
-i first.csv \
  -m temperature:temp1::0 \
  -m temperature:temp2::1 \
  -s 0 \
  -f 1 \
-i second.csv \
  -m temperature:temp::2 \
  -s 0 \
  -f 1 \
  -c 3 \
output.resd
```

CSV Field Types

All fields of a single column must be of the same type. Below is a description of how values in CSV will be interpreted:

tabulatorytabulatory Table 16.12: CSV Field Type Rules

Type
uintXX, intXX
float, double
string
blob

16.3.2 RESD introspection

Renode comes with a command for inspecting RESD files without a need to load them to any particular sensor model.

To analyze the content of a RESD file, first load it with:

```
(monitor) resd load r1 @my_samples.resd
RESD file from 'my_samples.resd' loaded under identifier 'r1'
```

Now, you can print information about sample blocks with:

```
(monitor) resd list-blocks r1
Blocks in r1:
1. [00:00:00.000000..00:00:02.000000] Acceleration:0
2. [00:00:02.000000..00:01:05.500000] Acceleration:0
(monitor) resd describe-block r1 1
Index: 1
Sample type: Acceleration
Channel ID: 0
Start Time: 00:00:00.000000
End Time: 00:00:02.000000
Duration: 00:00:02.000000
Samples count: 2
Period: 00:00:01.000000
Frequency: 1Hz
```

You can also dump selected samples (e.g., between timestamps of 2.2 and 2.3 seconds) with:

```
(monitor) resd get-samples-range r1 2 "2.2" "2.3"
00:00:02.203125: [0, 0, 0.1] g
00:00:02.218750: [0, 0, 0.2] g
00:00:02.234375: [0, 0, 0.3] g
00:00:02.250000: [0, 0, 0.2] g
00:00:02.265625: [0, 0, 0.2] g
00:00:02.281250: [0, 0, 0.1] g
00:00:02.296875: [0, 0, 0.0] g
```

For more details, see the command's help output:

```
(monitor) help resd
resd
introspection for RESD files

You can use the following commands:
'resd load NAME PATH'    loads RESD file under identifier NAME
'resd unload NAME'       unloads RESD file with identifier NAME
'resd list-blocks NAME'  list data blocks from RESD file with identifier NAME
'resd describe-block NAME INDEX'    show information about INDEXth block from
↳ RESD with identifier NAME
'resd get-samples NAME INDEX "START_TIME" COUNT'    lists COUNT samples
↳ starting at START_TIME from INDEXth block of RESD with identifier NAME
'resd get-samples-range NAME INDEX "START_TIME" "DURATION"'    lists DURATION
↳ samples starting at START_TIME from INDEXth block of RESD with identifier NAME
'resd get-samples-range NAME INDEX "START_TIME..END_TIME"'    lists samples
↳ between START_TIME and END_TIME from INDEXth block of RESD with identifier NAME
'resd get-prop NAME INDEX PROP' read property PROP from INDEXth block of RESD
↳ with identifier NAME
    possible values for PROP are: SampleType, ChannelID, StartTime, EndTime,
↳ Duration, SamplesCount
```

16.3.3 UART RESD Feeder

The UARTRESDFeeder is a special peripheral designed for feeding UART data to emulated UART buses. It's based on *Virtual Console* therefore it inherits and extends the Python scripting capability.

An instance of the feeder can be created with a Monitor command:

```
(machine-0) machine CreateUARTRESDFeeder "feeder"
```

or in a .repl file:

```
feeder: UART.UARTRESDFeeder @ sysbus
```

Usage

This peripheral uses Binary Data samples from .resd files as a source of data to be inject into the emulation. Such file can be loaded using FeedDataFromRESD method.

```
(machine-0) feeder FeedDataFromRESD @hci.resd mode=Normal channelId=1
↳ sampleOffsetTime=2500000000
```

Timing of the data transmission depends on the mode selected while loading a file. Every block should also specify the intended mode to be used with. A feeding_mode metadata can have one of the string values specified for each mode. If the metadata is not set to a legal value than a warning will be logged and the block will be used, but if a mismatch with the current mode is detected then the block will be skipped.

Normal mode

Normal mode follows the standard RESD behaviour.

The `feeding_mode` metadata should be set to `normal` with this mode.

Trigger mode

Trigger mode relays on the `Trigger` method to be called. This action can be interactive as the methods can be called from the Monitor, or scripted with Python. Also, before the sample is enqueued, it can be retrieved to be inspected with `TryGetNextTriggerSample` method and skipped with `SkipTriggerSample` method.

When the `Trigger` is called the next sample will be transmitted after a delay. The interpretation of the sample's timestamp is changed in this mode and it's used as the delay.

This means that the *arbitrary timestamp type of RESD blocks* is preferred. With such interpretation the fixed frequency based block type is unlikely to be useful due to the accumulation of the period in the samples' timestamp. That is, the delay since the trigger would be $(n-1) * \text{period} + \text{start_time}$ for the n th sample.

The `feeding_mode` metadata should be set to `trigger` with this mode.

Scripting

The UART RESD Feeder extends Virtual Console's API and changes the echoing behaviour. The unchanged parts of the API are not listed here, but rather can be checked in *Virtual Console documentation*.

Table 16.13: UARTRESDFeeder properties

Property name
Echo
AllowUserOutput
RESDMode

Table 16.14: UARTRESDFeeder methods

Method name	
FeedDataFromRESD	Starts data outputting from a RESD
DisplayChar	Transmits a byte if not in RESDMode or AllowUserOutput
Trigger	Triggers a scheduled transmission
TryGetNextTriggerSample	Retrieves next trigger sample
SkipTriggerSample	Skips use of the next trigger sample

Table 16.15: UARTRESDFeeder events

Event name	Description
SampleWritten	Invoked when a sample has been transmitted
RESDFileLoaded	Invoked when RESD file has been loaded

A more detailed description can be found in the source file: [UARTRESDFeeder.cs](#).

SENSORS AND VIRTUAL ENVIRONMENT

Renode enables you to manipulate virtual environmental conditions like Temperature and Humidity for sensors in the emulation. Sensors can be either controlled individually by directly setting their values (in the simple use case) or globally using an Environment medium (in order to implement scenarios with multiple sensors reading consistent virtual environmental conditions).

17.1 Controlling individual sensors

To control an individual sensor in your Renode machine, you need to access the appropriate property available to the sensor. To change the Temperature property of a sensor, use:

```
(machine-0) spi0.temperatureSensor Temperature 36.6
```

This value can then be read by accessing the same property::

```
(machine-0) spi0.temperatureSensor Temperature  
36.6
```

17.2 Controlling sensors globally

For more advanced use cases, Renode offers the Environment object, which is an abstract medium representing a space with physical properties. It allows you to group and manage sensors that should observe the same environmental conditions.

17.2.1 Creating an environment

To create an environment called env, execute:

```
(monitor) emulation CreateEnvironment "env"
```

You can create multiple environments as needed. Currently, Environment supports two parameters: temperature and pressure.

You can set the temperature in the environment by calling:

```
(monitor) env Temperature 36.6
```

To check the current setting:

```
(monitor) env Temperature  
36.6
```

17.2.2 Adding sensors to an environment

There are two ways of adding sensors to an environment:

- add a single sensor from a machine,
- add a machine to an environment.

You have to be in a *machine context* to be able to execute the following commands.

Adding a single sensor

To add the temperatureSensor peripheral, connected via the i2c bus, run:

```
(machine-0) i2c.temperatureSensor SetEnvironment env
```

Only the specified sensor will be added to the env environment and will observe the temperature value of that environment.

Adding a machine

To add the current machine to the environment, run:

```
(machine-0) machine SetEnvironment env
```

This command will connect the machine and all of its sensors to the environment env. They will have the same temperature value as the environment.

Note

When connecting a machine to an environment, all its sensors that were previously connected to other environments will be reconnected as well.

Sensors added to an environment will be updated every time the environment changes. However, you can still set your own value on a specific sensor, and it will not be propagated to other sensors (until a change in the environment overrides this).

METRICS ANALYZER

Renode enables collecting execution data from the simulation and allows profiling the execution itself. Currently supported execution metrics:

- executed instructions,
- memory accesses,
- peripheral accesses,
- exceptions.

18.1 Profiling

To enable profiling in Renode, type:

```
(monitor) machine EnableProfiler "path_to_dump_file"
```

Run the simulation. The profiler is now collecting data from the metrics. Close Renode once this step is finished. As a result, you will get a dump file with collected metrics.

The dump can be analyzed using the [metrics_parser Python library](#) or visualised with the provided helper script.

18.2 Visualizing

To display graph representations of the collected data by means of the visualizer bundled with Renode follow these steps:

18.2.1 Additional prerequisites

To install prerequisites for the metrics visualization layer, run the following command from the root Renode directory :

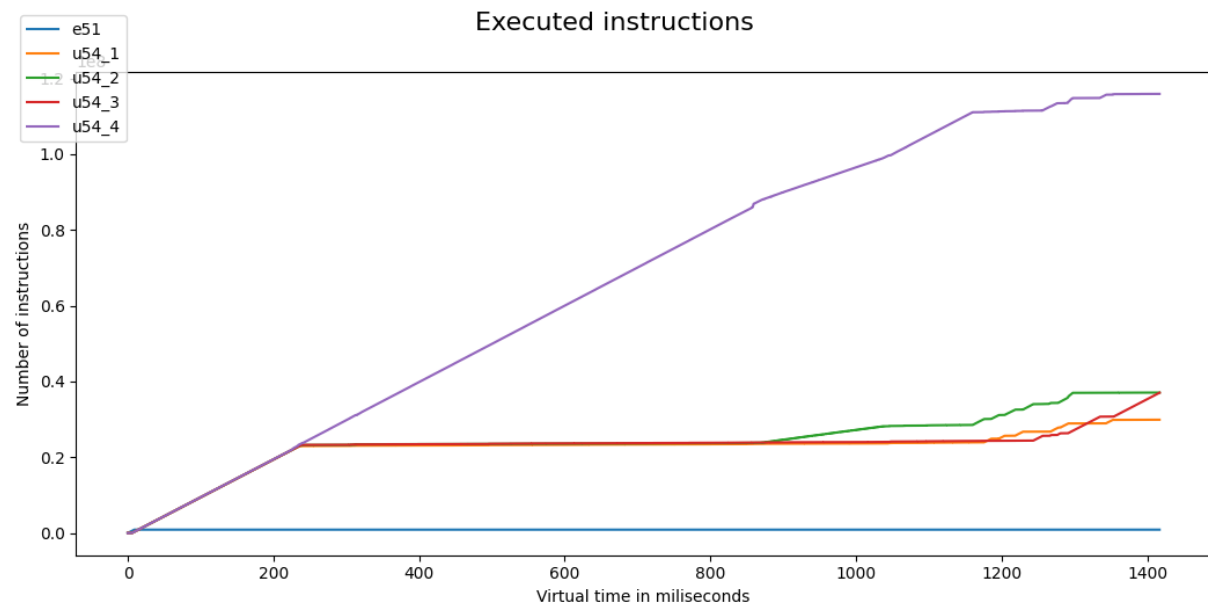
```
python3 -m pip install --user -r tools/metrics_analyzer/metrics_visualizer/  
↳requirements.txt
```

18.2.2 Run script

Run the following script:

```
python3 tools/metrics_analyzer/metrics_visualizer/metrics-visualizer.py path_to_  
↳dump_file
```

As a result, a window with graphs should appear, similar to the one presented below.



RUNNING RENODE IN DIFFERENT MODES

By default Renode runs in GUI mode. When started, it opens a new window for *the Monitor* and additional windows for different analyzers (e.g., UART).

It is, however, possible to start Renode in other modes too. Below we present alternative ways of running and interacting with Renode.

19.1 Telnet mode

Renode can be started with the Monitor interface available over a network socket instead of a window. In this mode it will still open other windows locally, but the simulation can be controlled remotely.

To start Renode in telnet mode, run it with the `-P` switch:

```
$ renode -P 1234
17:01:20.6362 [INFO] Loaded monitor commands from: /home/antmicro/renode/scripts/
↪monitor.py
17:01:20.6800 [INFO] Monitor available in telnet mode on port 1234
17:02:17.2373 [INFO] Script: hello
```

After that you can connect to it with:

```
$ telnet 127.0.0.1 1234
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
Renode, version 1.12.0.33182 (0cd6e174-202108311750)

(monitor) log "hello"
(monitor)
```

You can disconnect from the telnet session and the Renode instance will continue running in the background.

In order to close Renode opened in telnet mode, you have to connect to it and issue the quit command.

19.2 Headless mode

Renode can be built/run in headless mode that doesn't require any graphical environment (e.g., X11) to be present in the system. This mode is especially useful when running simulation in a CI environment.

19.2.1 Building for headless use

To build Renode in a headless environment use the `--no-gui` switch:

```
$ ./build.sh --no-gui
```

19.2.2 Running in a headless environment

Even if you have Renode with a compiled-in GUI support (the default configuration), you can start it in headless mode with:

```
$ renode --disable-gui
```

It is similar to telnet mode as the Monitor will be available on port 1234 by default (the port number can be changed with the `-P` switch). The difference is that in headless mode no graphical windows will be created for analyzers - e.g., UART analyzers will output to log by default.

19.3 Console mode

It is possible to start Monitor in the same console window where Renode is started. In this mode the prompt will be intertwined with log messages:

```
$ renode --console
16:54:37.2215 [INFO] Loaded monitor commands from: /home/antmicro/renode/scripts/
↳monitor.py
Renode, version 1.12.0.33182 (0cd6e174-202108311750)

(monitor) log "hello"
16:54:41.0966 [INFO] Script: hello
(monitor)
```

Note

Console mode can be mixed together with headless mode by passing both `--console` and `--disable-gui`.

19.4 UART interactions in the Monitor

With the `uart_connect` command it is possible to switch between the Monitor and an interactive UART session:

```
[...]
(machine-0) uart_connect sysbus.uart0
Redirecting the input to sysbus.uart0, press <ESC> to quit...
```

(continues on next page)

(continued from previous page)

```
Welcome to buildroot!
master login: root
# ls
# ls /
bin      etc      lib      media    opt      root     sbin     tmp      var
dev      home     linuxrc  mnt      proc     run      sys      usr
# Disconnected from sysbus.uart0
(machine-0)
```

In this mode all input from the user is directed to UART and all output from UART is displayed in place of the Monitor prompt. This works in all Monitor modes (window, telnet, console).

USING PYTHON IN RENODE

Renode offers several different Python integrations which can be used to extend it, both from the inside and outside. It is important to differentiate between them to avoid confusion:

- built-in IronPython integration (this is Python 2 running inside Renode) which is the main focus of this chapter
- external **pyrenode** integration (this is a Python 3 wrapper using RPC calls to call Renode / execute Robot keywords)
- (NEW!) external **pyrenode3** integration (this is the new, native CLR bindings, essentially a Python 3 wrapper around all of Renode, set to replace the ‘older’ pyrenode when stabilized)

This chapter currently focuses on the built-in IronPython integration and use of Python from within Renode, hence the Python 2 syntax with old-style print statements you will notice throughout. Eventually, the internal (Iron)Python integration will either be migrated to IronPython3 or moved over to the the same mechanism pyrenode3 is using.

In the meantime, documentation will be added for pyrenode3 to complement this chapter.

Thanks to the IronPython integration, you can use Python in runtime to do things like reacting to lines on UART, change of user state, peripheral access, or a value appearing in the memory.

Python offers a real programming language syntax with its flow control constructs, loops, etc.

Python also gives you access to emulation components available under context-specific variable names. These variables are described in respective sections. They serve as an API between the script and the rest of Renode.

20.1 Direct Python execution in the Monitor

Python code can be executed directly in the Renode Monitor using the python command in the Monitor:

```
(monitor) python [ py ]
```

You can also use the include command to load an existing .py file:

```
(monitor) include @path/to/file.py
```

20.1.1 Extended context

Scripts ran this way have access to additional variables:

Table 20.1: Custom variables in Monitor

Variable name
monitor, self
externals
variables

The Monitor instance is an C# object, which provides context like currently selected machine, so e.g. the expression `monitor.Machine['sysbus.uart0']` can be used to access the `uart0` peripheral from the machine.

Accessor objects can be either treated as objects or indexed similarly to a dictionary, i.e. `variables.foo` and `variables['foo']` are equivalent, but the index syntax can accept any string.

Moreover, the scope is shared between executions, so any modifications to the scope, i.e. defined function or variables, will be carried over to all future Python script executions.

20.1.2 Providing Python scripts from the Monitor

To provide a multiline Python script from the Monitor or a `.resc` file, use the following notation:

```
(monitor) set my_script """
> print "Hello"
> print "This is a multiline Python script"
> if 0 > 1:
>     print "This will not be printed"
> """
(monitor) python $my_script
```

Using the `print Python` command will output the text on the Monitor:

```
(monitor) python "print 'Hello'"
Hello
```

20.1.3 Creating Monitor commands in Python

Running the `python` command in the Monitor executes the provided code immediately. However, it is possible to use these Python scripts to define functions available for later use.

Such functions can be executed with subsequent `python` calls, but you can also expose them directly to the Monitor, if you follow a specific naming convention.

If the name of the function you define starts with the `mc_` prefix, it will become available as a new Monitor command, stripped of this prefix.

For example, consider the following Python definition:

```
def mc_sleep(time):
    sleep(float(time))
```

When executed in Renode, this definition provides a sleep function in the Monitor:

```
(monitor) sleep 5
```

A small set of predefined Python commands is available in the `monitor.py` file.

20.2 Python peripherals in a platform description

You can use Python in the `.repl` file of your platform description. Several platforms included in Renode like `Tegra 2` use Python peripherals to implement simple logic. The most common use of Python peripherals is mocking certain blocks that are not fully implemented but are required by the software.

To create a Python peripheral, you need to specify several variables:

```
variableName: Python.PythonPeripheral @ sysbus 0x7000F410
    size: 0x4
    initable: false
    script: "request.Value = 0x60000"
```

To define Python peripherals, you can specify the following variables:

Table 20.2: Request variable fields

Variable name
size
initable
script
filename

When programming a Python peripheral, you have access to the request variable, which describes the current transaction.

The variable request in Renode gives you access to the following fields:

Table 20.3: Request variable fields

Field name
IsInit
IsRead
IsWrite
Value
Offset
Length

Note

The variable `IsInit` is true when the peripheral is marked as possible to initialize and is currently being initialized. It can be useful for initializing the internal state of a peripheral.

An example Python peripheral available in mainline Renode is `repeater.py`, returning the same value that was written to it by software:

```

if request.IsInit:
    lastVal = 0
elif request.IsRead:
    request.Value = lastVal
elif request.IsWrite:
    lastVal = request.Value

self.NoisyLog("%s on REPEATER at 0x%x, value 0x%x" % (str(request.Type), request.
↪Offset, request.Value))

```

There are also other variables that can be used in Python peripherals:

Table 20.4: Other Python peripherals variables

Field name	
self	
size	Size

20.3 Python hooks in Renode

There are multiple types of hooks in Renode, which allow you to execute code, e.g. a Python script, when certain specified conditions are met. Each separate hook provides you with different functionality, and multiple types of hooks can be used in a single project. Types of hooks in Renode:

- *UART hooks,*
- *CPU hooks.*
- *system bus hooks,*
- *watchpoint hooks,*
- *packet interception hooks,*
- *user state hooks,*

20.3.1 UART hooks

UART hooks enable you to code a reaction to the event when a specific character or substring appears on UART.

UART hooks in Renode have access to the following variables:

Table 20.5: UART hooks variables

Variable name	
line	
machine	
uart	
self	

You have to specify the string you want to search for in the UART’s output and the Python script that will be executed if the substring appears on UART:

```
(machine) uart AddLineHook "searched value" "print 'Found the %s string' % line"
```

Note

You have to use a UART name that matches your use case.

20.3.2 CPU hooks

CPU hooks enable you to code a reaction to reaching a specific stage in code execution. The hook can be triggered at the beginning, end of code of an executed block, or any specified point in the application. CPU hooks can also react to the interrupts in code and be triggered at the beginning or an end of an interruption or when the CPU enters or exits from WFI (“Wait-For-Interrupt”) state.

All CPU hooks in Renode have access to the following variables:

Table 20.6: CPU hooks common variables:

Variable name
machine
self

Specific hooks can have additional variables available:

Table 20.7: CPU block hooks variables

Variable name
pc
size
cpu

Table 20.8: CPU interrupt hooks variables

Variable name
exceptionIndex

Table 20.9: CPU Wait-For-Interrupt hooks variables

Variable name
isInWfi

To use a CPU hook, you need to name cpu you want to attach the hook to and specify the method. To create a hook that triggers at the beginning of interrupt:

```
(machine) cpu AddHookAtInterruptBegin "print 'Interrupt has started'"
```

To create a hook that triggers at the end of interrupt:

```
(machine) cpu AddHookAtInterruptEnd "print 'Interrupt has ended'"
```

To create a hook that triggers when the CPU enters or exits “Wait-For-Interrupt” state:

```
(machine) cpu AddHookAtWfiStateChange "print 'Entered/Exited WFI'"
```

Note

WFI state can be triggered by various CPU instructions depending on the architecture. For example, on Arm, both WFI and WFE instructions can be used to enter this state.

To create a hook that triggers at the beginning of an executed block:

```
(machine) cpu SetHookAtBlockBegin "print 'Execution of a code block has started'"
```

To create a hook that triggers at the end of an executed block:

```
(machine) cpu SetHookAtBlockEnd "print 'Execution of a code block has ended'"
```

You can also create a hook for any specified point in the application:

```
cpu AddHook 0x60000000 "print 'You have reached a hook'"
```

and even combine the previous command with another one to add a hook at a symbol:

```
cpu AddHook `sysbus GetSymbolAddress "main"` "print 'You have reached the main_↵
↵function'"
```

Alternatively, you can use

```
cpu AddSymbolHook "main" "print 'You have reached the main function'"
```

to set a hook that will ‘follow’ the symbol and set additional hooks when it is loaded at a different address. This is useful for software that relocates itself at runtime.

20.3.3 OS-aware modes

OS-aware modes make use of OS-specific behavior to enhance the simulation performance and add quality of life improvements.

Table 20.10: OS-aware modes

Mode	
Features	Skips busy loop in _udelay, automatically
How to enable?	

20.3.4 System bus hooks

System bus hooks enable you to code a reaction to the event when peripherals are accessed to read or write. This hook has access to the peripherals, and it can be assigned to react only to a specific peripheral.

System bus hooks in Renode have access to the following variables:

Table 20.11: System bus hooks value to write variables

Variable name
self
sysbus
machine
value
offset

Table 20.12: System bus hooks value to read variables

Variable name
self
sysbus
machine
value
offset

To create a system bus hook that executes a Python script after a specific peripheral is accessed to read:

```
(machine) sysbus SetHookAfterPeripheralRead peripheral "print '%s peripheral has_
↪been accessed to read'"
```

To create a system bus hook that executes a Python script before a specific peripheral is accessed to write:

```
(machine) sysbus SetHookBeforePeripheralWrite peripheral "print '%s peripheral_
↪has been accessed to write'"
```

20.3.5 Watchpoint hooks

Watchpoint hooks enable you to code a reaction to the event when a specific value appears in a specific memory address.

Watchpoint hooks in Renode have access to the following variables:

Table 20.13: Watchpoint hooks variables

Variable name	
cpu	
address	
width	
value	
self	

To create a watchpoint hook that executes a Python script, when a Read access with width of 32 bits (DoubleWord) appears at address 0x70001000, run:

```
(machine) sysbus AddWatchpointHook 0x70001000 DoubleWord Read "print '32 bit_
↪value appeared at the address 0x70001000'"
```

Note

You can use either word (DoubleWord) or numeric (4) notation in width, as listed in the table above.

20.3.6 Packet interception hooks

Packet interception hooks enable you to code a reaction to the event when a packet appears on a radio medium.

Packet interception hooks in Renode have access to the following variables:

Table 20.14: Packet interception hooks variables

Variable name	
packet	
self	
machine	

Note

Keep in mind that the hook is executed for the recipient, not the sender.

To have a functioning Packet interception hook, your machine needs to have a wireless medium. To create a packet interception hook, run:

```
(machine) wireless SetPacketHookFromScript sysbus.radio "if packet[5] == 0x4:
↪print('I\'m interested in packets with 0x4 as their sixth byte')"
```

You can also execute a script from a file:

```
(machine) wireless SetPacketHookFromFile sysbus.radio @path/to/file.py
```

20.3.7 User state hooks

User state hooks enable you to code a reaction to the event when “UserState” string of a machine changes.

User state hooks in Renode have access to the following variables:

Table 20.15: User states hooks variables

Variable name
state
self

To create a user state hook that executes Python script when a specific state is set:

```
(machine) machine AddUserStateHook "state" "print 'User state has changed to: %s'"
↪state "
```

20.4 Python enabled classes

The hooks described in previous sections use special methods that setup the execution environment for your Python script, but IronPython also allows you to attach Python functions to C# events.

As an example let’s create a trivial Ethernet sniffer:

```
(machine-0) emulation CreateSwitch "switch"
(machine-0) python "externals.switch.FrameProcessed += lambda switch, sender,
↪data: sender.LogDebug(str(data))"
```

For an in-depth explanation of setting up the network see the *Setting up a wired network* chapter.

20.4.1 Dummy I2C Slave

The `Mocks.DummyI2CSlave` peripheral provides a set of methods and events that allow you to implement a simple I2C peripheral.

Table 20.16: DummyI2CSlave events

Event name	
DataReceived	Invoked when the peripheral receives data
ReadRequested	Invoked when a read is requested from the peripheral
TransmissionFinished	Invoked when the controller finishes transmission

The `DataReceived` event doesn’t expect any data to be provided, but you can enqueue response bytes using the methods listed below. By default, when the internal buffer is empty, the peripheral returns zeros and will pad any enqueued data with zeros when not enough bytes is in the buffer.

Table 20.17: DummyI2CSlave methods

Method name	
Reset	Clears the internal buffer, but does not
EnqueueResponseByte	Enqueues a single byte
EnqueueResponseBytes	Enqueues a sequence of bytes

With those available, we can change the dummy I2C peripheral into a simple I2C echo peripheral:

```
dummy = monitor.Machine["sysbus.i2c.dummy"]
dummy.DataReceived += lambda data: dummy.EnqueueResponseBytes(data)
```

20.4.2 Virtual Console

The VirtualConsole is meant to be a helper for Python scripts as it is supported with the same infrastructure, e.g. you can use it in Robot Framework tests with UART keywords and create an analyzer to interact with the console. It provides an internal buffer for the received data and methods and events listed below that allow for control from a Python script.

An instance of the console can be created with a Monitor command:

```
(machine-0) machine CreateVirtualConsole "vconsole"
(machine-0) showAnalyzer vconsole
```

or in a repl file:

```
vserial: UART.VirtualConsole @ sysbus
```

Table 20.18: VirtualConsole methods

Method name	
Clear	
IsEmpty	
Contains	Checks if the buffer contains the specified character
WriteChar	Writes a single character to the buffer
ReadBuffer	Retrieves data from the buffer
GetBuffer	Retrieves the buffer
WriteBufferToMemory	Writes some or all of the bytes from the buffer to memory
DisplayChar	

Using the methods listed above and some other hooks for accesses you can implement a polling-based peripheral, e.g. the [Segger RTT](#), which is used in demos (e.g. [scripts/single-node/ek-ra2e1.resc](#)) with interactive console and in Robot Framework tests using UART keywords (e.g. [tests/platforms/EK-RA2E1.robot](#)).

Table 20.19: VirtualConsole events

Event name	Description
CharReceived	Invoked when the peripheral transmits a character
CharWritten	Invoked when a byte is pushed to the receive buffer

The VirtualConsole also has the Echo property to enable or disable displaying of received characters.

20.4.3 UART RESD Feeder

The UARTRESDFeeder extends VirtualConsole with RESD support for feeding UART data into the emulation. For more information please refer to the *RESD chapter*.

20.5 RISC-V extensions

Renode has extensive support for RISC-V architecture and lets you code extensions using Python. You can write the logic of Control/Status Register and custom instructions and then install them either from a file or a string. Those instructions can be either 16-bit, 32-bit, or 64-bit.

Python implementations of RISC-V custom instructions have access to the following variables:

Table 20.20: RISC-V custom instructions variables

Variable name
cpu
machine
state
instruction

Note

Variable state provides you with additional CPU state, mapping user-defined strings to arbitrary objects. It is useful when you want to share the state between different instructions.

Custom instructions from a string can be installed using:

```
(machine) sysbus.cpu InstallCustomInstructionHandlerFromString
↪ "101100111100011110000111110000010" "cpu.DebugLog('custom instruction executed!')
↪ "
```

If you want to install a custom instructions from a file, use:

```
(machine) sysbus.cpu InstallCustomInstructionHandlerFromFile
↪ "101100111100011110000111110000010" "path/to/file.py"
```

RISC-V custom instructions extensions in Renode have access to the following variables:

Table 20.21: RISC-V custom CSR variables

Variable name
cpu
machine
request

Custom instructions have access to the request object, which is similar to bus access. It has access to various properties:

Table 20.22: The request structure

Property name
CSR number
Value
Type
IsInit
IsRead
IsWrite

To install custom Control/Status Register from a string, use:

```
(machine) sysbus.cpu RegisterCSRHandlerFromString 0xf0d "print 'CSR has been_↵
↵accessed' "
```

To provide a custom CSR from a file, instead use:

```
(machine) sysbus.cpu RegisterCSRHandlerFromString 0xf0d "path/to/file.py"
```

i **Note**

You should adjust the CSR number, 0xf0d in this example, according to your needs.

Examples of custom CSR and custom instructions can be found in [Renode’s custom instructions tests](#).

CONFIGURING A RISC-V CPU

Among the architectures supported in Renode, RISC-V is one of the most prominent. Renode supports both 32 and 64-bit versions of RISC-V, Privileged Architecture in various versions and a broad range of extensions.

The most common base ISA sets supported by Renode are:

- RV32/64I Base (I)
- Integer Multiplication and Division (M)
- Atomic Instructions (A)
- Single-Precision Floating-Point (F)
- Double-Precision Floating-Point (D)
- Control and Status Register Instructions (Zicsr)
- Instruction-Fetch Fence (Zifencei)

All of the above constitute the G set.

Renode also supports many of the ISA extensions like:

- Compressed Instructions (C)
- Vector Operations (V)
- Bit-manipulation
 - Address generation instructions (Zba)
 - Basic bit-manipulation (Zbb)
 - Carry-less multiplication (Zbc)
 - Single-bit instructions (Zbs)
- Half-Precision Floating-Point (Zfh)
- Atomic Compare-and-Swap (CAS) (Zacas)
- PMP Enhancements for memory access and execution prevention in Machine mode (Smepmp)

Lastly, Renode also supports custom, non standard instruction sets. Some of them can be selected like other instruction sets (e.g. Xandes), others are defined directly in their respective core classes (e.g. CV32E40P).

To define a RISC-V core in your simulation, you must edit the Renode Platform file (.repl) and add a node for the CPU.

21.1 Picking specific ISA variants

Start by adding CPU.RiscV32 or CPU.RiscV64 to the .repl file:

```
cpu: CPU.RiscV32 @ sysbus
```

then use the cpuType argument to specify the ISA and the required extensions.

Start with “rv32” or “rv64”, depending on the architecture width, followed by a list of enabled ISA sets. Extensions with names longer than one character have to be separated by an underscore.

For example:

```
cpu: CPU.RiscV32 @ sysbus
  cpuType: "rv32imaf_zicsr_zifencei"
```

which denotes a 32-bit RISC-V with the base instruction set (I), integer multiplication and division (M), atomic instructions (A), single-precision floating-point (F), CSR instructions (Zicsr), and instruction-fetch fence (Zifencei).

21.2 Customizing the CPU

There are additional parameters you can pass to the CPU while creating it in the .repl file. All of these are optional.

- `timeProvider` - sets the peripheral to be used as the time provider for the CPU, which is used to populate the time CSR. Typically you would provide your instance of the clint interrupt controller
- `privilegedArchitecture` - selects which version of the Privileged Architecture the CPU should follow. The default is 1.11. Available values are:
 - `PrivilegedArchitecture.Priv1_09`
 - `PrivilegedArchitecture.Priv1_10`
 - `PrivilegedArchitecture.Priv1_11`
 - `PrivilegedArchitecture.Priv1_12` - Currently support for Privileged Architecture v1.12 is experimental and not everything is implemented.
- `endianness` - specifies the endianness of the CPU, defaulting to little endian
- `nmiVectorAddress` and `nmiVectorLength` - allow for customizing the non-maskable interrupt vector, if supported by the CPU
- `allowUnalignedAccesses` - defines if an exception should be raised whenever the software performs an unaligned access on memory. Defaults to false
- `interruptMode` - Allows you to enforce interrupt handling mode. Defaults to auto. Available modes:
 - Auto (0) - Checks mtvec’s LSB to detect the mode

- Direct (1) - All exceptions set PC to mtvec's BASE value
- Vectored (2) - Asynchronous interrupts set PC to mtvec's BASE + 4 * cause
- `privilegeLevels` - specifies implemented privilege levels of the CPU. The default is Machine, Supervisor and User modes. Available values are:
 - `PrivilegeLevels.Machine`
 - `PrivilegeLevels.MachineUser`
 - `PrivilegeLevels.MachineSupervisorUser`

21.3 Adding a custom RISC-V instruction

One of the most important features of RISC-V is its customizability, also with non-standard instructions.

There are several ways in which a custom instruction can be added to a RISC-V CPU in Renode.

They all revolve around two arguments

- **Pattern** - A bit pattern which specifies which instructions to match to execute your custom handler. Characters 1 and 0 denote which bit has to be set in that position, while any other character means "any value". The pattern has to have a length of 64, 32, or 16 characters.
- **Handler** - The code to execute when the pattern matches

21.3.1 Python

You can use a *Python script* for handling custom instructions by using the `InstallCustomInstructionHandlerFromString` or `InstallCustomInstructionHandlerFromFile` methods present on RISC-V CPUs.

For example:

```
sysbus.cpu InstallCustomInstructionHandlerFromString
↪ "1011001110001111000011110000010" "cpu.DebugLog('custom instruction executed!')
↪ "
```

The Python script has the instruction variable available, which contains the opcode of the instruction that was called.

21.3.2 C#

You can use a *C# function* for handling custom instruction by using the `InstallCustomInstruction` method. It takes the same pattern argument, but the handler argument is `Action<UInt64>`

```
public class MyCustomRiscV : RiscV32
{
    public MyCustomRiscV(IMachine machine, IRiscVTimeProvider timeProvider = null,
↪ uint hartId = 0,
        PrivilegedArchitecture privilegedArchitecture = _
↪ PrivilegedArchitecture.Priv1_11,
        Endianess endianness = Endianess.LittleEndian, string cpuType_)
```

(continues on next page)

(continued from previous page)

```

=> "rv32imfc_zicsr_zifencei")
    : base(machine, cpuType, timeProvider, hartId, privilegedArchitecture,
=>endianness, allowUnalignedAccesses : true)
    {
        // As mentioned, the pattern arguments takes a 16, 32, or 64 character long
=>string.
        // Characters 0 and 1 specify the bits that should be set,
        // while all other characters mean that given bit can be either 1 or 0.
        // In this case we use F for the Imm field, B for rs1, and D for rD to make
=>the pattern more readable
        //
        // FFFFFFFFFFBBBBB100DDDD0001011
        //      |           |           +-- [ 7:11] - rD
        //      |           +----- [15:19] - rs1
        //      +----- [20:31] - Imm
        //
        // We could've used "-----100-----0001011" as the pattern and it
=>'d mean the same thing,
        // but suddenly it's not so clear where the different fields are inside the
=>opcode we're matching.
        InstallCustomInstruction(pattern: "FFFFFFFFFBBBBB100DDDD0001011",
=>handler: opcode =>
        {
            this.Log(LogLevel.Noisy, "(p.lbu rD, Imm(rs1!)) at PC={0:X}", PC.
=>RawValue);
            // rD = Zext(Mem8(rs1))
            var rD = (int)BitHelper.GetValue(opcode, 7, 5); // Extract value from
=>opcode starting at bit 7 and length of 5
            var rs1 = (int)BitHelper.GetValue(opcode, 15, 5); // Extract value from
=>opcode starting at bit 15 and length of 5
            var rs1Value = (long)GetRegisterUnsafe(rs1).RawValue;
            SetRegisterUnsafe(rD, ReadByteFromBus((ulong)rs1Value));

            // rs1 += Imm[11:0]
            var imm = (int)BitHelper.SignExtend((uint)BitHelper.GetValue(opcode, 20,
=>12), 12); // Extract value from opcode starting at bit 20 and length of 12

            // and sign-extend it to full int
            SetRegisterUnsafe(rs1, (ulong)(rs1Value + imm));
        });
    }
}

```

21.3.3 Verilated Custom Function Units

You can connect up to 4 CFUs to every RISC-V core that's simulated in Renode.

After you've compiled your CFU with the Cosimulation Library (see: [Example CFU project](#)) you can attach the CFU to your CPU.

To do so, add this line to your .repl:

```
cfu0: CoSimulated.CoSimulatedCFU @ cpu 0
```

and this line to your .resc:

```
cpu.cfu0 SimulationFilePathLinux @<PATH_TO_COMPILED_CFU_BINARY>
```

If you're on Windows as opposed to Linux you must use `SimulationFilePathWindows`, and respectively `SimulationFilePathMacOS` on macOS. Instructions referencing your CFU will then be forwarded to the Verilated CFU.

All CFU instructions follow this pattern: FFFFFFFFAAAAABBBBBIIICCCCN01011

- N - CFU number to forward the instruction call to
- C - Register in which the resulting value will be placed
- I, F - Function ID. The final ID that's passed to the CFU is FFFFFFFIII
- B - Source register 1. Its value will be read and passed into the CFU
- A - Source register 2. Same as the above

21.4 Adding a custom RISC-V Control and Status Register

In a similar fashion to custom instructions, you can also define custom CSRs.

21.4.1 Python

To use a *Python script* to handle your custom CSR you can use `RegisterCSRHandlerFromString` and `RegisterCSRHandlerFromFile`. The first argument is the CSR ID, while the second refers to the handler script.

The script gets passed a request variable, which contains fields `isRead`, `isWrite`, and `value`.

```
if request.isRead: # If the CPU tries to read your CSR, isRead will be True
    cpu.DebugLog('CSR read!')
elif request.isWrite: # Otherwise isWrite will be True and value will contain the
    ↪ value being written
    cpu.DebugLog('CSR written: {}'.format(hex(request.value)))
```

21.4.2 C#

To use C# functions to handle custom CSRs you can use the `RegisterCSR` method. Similarly it takes the ID as the first argument, but then takes the read handler and the write handler separately, in that order.

Let's take the previously defined custom RISC-V CPU and add a custom CSR to it which will return a random value every time it's read.

```
public class MyCustomRiscV : RiscV32
{
    public MyCustomRiscV(IMachine machine, IRiscVTimeProvider timeProvider = null,
    ↪ uint hartId = 0,
        PrivilegedArchitecture privilegedArchitecture = _
```

(continues on next page)

(continued from previous page)

```

↪PrivilegedArchitecture.Priv1_11,
    Endianness endianness = Endianness.LittleEndian, string cpuType_
↪= "rv32imfc_zicsr_zifencei")
    : base(machine, cpuType, timeProvider, hartId, privilegedArchitecture,
↪endianness, allowUnalignedAccesses : true)
    {
        this.random = EmulationManager.Instance.CurrentEmulation.RandomGenerator;

        InstallCustomInstruction(pattern: "FFFFFFFFFFFFBBBBB100DDDDDD0001011",
↪handler: opcode =>
        {
            // Custom Instruction implementation from before
        });

        RegisterCSR(
            (ulong)CustomCSR.Rnd,
            () => (ulong)random.Next(),           // Read handler
            value => { /* Ignore all writes */ }, // Write handler
            name: "RND"                           // Optionally add a name of your CSR
        );
    }

    private readonly PseudorandomNumberGenerator random;

    // Creating an enum isn't required, but it's cleaner
    // and allows to name otherwise arbitrary CSR IDs
    private enum CustomCSR : ulong
    {
        Rnd = 0xfc0,
    }
}

```

DEBUGGING WITH GDB

Renode allows you to debug applications running on emulated machines using **GDB**.

It uses the GDB remote protocol and allows using the most common GDB functions, like breakpoints, watchpoints, stepping, memory access etc.

The most important difference to debugging on the real hardware, is that the virtual time does not progress when the emulated CPU is halted. This makes the debugging process transparent for the emulated machines.

22.1 Connecting to GDB

To start a GDB server on port 3333, run:

```
(machine-0) machine StartGdbServer 3333
```

This allows you to start GDB from an appropriate toolchain and connect to a remote target:

```
$ arm-none-eabi-gdb /path/to/application.elf  
(gdb) target remote :3333
```

Note

This behavior can be different if there are several CPUs of different architectures present. For starting a GDB server on complex multicore platforms, see *complex scenarios* for details on how to proceed.

22.2 Starting emulation

After GDB connects to Renode, the emulation needs to be started. Simply telling GDB to continue is not enough to start the time flow, as it would disrupt more complicated multinode scenarios.

There are three ways to start the whole setup.

You can start the emulation manually from Renode's Monitor, typing the usual:

```
(machine-0) start
```

Then, in GDB, run:

```
(gdb) continue
```

Alternatively, GDB's monitor command may be used to pass the commands to Renode's Monitor:

```
(gdb) monitor start  
(gdb) continue
```

The third option, suited for the simplest scenarios, makes Renode start the whole emulation as soon as GDB connects. It requires an additional parameter for StartGdbServer, named `autostartEmulation`:

```
(machine-0) machine StartGdbServer 3333 true
```

22.3 Complex scenarios

By default, the command `StartGdbServer` tries to add all CPUs of a machine to the newly created server, only if all of them are of the same architecture. Otherwise, specifying the cluster with `cpuCluster="cluster-name"`, or a specific CPU with `cpu=cpuName` is needed:

```
(machine-0) machine StartGdbServer 3333 true cpuCluster="cortex-r5f"
```

If an invalid cluster is provided, the list of available clusters will be printed out to choose from.

However, if you are certain that your debugger can handle heterogeneous CPUs, simply use `cpuCluster="all"` to attach all available cores to the one stub. You can also add clusters/CPU's one-by-one:

```
(machine-0) machine StartGdbServer 3333 true cpuCluster="cortex-r5f"  
(machine-0) machine StartGdbServer 3333 true cpu=sysbus.apu0  
(machine-0) machine StartGdbServer 3333 true cpu=sysbus.apu2
```

This will result in GDB server running on port 3333 and having CPUs from `cortex-r5f` cluster, and additionally CPUs called `apu0` and `apu2`, attached.

It is also possible to add a specific CPU to an existing server, or create a new server with that CPU. This allows you to create more complex setups, with multiple GDB instances running debug sessions with different CPUs.

To start a GDB server on port 3333 with one CPU, two more parameters are required - previously mentioned `autostartEmulation`, and `cpu`:

```
(machine-0) machine StartGdbServer 3333 true sysbus.cpu1
```

To add a second CPU to that server, run:

```
(machine-0) machine StartGdbServer 3333 true sysbus.cpu2
```

To start a new GDB server on port 3334 with another CPU, run:

```
(machine-0) machine StartGdbServer 3334 true sysbus.cpu3
```

These commands will give you a setup consisting of two GDB servers - on port 3333 with two CPUs, and on port 3334 with one CPU.

Furthermore, the `StartGdbServer` command will prohibit you from adding one CPU to more than one GDB server.

If a CPU was added to a GDB server by providing the `autostartEmulation` and `cpu` parameters, it will be impossible to run the general version of the command on that machine.

22.4 Reverse Execution

Renode supports **GDB reverse execution** for single core emulations.

It is based on *snapshots* mechanism. Reverse execution loads a snapshot taken during the emulation and executes simulation up to the desired state. In order to enable the use of reverse execution snapshots are taken periodically, but they can also be taken manually, according to users' strategy to make the debugging more efficient. You can *take snapshots* in places that are most convenient for you or use a monitor command which will enable the autosaving functionality for you:

```
(machine-0) reverseExecMode true
```

You can also configure autosaving *manually* and specify snapshot period.

Once reverse execution is enabled, you can connect GDB and use one of the following commands:

To step back one instruction:

```
(gdb) reverse-step
```

To step back one assembly instruction:

```
(gdb) reverse-stepi
```

To return to the last breakpoint, or to the first snapshot if no breakpoints were set before:

```
(gdb) reverse-continue
```

There are shorthand options for those commands, respectively `rs`, `rsi` and `rc`.

You can also step back more than one instruction. To do that just use an optional argument with number of steps, for example:

```
(gdb) rsi 5
```

DEBUGGING SOFTWARE WITH VISUAL STUDIO CODE

Debugging software on machines emulated in Renode using GDB is also available for VS Code users.

In order to enable single-click debugging in Renode, you need to configure VS Code appropriately.

Renode can run any software, but for the sake of this chapter, we will use a sample configuration for running a TFLite Micro demo on Zephyr RTOS built for the nRF52840 board.

Such a configuration consists of 4 files:

- `.vscode/launch.json`
- `.vscode/tasks.json`
- `platform.resc`
- `debug.conf`

The sample configuration files are located in the `/tools/vscode_config/` directory in the Renode repository.

All of these files include inline comments with additional explanations.

Note

Please note that JSON format does not support comments, but VSCode handles them without issue.

For the provided config files, the VSCode workspace root directory is the directory where “west init” was run, i.e. the directory that contains Zephyr, modules, tools, etc. - all paths are relative to that location.

Place the configuration files in the workspace root directory or adjust the paths in the scripts accordingly.

Note

The `.vscode` directory might be hidden in some file explorers.

23.1 Launch configuration

The “launch.json” file describes a launch configuration named “Debug application in Renode”. If needed, adjust the path to the ELF file and the GDB version to be used for debugging.

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Debug application in Renode",
      "type": "cppdbg",
      "request": "launch",
      "preLaunchTask": "Run Renode",
      "postDebugTask": "Close Renode",
      "miDebuggerServerAddress": "localhost:3333",
      "cwd": "${workspaceRoot}",
      "miDebuggerPath": "arm-zephyr-eabi-gdb",
      "program": "${workspaceRoot}/build/zephyr/zephyr.elf"
    }
  ]
}
```

23.2 Defining tasks

The sample tasks.json file defines three tasks:

1. “Build application” is a sample build task for this particular scenario. Please adjust to your needs. Mind that the generated binary (needs to be an ELF file for effective debugging) is referenced in “platform.resc” and “launch.json”.
2. “Run Renode” runs after “Build application” finishes. It starts the “platform.resc” script and waits for the GDB server to start. It requires a proper path to the resc script.
3. “Close Renode” is a task fired when you close the debugging session. It will shut down Renode, and it’s a matter of personal preference if you want to use it or not. You can disable it in “launch.json” by removing/commenting-out the “postDebugTask” line.

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Build application",
      "type": "shell",
      "command": "west",
      "args": [
        "build",
        "--pristine",
        "--board",
        "nrf52840dk_nrf52840",
        "zephyr/samples/modules/tflite-micro/hello_world",
        "--",
        "-DEXTRA_CONF_FILE=${workspaceFolder}/debug.conf"
      ]
    }
  ]
}
```

(continues on next page)

(continued from previous page)

```

    ],
    "problemMatcher": [
        "$gcc"
    ],
    "group": "build",
    "presentation": {
        "reveal": "always",
        "panel": "dedicated"
    }
},
{
    "label": "Run Renode",
    "type": "shell",
    "command": "renode",
    "args": [
        "${workspaceFolder}/platform.resc"
    ],
    "dependsOn": [
        "Build application"
    ],
    "isBackground": true,
    "problemMatcher": {
        "source": "Renode",
        "pattern": {
            "regexp": ""
        },
        "background": {
            "activeOnStart": true,
            "beginsPattern": "Renode, version .*",
            "endsPattern": ".*GDB server with all CPUs started on port.*"
        }
    },
    "group": "build",
    "presentation": {
        "reveal": "always",
        "panel": "dedicated"
    }
},
{
    "label": "Close Renode",
    "command": "echo ${input:terminate}",
    "type": "shell",
    "problemMatcher": []
}
],
"inputs": [
    {
        "id": "terminate",
        "type": "command",

```

(continues on next page)

(continued from previous page)

```
        "command": "workbench.action.tasks.terminate",  
        "args": "terminateAll"  
    }  
]  
}
```

23.3 Renode simulation script

The “platform.resc” file is a sample Renode simulation script:

```
$bin ?= $CWD/build/zephyr/zephyr.elf  
include @scripts/single-node/nrf52840.resc  
  
machine StartGdbServer 3333
```

It contains two important parts:

- specifying the ELF file
- starting GDB as the last command in the script. You will need to adjust the file to your particular needs.

You can use this script to setup the entire simulation or rely on a predefined script. Most of Renode scripts use the bin variable to specify an executable to run. It should also match the configuration in launch.json.

23.4 debug.conf

The “debug.conf” file is Zephyr-specific. It is used to disable optimizations during compilation, as certain optimizations may hinder the debugging process. You can remove this file altogether, depending on the way you select optimizations.

EXECUTION TRACING IN RENODE

Renode has complete awareness of the internal state of all simulated components and can use this information to trace the execution of unmodified binaries without any behavioral changes to the simulation.

Note

Some logging features may significantly impact the simulation's performance, but they do not affect the execution itself.

Note

Commands listed below assume that you have a platform loaded, a CPU node named `cpu` and that the using `sysbus` command was executed. For more details, see documentation chapters on *Renode Script syntax* and *Accessing and manipulating peripherals*.

24.1 Logging executed function names

Renode can log the names of functions currently executed by the guest application. This requires you to use the `sysbus LoadELF @path/to/elf` command to *load your application* or `sysbus LoadSymbolsFrom @path/to/elf` if you prefer executing a binary or a hex file. Function name logging can be enabled using:

```
cpu LogFunctionNames true
```

You can disable function name logging using:

```
cpu LogFunctionNames false
```

Note

You can also add another `true` at the end of this command to remove duplicate function names from subsequent code blocks and achieve better overall performance.

To filter function names based on a prefix, add the prefix as a string at the end of the function:

```
cpu LogFunctionNames true "uart"
```

To use more than one prefix and log functions that start with either of these two prefixes, simply separate them with a space:

```
cpu LogFunctionNames true "uart irq"
```

Note

Renode will try to demangle C++ function names. It will also use the names stored in the ELF file.

If you want to learn more about logging executed function names in Renode, visit the *Using the logger chapter*

24.2 Logging peripheral accesses

While the information about the executed functions gives you an overall understanding of the execution, it is often beneficial to know the additional context and understand why the application took a specific path.

To provide this additional context, Renode can log access to peripherals. This feature allows you to see how your program uses or doesn't use specific parts of the SoC.

You can enable this feature by using the following:

```
sysbus LogPeripheralAccess <peripheral-name> true
```

Note

In most cases, your peripheral names need a prefix `sysbus` like `sysbus.uart0`. You can omit this prefix if you're using the `using sysbus` command in your script.

The logs contain:

- the name of the peripheral
- current value of the program counter
- type and width of the access
- offset of the access (relative to the peripheral's base address) and the name of the register that this offset maps to
- the value that was either loaded to or returned from the register

You can also log accesses to all peripherals connected to the system bus:

```
sysbus LogAllPeripheralsAccess true
```

You can disable both peripheral access logging commands by providing `false` instead of `true` as the last parameter:

```
sysbus LogAllPeripheralsAccess false
```

If you want to learn more about logging peripheral accesses in Renode, visit the *Using the logger chapter*

24.3 Execution tracing

In Renode, you can see what the CPU does at any given time without changing the code or using specialized hardware. To enable execution tracing, use:

```
cpu CreateExecutionTracing "tracer_name" @path-to-file <mode>
```

mode can be one of the following values:

- PC - this mode saves all program counter values. Example:

```
0x20400000
0x20400004
0x20400008
0x2040000c
0x20401bc4
0x20401bc8
...
```

- Opcode - this mode saves all executed opcodes. Example:

```
0x0297
0x1028293
0x30529073
0x3B90106F
0x5FC00297
0x88C28293
...
```

- PCAndOpcode - this mode saves program counter values and the corresponding opcode that was executed. Example:

```
0x20400000: 0x0297
0x20400004: 0x1028293
0x20400008: 0x30529073
0x2040000c: 0x3B90106F
0x20401bc4: 0x5FC00297
0x20401bc8: 0x88C28293
...
```

- Disassembly - in addition to the value of the program counter and the corresponding opcode, this mode uses a built-in LLVM-based disassembler to convert opcodes to human-readable instruction names with all of the used arguments. The output also includes the name of the symbol that this entry belongs to. Example:

```

0x20400000: 00000297 auipc t0, 0          [vinit (entry)]
0x20400004: 01028293 addi t0, t0, 16       [vinit+0x4 (guessed)]
0x20400008: 30529073 csrw mtvec, t0      [vinit+0x8 (guessed)]
0x2040000c: 3b90106f j 7096             [vinit+0xC (guessed)]
0x20401bc4: 5fc00297 auipc t0, 392192     [__start (entry)]
0x20401bc8: 88c28293 addi t0, t0, -1908    [__start+0x4 (guessed)]
...

```

You can save the output from the PC, Opcode, and PCAndOpcode modes to a binary format that can optionally be compressed. This format is faster to encode and produces smaller output files. To save to a binary file, use the following:

```
cpu CreateExecutionTracing "tracer_name" @path-to-file <mode> true
```

If you also want to compress the output, you can add another true to this command:

```
cpu CreateExecutionTracing "tracer_name" @path-to-file <mode> true true
```

You can view the content of the binary file by using a script bundled with Renode. It can be invoked by running this command in your shell:

```
python3 <renode>/tools/execution_tracer/execution_tracer_reader.py inspect path-
↪to-dump-file
```

This command will print the file's text content to the standard output.

To disable execution tracing, simply use:

```
cpu DisableExecutionTracing
```

24.3.1 Tracer add-ons

You can use the tracer to track additional data. To do so, type:

```
tracer_name <tracker_name>
```

where tracker_name can be one of the following:

- TrackMemoryAccesses - tracks memory accesses.
- TrackVectorConfiguration - tracks vector configuration for the RISC-V architecture.
- TrackRiscvAtomics - tracks Zaamo instruction operands for the RISC-V architecture.
- TrackRegisters <registers...> <mask=0x0> <value=0x0>
<traceRegistersBefore=true> - tracks register values.
 - <registers...> can take either register names or indices. If the <registers...> field is omitted, then all CPU registers will be tracked.
 - <mask> and <value> fields are responsible for specifying instructions for which registers will be written out. Instructions whose opcode matches according to opcode & mask == value will be traced. For example, mask = 0b1111_0000 and value = 0b1010_0000 will match opcodes such as 0b1010_0000, 0b1010_1111 or 0b1010_1001

but will not match `0b1000_0000` nor `0b1111_1111`. By default, `mask = 0` and `value = 0` meaning registers will be written out for every instruction.

- `<traceRegistersBefore>` allows to enable or disable printing of registers before opcode executes. It is enabled by default, but it might be useful to disable in case we're matching all opcodes, so when `mask == 0` and `value == 0`.

Usage Example:

```
tracer_name TrackRegisters ["R0", "R1", "R3"] 0xE 0xE
```

Will print out state of registers R0, R1 and R3 before and after opcode starting with `0b111` executes.

24.4 Usage of gathered data

Certain tools shipped with Renode can use the obtained execution traces for post-mortem analysis of a program's execution. These include:

- the *Coverage Report Generator* which can be used to generate code coverage reports,
- the *Guest CPU Cache Modelling tool* which simulates CPU caches and generates usage statistics.

24.5 Specialized uses of execution tracing

In addition to generic program execution tracing, Renode also has more specialized tracing-related facilities, such as:

- *counting executed opcodes*
- *generating interactive flamegraphs of the guest's execution trace*
- *obtaining execution metrics*

Refer to the *Execution metrics, profiling and opcode counting* section for more details.

EXECUTION METRICS, PROFILING AND OPCODE COUNTING

Renode lets you tailor the execution tracking functionality to meet specific needs in terms of metrics, profiling and opcode counting.

25.1 Execution metrics

Renode can gather and show you metrics in the form of graphs. The metrics that are available out of the box are:

- Executed instructions
- Exceptions
- Memory access
- Peripheral access

If you want to learn more about this feature and see it in action, visit the *Metrics analyzer chapter*.

25.2 Guest application profiling

In Renode, you can display a trace of the guest application's execution in the form of an interactive flame graph. You can use this to visualize the flow of the application, analyze the relative time taken by each function, and discover potential issues with the implemented functionality or its performance.

Note

This feature is available for RISC-V, Cortex-A, Cortex-R and Cortex-M CPUs.

To enable guest profiling, use:

```
cpu EnableProfiler <output-format> @path-to-output-file [<flush-instantly>]
```

Renode supports two output formats:

- CollapsedStack - a text-based, collapsed stack format that can be viewed interactively using [speedscope](#)
- Perfetto - a binary format that can be viewed using [Perfetto](#)

You can also use the `cpu EnableProfilerCollapsedStack` and `cpu EnableProfilerPerfetto` commands for the same effect.

By default, Renode will buffer file operations for better performance, but you can instantly write everything to a file by passing `true` as the optional `<flush-instantly>` argument. If you don't want to write to a file instantly but still want to have some control over when a flush occurs, use the following command to flush the buffer manually:

```
cpu FlushProfiler
```

You can inspect traces generated from Zephyr samples in the [System Designer](#) platform - see examples for [RISC-V](#) and [Cortex-M](#).

25.3 Opcode counting

Renode can count how many times a specific instruction was executed. This can be helpful if you want to know if some special instructions, for example, vector instructions, are executed in your program.

To enable opcode counting, use the following command:

```
cpu EnableOpcodesCounting true
```

Then, you need to install a pattern of the opcode you want to trace.

Opcode patterns have to be built from the following characters:

- 1 matches set bits
- 0 matches unset bits
- all other characters match any value

The patterns resemble a regex that matches the opcode's bits. As an example, you can use the [scripts/single-node/versatile.resc](#) demo to detect ARM's branch and branch-with-link instructions. To load this demo, use:

```
include @scripts/single-node/versatile.resc
```

To install the counter patterns, this example uses the following two commands, but you can search for any pattern you want:

```
cpu InstallOpcodeCounterPattern "b1" "****1011*****"
cpu InstallOpcodeCounterPattern "b"  "****1010*****"
```

To print the values of the counters to the Monitor window, use:

```
cpu GetAllOpcodesCounters
```

The first parameter is the name of the counter, and the second is the pattern. After running the demo, you can use:

```
-----
|Opcode|Count  |
-----
```

(continues on next page)

(continued from previous page)

```
|b1      | 376816 |
|b       | 9587263|
-----
```

You can also get the value of a specified counter using:

```
cpu GetOpcodeCounter "<counter-name>"
```

25.3.1 Installing RISC-V opcode patterns

Renode offers predefined functions for RISC-V to install the patterns for you. To install RISC-V specific opcode patterns, use the functions below:

- `cpu EnableRiscvOpcodesCounting` - to install patterns for general instructions
- `cpu EnableCustomOpcodesCounting` - to install patterns for custom RISC-V instructions
- `cpu EnableVectorOpcodesCounting` - to install patterns for RISC-V vector instructions

These patterns are created from RISC-V opcode definition [files](#). You can create a similar file for RISC-V with your instructions and load the patterns using:

```
cpu EnableRiscvOpcodesCounting @path-to-file
```

GENERATING A COVERAGE REPORT

Renode offers many execution tracing features, as described in the *Execution tracing* section, and allows you to dump the entire execution of a binary into a single file. With an execution trace of a properly built binary and a dedicated script, you can create a code coverage report.

The `script` combines data in the `DWARF format` (debug information available in ELF files which are typically used to load software to Renode) with the number of executions of each instruction counted by Renode. The following tutorial will walk you through the steps necessary to generate such a report in Renode.

26.1 Prerequisites

To generate a report, you need to have Renode installed along with the `execution tracer script`. On Linux you can simply download a portable package by following [these instructions](#).

It's assumed that you execute the rest of commands in the root directory of the Renode repository. Otherwise, you might need to adjust them accordingly.

To run the script you can install it as a CLI utility, for example using `pipx`:

```
pipx install tools/execution_tracer/
```

You will then be able to use call:

```
renode-retracer --help
```

to see the list of available commands

Alternatively, it's possible to invoke the script directly from the Renode directory. Let's assume that the variable `RENODE_PATH` points to your Renode installation, or cloned repository. First, initialize the virtual environment and install the dependencies:

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r ${RENODE_PATH}/tools/execution_tracer/requirements.txt
```

Then, you should be able to run:

```
python3 ${RENODE_PATH}/tools/execution_tracer/execution_tracer_reader.py --help
```

to see the list of available commands.

26.2 Building a binary for coverage gathering

Renode can trace the execution of a binary built in any way, but to extract coverage information we require a binary with debug information in the DWARF format. To provide the best result, it's recommended to use the `-g` and `-O0` (or `-Og`) flags (for GCC) during compilation. Please note that different compilers may require different options.

The `-g` switch adds debug information to the binary, specifically code line numbers for each machine instruction. The coverage functionality requires that information, and you will see an error if it's missing from the ELF file. It's recommended to use the highest available debug level (at this time `-g3`).

Using the `-O0` switch to disable all optimizations is strongly recommended. The `-Og` switch disables most optimizations and is intended to make the debugging process easier.

Any optimization done by a compiler may prevent redundant lines of code from being executed. The compiler might also replicate the same code line across several addresses in the program or reorder the execution flow, which will make obtaining legitimate results impossible. For the purpose of generating a coverage report, same as for debugging, it's recommended to execute the code without any optimizations. Generating a report for an optimized binary may create imprecise or even unexpected results.

26.3 Using PC-to-Line# mapping files instead of DWARF info

The execution tracing script also supports using a file describing direct PC to line number mappings. This can be used when integrating retracer with other tools where DWARF information is not available.

The format is very simple: each line contains a PC value in hexadecimal, a filename/filepath, and a line number. The PC value is followed by a space, and the filepath and line number are separated with a single `..`. The file can either be just the filename, or a full file path. Below are two examples of the format, one with short file names and the other with full paths.

```
0x800003d8 k210_ops.cpp:252
0x800003da iostream:74
0x800003de iostream:74
0x80000bb6 main.c:6
0x80000bb8 main.c:6
0x80000bba main.c:6
0x80000bbc main.c:6
0x80000bc0 main.c:6
```

or with full filenames

```
0x800003d8 /home/dev/repositories/kendryte-standalone-sdk/lib/nncase/v0/runtime/
↳k210/k210_ops.cpp:252
0x800003da /home/dev/repositories/kendryte-standalone-sdk/toolchain/riscv64-
↳unknown-elf/include/c++/8.2.0/iostream:74
0x800003de /home/dev/repositories/kendryte-standalone-sdk/toolchain/riscv64-
↳unknown-elf/include/c++/8.2.0/iostream:74
0x80000bb6 /home/dev/repositories/kendryte-standalone-sdk/src/test2/main.c:6
0x80000bb8 /home/dev/repositories/kendryte-standalone-sdk/src/test2/main.c:6
0x80000bba /home/dev/repositories/kendryte-standalone-sdk/src/test2/main.c:6
```

(continues on next page)

(continued from previous page)

```
0x80000bbc /home/dev/repositories/kendryte-standalone-sdk/src/test2/main.c:6
0x80000bc0 /home/dev/repositories/kendryte-standalone-sdk/src/test2/main.c:6
```

To use one of these mapping files for coverage generation, invoke retracer with `--pc2line sample.pclin` instead of `--binary sample.elf`

26.4 Gathering coverage for a sample program

For the purpose for this tutorial, we prepared a sample program consisting of two files. The tutorial describes how to gather the execution metrics for this program, running on the emulated **Kendryte K210 RISC-V platform**.

Note

The pre-built sample and sources are available for download [here](#).

The program's sources look as follows:

```
const int buf_size = 100;

void funB(int *buf, int b) {
    for (int i = 0; i < buf_size; i++) {
        buf[i] -= b;
    }
}

void funA(int *buf, int b);
void funC();

void buf_increment_all(int *buf) {
    for (int i = 0; i < buf_size; i++) {
        buf[i] += 1;
    }
}

int main() {
    int buf[buf_size];
    for (int i = 0; i < buf_size; i++) {
        buf[i] = i;
    }

    buf_increment_all(buf);

    for (int i = 0; i < buf_size; i += 1) {
        if (buf[i] % 3 == 0) {
            funA(buf, i);
        }
        if (buf[i] % 5 == 0) {
```

(continues on next page)

(continued from previous page)

```
        funB(buf, i);
    }
}

int i = 11;

if (i < 10) {
    funC();
}

buf_increment_all(buf);

while (1) ; // Don't let the program run-off!

return 0;
}
```

```
extern const int buf_size;

int funC() {
    return 42;
}

void funA(int *buf, int b) {
    for (int i = 0; i < buf_size; i++) {
        buf[i] += b;
    }
}
```

To compile it, first you need to obtain a riscv64 toolchain. Refer to the package repositories of your OS distribution, or compile it yourself from the [sources](#).

Note

Hint: You can use prebuilt toolchains, shipped with the [Zephyr SDK](#).

Then, compile the program with (adjust the compiler name, as needed):

```
riscv64-unknown-elf-gcc -O0 -g3 main.c additional.c --freestanding -nostdlib -Wl,-  
↪emain -o coverage-sample.elf
```

The additional flags inform the compiler that we operate in a bare-metal environment, so we don't want to link with the standard libraries. They have no impact on the coverage report.

26.4.1 The platform and tracing the execution

To trace execution of the binary from the previous section, you can simply use the [pre-prepared RESC script](#).

Note

The script runs a binary pre-built for the Kendryte K210 platform, but the tracing mechanism itself (CreateExecutionTracing) is generic and works for all platforms.

The script is functionally similar to this one:

```
$bin=$CWD/coverage-sample.elf
include @scripts/single-node/kendryte_k210.resc
cpu2 IsHalted true
cpu1 SP 0x1000

cpu1 CreateExecutionTracing "trace" $CWD/trace.bin.gz PC isBinary=True
↪compress=True
```

We focus on a single core and disable the other one. We also set the Stack Pointer manually, to simplify the software.

You can load the script in Renode, by typing:

```
include @scripts/complex/coverage/kendryte_k210_coverage.resc

emulation RunFor "0.003" # Run 300 000 instructions (default performance is
↪100MIPS)
quit
```

The command above gathers the trace for 0.003 seconds of the guest's time. After the execution finishes, close the Renode instance with `quit`. The trace will be saved in a binary format to a file named `trace.bin.gz`. The file is additionally compressed to reduce size.

The `CreateExecutionTracing` command initializes execution tracing. The *Execution tracing* section contains details on configuring the command. The script that generates the coverage report requires at least PCs (addresses of executed instructions) to be available in the trace file.

Note

Tracing an additional type of data may cause an increase in the size of the output file. It's also recommended to use a binary format and compression by setting the fourth and fifth arguments to `True`, as shown above.

26.4.2 Generating the report

At this point, you can run the script that generates the report:

```
renode-retracer coverage trace.bin.gz \
--binary coverage-sample.elf \
--sources main.c additional.c \
--output coverage.zip \
--export-for-coverview
```

Note

If no sources are provided using the `--sources` argument, the script will attempt to automatically discover their locations based on the DWARF data extracted from the binary.

You might need to perform path substitution if the sources' locations have changed from the time when the binary was built (or binaries were built on a different machine).

To do this, you can use the `--sub-source-path` argument by providing `old_path:new_path` for each pair of paths to be substituted; this argument can be provided multiple times.

By adding `--export-for-coverview` to the command line, the tool can pack an archive, ready to be processed by *Coverview* - a tool for generating coverage dashboards. For the `main.c` file, the output will look similar to the one shown below.

```

18  1/1      :      int main() {
19  1/1      :          int buf[buf_size];
20  1/1      :          for (int i = 0; i < buf_size; i++) {
21  1/1      :              buf[i] = i;
22          :          }
23          :
24  1/1      :          buf_increment_all(buf);
25          :
26  1/1      :          for (int i = 0; i < buf_size; i += 1) {
27  1/1      :              if (buf[i] % 3 == 0) {
28  1/1      :                  funA(buf, i);
29          :              }
30  1/1      :              if (buf[i] % 5 == 0) {
31  1/1      :                  funB(buf, i);
32          :              }
33          :          }
34          :
35  1/1      :          int i = 11;
36          :
37  1/1      :          if (i < 10) {
38  0/1      :              funC();
39          :          }
40          :
41  1/1      :          buf_increment_all(buf);
42          :
43  1/1      :          while (1) ; // Don't let the program run-off!
44          :
45          :          return 0;
46          :      }

```

It's also possible to use a format (`.info`) compatible with *LCOV*. While not as easy to read for a human, it's more easily processed by automated scripts and tools.

26.5 Report formats

Normally, the script will output the data in a format (`.info`) compatible with *LCOV*. The format is described in the manual pages for `geninfo` (`man geninfo`). `renode-retracer` supports generation of line coverage info (DA). This format can be later used with various third-party tools (e.g. `genhtml`) to display the coverage data.

By using the `--export-for-coverview` switch, the data will be packed into an archive, ready to be processed by *Coverview*.

`renode-retracer` also supports a legacy text-based report format for easy result interpretation, as shown in *the last section*.

26.6 Coverview integration

The execution tracer can export the data in a format which can be directly loaded into Coverview. Use the `--export-for-coverview` switch for this and remember to provide an output file name with the `--output filename.zip` option.

Coverview is a tool for generating coverage dashboards. The tool runs fully on the client's side, and you can build it locally via npm or you can visit a page [deployed on GitHub pages](#). You can upload the archives obtained from the scenarios presented in this chapter, and investigate the line coverage. No uploaded data is stored on the server.

You can then load the archive into the dashboard and browse it file by file.

26.7 Gathering Zephyr's coverage

This example uses the script to gather coverage of a Zephyr's app running on an emulated **Nucleo H753ZI**. Follow along to prepare a test scenario and see how much code is covered by the test.

First, download Zephyr and set up your environment as described [in the official guide](#).

After following the steps as described in the Getting Started section, building a shell sample requires just one command:

```
west build -b nucleo_h753zi zephyr/samples/subsys/shell/shell_module/ -- -DCONFIG_
↪NO_OPTIMIZATIONS=y
```

Notice that the optimizations are disabled completely to increase the quality of gathered coverage info. Alternatively you could build the sample with just the debug optimizations enabled (`-DCONFIG_DEBUG_OPTIMIZATIONS=y`). After running west you can find the compiled `zephyr.elf` file in the `build/zephyr/` directory - the shell application for which want to obtain coverage info.

Below is a deterministic Robot test file that tests the coverage of our case:

```
*** Test Cases ***
Should Report Shell Coverage
    Execute Command      include @platforms/boards/nucleo_h753zi.repl
    Execute Command      sysbus LoadELF $CWD/build/zephyr/zephyr.elf
    Execute Command      cpu CreateExecutionTracing "trace" @${CURDIR}/trace.
↪bin.gz PC True True
    Create Terminal Tester sysbus.usart3 defaultPauseEmulation=True

    Wait For Prompt On Uart  uart:~$
    Write Line To Uart       demo board
    Wait For Line On Uart    nucleo_h753zi
    Wait For Prompt On Uart  uart:~$
```

In this sample, we await the prompt which signals that the shell is ready to receive data. Then we query the shell module for the board name, and again await the prompt, after the command finishes. Immediately after receiving the response on the UART, the emulation stops, and the execution trace is dumped into the file.

To run the test, execute `renode-test path/to/test.robot`.

The trace will be saved into a `trace.bin.gz` file, located in the same directory as the Robot test file.

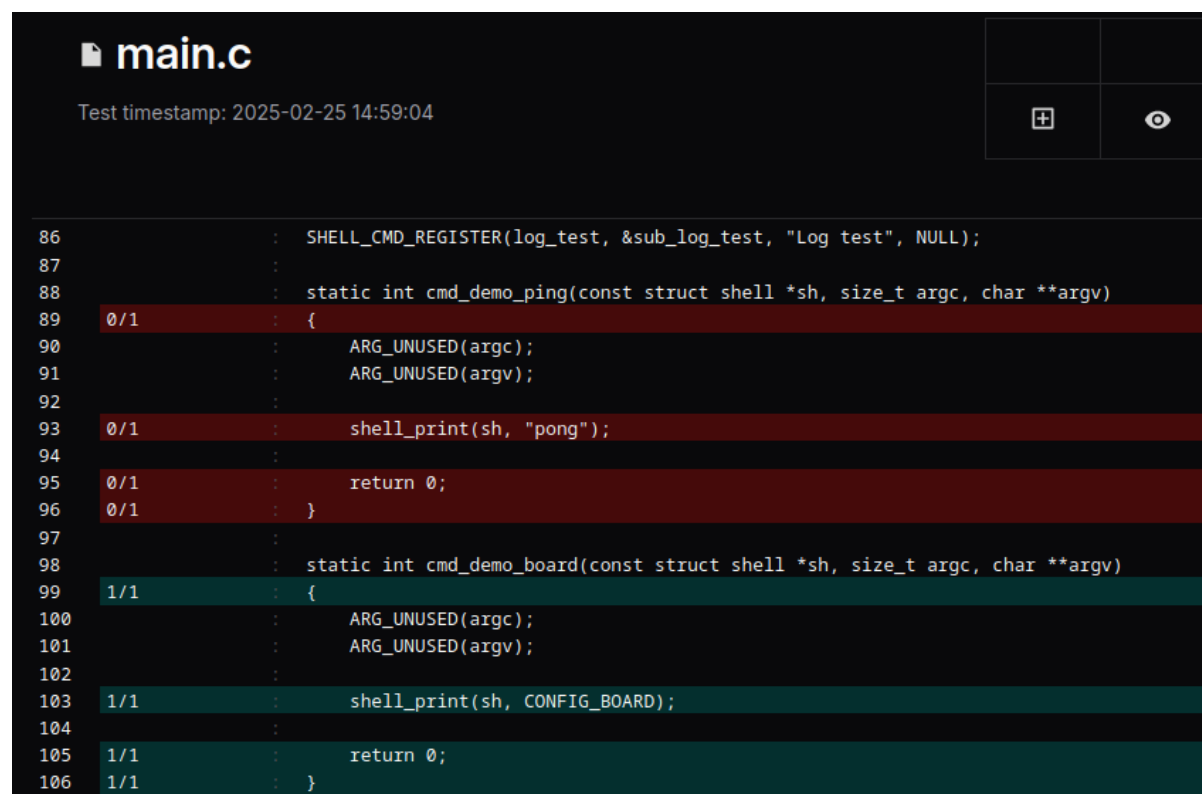
To create a coverage report, execute the following command:

```
renode-retracer coverage trace.bin.gz \
  --binary build/zephyr/zephyr.elf \
  --output coverage.zip \
  --export-for-coverview
```

Intentionally, no sources are provided. The script will discover and load them automatically, provided that they still exist on the current machine, where the build happened. Otherwise, you might need to adjust paths with `--sub-source-path`.

Generating coverage data can take some time to process, depending on the application's code-base size and the execution time of the program (size of the trace).

Afterwards, the archive can be loaded into *Coverview* and browsed interactively. For the code responsible for printing the board name (`samples/subsys/shell/shell_module/src/main.c`), it might look like so:



26.8 Legacy report mode

It's also possible to use a simple text-based mode for investigating coverage. To do so, pass the `--legacy` switch. For example, to obtain coverage data for `main.c` from the *sample application* described above, it's possible to do the following:

```
renode-retracer coverage trace.bin.gz \
  --binary coverage-sample.elf \
```

(continues on next page)

(continued from previous page)

```
--sources main.c \  
--legacy
```

The output should start with the lines, as shown below. The number before the colon indicates the number of executions of each line.

```
0:  const int buf_size = 100;  
0:  
28:  void funB(int *buf, int b) {  
2828:      for (int i = 0; i < buf_size; i++) {  
2800:          buf[i] -= b;  
0:      }  
28:  }  
0:  
0:  void funA(int *buf, int b);  
0:  void funC();  
0:  
2:  void buf_increment_all(int *buf) {  
202:      for (int i = 0; i < buf_size; i++) {  
200:          buf[i] += 1;  
0:      }  
2:  }  
...
```

GUEST CPU CACHE MODELLING

Renode can perform a post-mortem analysis of memory accesses (generated by the execution tracing subsystem) to simulate CPU cache behavior and generate usage statistics.

To generate memory access traces from a simulation, add the following lines to the resc file:

```
cpu MaximumBlockSize 1
cpu CreateExecutionTracing "tracer" $ORIGIN/trace.log PCAndOpcode
tracer TrackMemoryAccesses
```

Note

Replace `cpu` with the core name (from the `repl` file) that you want to analyse.

You can then pass the generated `trace.log` file to the [Renode Cache Modeling Analyzer](#). Note that you need to install the dependencies listed in `requirements.txt` before running the analyzer.

You can use the analyzer with its built-in presets:

```
./renode_cache_interface.py trace.log presets 'fu740.u74'
```

The presets are stored in the `presets.py` file. To create a new preset add a new entry:

```
'new_preset': {
    'l1d': Cache(
        name='l1i,name',          # Cache name - used in the `printf` debug_
↪helpers.
        cache_width=15,          # number of bits used to address the cache
        block_width=6,           # number of bits used in a cache block
        memory_width=64,         # number of bits used to address the main_
↪memory
        lines_per_set=2,         # 2 way associativity
        replacement_policy="FIFO" # replacement policy
    ),
    'flush_opcodes': {
        0x1000: 'd',             # flush `l1d` when after the `0x1000` opcode_
↪is executed
    },
    'invalidate_on_io': True     # flush the data cache when an MemoryMapped I/
```

(continues on next page)

(continued from previous page)

```
→0 operation is performed  
}
```

This will create a preset with data cache only. You can add Instruction cache by adding the `l1i` object. For more information about cache configuration using presets, please refer to the documentation in the `cache.py` file. Currently, only Level 1 instruction and data caches are supported.

You can also configure the cache using the CLI parameters:

```
./renode_cache_interface.py trace.log config \
    --memory_width 64 \
    --l1i_cache_width 15 \
    --l1i_block_width 6 \
    --l1i_lines_per_set 4 \
    --l1i_replacement_policy LRU \
    --l1d_cache_width 15 \
    --l1d_block_width 6 \
    --l1d_lines_per_set 8 \
    --l1d_replacement_policy LRU
```

For more information about cache configuration using CLI, please refer to the output of the `./renode_cache_interface.py trace.log config --help` command.

A sample output of the analysis:

```
$ ./renode_cache_interface.py trace.log presets fu740.u74  
l1i,u74 configuration:  
Cache size:      32768 bytes  
Block size:      64 bytes  
Number of lines: 512  
Number of sets:  128 (4 lines per set)  
Replacement policy: RAND  
  
l1d,u74 configuration:  
Cache size:      32768 bytes  
Block size:      64 bytes  
Number of lines: 512  
Number of sets:  64 (8 lines per set)  
Replacement policy: RAND  
  
Instructions read: 174620452  
Total memory operations: 68952483 (read: 50861775, write 18090708)  
Total I/O operations: 1875 (read: 181, write 1694)  
  
l1i,u74 results:  
Misses: 168  
Hits: 174620284  
Invalidations: 3  
Hit ratio: 100.0%
```

(continues on next page)

(continued from previous page)

```
l1d,u74
Misses: 17320212
Hits: 51632271
Invalidations: 17319700
Hit ratio: 74.88%
```

It is also possible to generate an output JSON file by passing the `--output <filename>` flag:

```
{
  "l1i,u74":{
    "hit":174620284,
    "miss":168,
    "invalidations":3
  },
  "l1d,u74":{
    "hit":51632271,
    "miss":17320212,
    "invalidations":17319700
  }
}
```

SETTING UP A WIRED NETWORK

Renode allows you to create complex network topologies using the Monitor.

Both wired and *wireless* networks can coexist, but are created with different commands.

28.1 Creating a switch

Wired network interfaces can be connected via switches. Depending on the desired topology, you can create one or more switches and attach interfaces accordingly.

The network traffic is not affected by the number of nodes connected to one switch.

To create a switch named `switch1`, run:

```
(monitor) emulation CreateSwitch "switch1"
```

28.2 Connecting interfaces

To connect an interface to a switch you have to set a proper *machine context*.

Then, use the connector mechanism to attach the interface:

```
(machine-0) connector Connect sysbus.ethernet switch1
```

Although it is not a common setup, each interface can be connected to many switches at the same time.

28.3 Disconnecting interfaces

You can disconnect network interfaces from a switch by running:

```
(machine-0) connector Disconnect sysbus.ethernet switch1
```

To disconnect from all connected switches, use:

```
(machine-0) connector DisconnectFromAll sysbus.ethernet
```

28.4 Starting the interface

A Switch object is created as “paused”. To enable communication via a switch you must start it manually, either by running:

```
(monitor) start
```

or, if your emulation is already started, with:

```
(monitor) switch1 Start
```

28.5 Controlling the traffic

If a packet has a specified destination MAC address, the switch will try to deliver it to the appropriate interface. If, however, the address is not set or the switch is not aware of an interface with this MAC, the packet is broadcast to all connected interfaces.

You can alter this behavior by enabling promiscuous mode for a specified interface:

```
(machine-0) switch1 EnablePromiscuousMode sysbus.ethernet
```

After this command, the `sysbus.ethernet` interface will receive all packets delivered to `switch1` from other interfaces.

To disable promiscuous mode, run:

```
(machine-0) switch1 DisablePromiscuousMode sysbus.ethernet
```

SETTING UP A WIRELESS NETWORK

Connecting nodes in a wireless network in Renode is as easy as creating *wired networks*.

Just like with a Switch object, you can create an abstraction of Wireless Medium, to connect radio interfaces to.

29.1 Creating a wireless medium

Renode allows you to create multiple virtual wireless media, separated from each other. The exchanged packets are not transferred between these media, so you can treat this mechanism as means for physical separation which may help you with debugging of your wireless setup.

The network traffic is not affected by the number of nodes connected to one wireless medium.

Renode supports two types of wireless media: IEEE802_15_4Medium and BLEMedium. Each one creates an abstraction of different type of wireless connection. They're using IEEE802_15_4 standard and Bluetooth Low Energy accordingly.

To create an IEEE802_15_4 medium called wireless, run:

```
(monitor) emulation CreateIEEE802_15_4Medium "wireless"
```

Analogously for a BLE medium:

```
(monitor) emulation CreateBLEMedium "wireless"
```

29.2 Connecting interfaces

To connect an interface to a wireless medium you have to set a proper *machine context*.

Then, use the connector mechanism to attach the interface:

```
(machine-0) connector Connect sysbus.radio wireless
```

Although it is not a common setup, each interface can be connected to many media at the same time.

29.3 Disconnecting interfaces

You can disconnect network interfaces from a wireless medium by running:

```
(machine-0) connector Disconnect sysbus.radio wireless
```

To disconnect from all connected wireless media, use:

```
(machine-0) connector DisconnectFromAll sysbus.radio
```

29.4 Positioning the nodes

A wireless medium uses 3D coordinates, without any specified unit of distance, to position the connected nodes.

To set a position of a node to coordinates $\{X = 3, Y = 5, Z = -8.5\}$, run:

```
(machine-0) wireless SetPosition sysbus.radio 3 5 -8.5
```

Again, the unit of these coordinates is not determined, so it's the user's responsibility of keeping them consistent in the emulation.

29.5 Controlling the traffic

By default, the wireless medium delivers all packets to all connected interfaces. This can be, however, configured, depending on your needs.

Renode exposes an abstraction of Medium Functions. Each medium function can accept a set of parameters to decide whether a packet exchanged between two nodes (knowing the positions of the sender and the receiver) will be successfully delivered or not.

Renode provides three medium functions by default:

- SimpleWirelessFunction

The default setting, delivering all packets to all connected nodes.

- RangeWirelessFunction

This function accepts a parameter indicating the maximal cartesian range between nodes.

If the nodes are within this range, all packets are delivered successfully. If they are not within range, the communication is not possible.

- RangeLossWirelessFunction

This function introduces a probabilistic loss of packets increasing gradually with the distance between nodes.

Given three parameters, lossRange (in distance units), txRatio and rxRatio (both ranging from 0 to 1.0, inclusively), each packet is subject to two tests.

The first test determines if the transmission is successful (with probability $p \gg txRatio$).

The second test takes the distance between the sender and the receiver and calculates the success ratio according the the formula: $1 - ((distance/lossRange) * (1 - rxRatio))$.

Note

Keep in mind that even though the `RangeLossWirelessFunction` relies on probability, you can still configure the RNG seed with `emulation.SetSeed` to preserve determinism of execution.

CONNECTING TO THE HOST NETWORK

Renode allows you to connect a host network interface to a *simulated wired network*.

To do that you need to create a Tap interface or VMNET interface based on `vmnet-helper` (the latter is supported only on macOS).

30.1 Regarding time flow

All devices simulated in Renode operate in *virtual time*, which is typically slower than the real time flow. Moreover, network packets are delivered in periodical synchronisation points, delaying the communication even further.

This means that time constraints (e.g. timeouts) placed by applications trying to connect to the simulated network from the host may need to be altered to take these delays into account.

30.2 Opening a TAP interface

To create and open a TAP interface which will be listed on your host system as `tap0` and inside Renode as `host.tap`, run:

```
(monitor) emulation CreateTap "tap0" "tap"
```

If you want the interface to be retained after Renode closes, add a `true` parameter:

```
(monitor) emulation CreateTap "tap0" "tap" true
```

Depending on your system configuration, you may be asked for a password to open the interface.

Note

The newly created interface needs to be enabled and configured on your host machine.

By default it has no IP address assigned and is in down state.

Please refer to your system documentation for further instructions.

30.3 Using TAP interface on Windows

In order to create a TAP device on Windows, you need to install a third-party driver that is a part of the [OpenVPN project](#). If OpenVPN is installed on your computer, Renode should detect it when trying to create a TAP interface. The feature has been specifically tested with OpenVPN 2.5.6.

Note

You need Administrator Privileges to create a TAP on Windows.

To create a TAP interface on your Windows PC, first, you need to create a new TAP interface in Renode using the usual command:

```
(monitor) emulation CreateTap "tap0" "tap"
```

Then you need to assign an IP address to your TAP using the command prompt:

```
netsh interface ipv4 set address name=tap0 static X.X.X.X
```

Note

You can also create a TAP interface directly using the tapctl.exe driver from OpenVPN:

```
tapctl.exe create --name tap0
```

Then assign it an IP address:

```
netsh interface ipv4 set address name=tap0 static X.X.X.X
```

Finally, connect it to Renode using:

```
(monitor) emulation CreateTap "tap0" "tap"
```

30.4 Connecting TAP interface to a switch

Assuming you have already *configured the simulated network*, you can connect a TAP interface to a switch device by running:

```
(monitor) emulation CreateSwitch "switch"  
(monitor) connector Connect host.tap switch
```

30.5 Using VMNET interface on macOS

30.5.1 Preparing vmnet-helper

Before setting up the interface in Renode, install the vmnet-helper program by running the following command:

```
brew install nirs/vmnet-helper/vmnet-helper
```

or, if you're on a macOS version earlier than 26, run:

```
curl -fsSL https://github.com/nirs/vmnet-helper/releases/latest/download/install.  
↪ sh | bash
```

30.5.2 Starting vmnet-helper

VMNET configuration consists of three steps:

1. Spawning a background vmnet-helper process:

```
vmnet_helper \  
  --socket=${SOCKET_PATH:-vmnet_helper.sock} \  
  --operation-mode=${mode:-shared} \  
  --start-address=${start_ipv4:-172.16.0.0} \  
  --end-address=${end_ipv4:-172.16.0.10} \  
  --subnet-mask=${mask:-255.240.0.0}
```

where:

- start-address: the IP address assigned to the host-side interface. This address also acts as the default gateway for the guests.
- end-address: the last IP address in the DHCP pool available for the guests.
- subnet-mask: the subnet mask defining the network size in dotted decimal notation (e.g. 255.255.255.0). The subnet mask enables vmnet to communicate with other vmnet interfaces on the same subnet that are in shared mode.
- operation-mode:
 - host creates an isolated network where the guest (the VM inside the Renode simulation) can communicate with the host (your computer) but with no access to the Internet. You can use this mode to increase security and privacy.

Note

The --start-address, --end-address, and --subnet-mask options are ignored in host operation mode in each version of vmnet-helper prior to **v0.12.0**.

- shared where the VM reaches the Internet through a network address translator (NAT) and the host's IP address.
- For full instructions on using the vmnet-helper process, refer to its [README](#).

Note

All provided IP addresses cannot be arbitrary; they must belong to the private IP address ranges defined by [RFC 1918](#)

2. Connecting Renode to the vmnet-helper process via a Monitor command:

```
emulation CreateVmnetHelper "${SOCKET_PATH}" "vmnet"
```

3. Connecting the machine interface and host.vmnet to the same switch to allow packet transmission between them:

```
emulation CreateSwitch "switch"
connector Connect host.vmnet switch
connector Connect gem0 switch # use your platform's interface name instead
↪ of `gem0`
```

If the guest runs Linux, you can test Internet connectivity in the following way:

```
ifconfig eth0 up $((start_ip+1)) # assign any address available in the pool (e.g.
↪ the first available address)
ping -c 5 ${start_ip} # any address accessible via the vmnet_helper interface (i.
↪ e. your local computer)
```

30.6 Connecting VMNET interface to a switch

Assuming you have already *configured the simulated network*, you can connect the VMNET interface vmnet1 to the switch device by running:

```
(monitor) emulation CreateSwitch "switch"
(monitor) connector Connect host.vmnet1 switch
```

30.7 Starting the interface

The TAP and VMNET interfaces are created as “paused”. To enable communication with the host system you must start it manually, either by running:

```
(monitor) start
```

or, if your emulation is already started, with:

```
(monitor) host.tap Start
```

```
(monitor) host.vmnet Start
```

30.8 Transferring files from host

If you successfully created a TAP interface, files can be transferred from the host computer to emulation using wget. To do it you will need to use an IP address associated with the TAP interface on the host machine, for example:

```
wget http://192.168.100.1/home/user/file.txt
```

Note

There are other methods of file transfer: *built-in TFTP server and Virtio*. Those methods are recommended if you want to avoid the limitations of the host-guest networking via TAP.

INSPECTING THE TRAFFIC WITH WIRESHARK

Wireshark is an open source network packet analyzer that can be used to sniff the network traffic between emulated nodes and/or the host network interface.

Renode uses the libpcap format to provide data to Wireshark.

31.1 Logging the whole traffic

Renode supports multiple link layer protocols, i.e. Ethernet, Bluetooth Low Energy, IEEE 802.15.4 and CAN. Each protocol has its own set of commands to enable logging of all traffic or with filtering.

The emulation `Log<Protocol Name>Traffic` commands will automatically connect Wireshark to all existing and new networks for the given protocol.

Table 31.1: Protocol specific global logging

tabulartyabulary

	Protocol		Com
	Ethernet		LogEthernet
	Bluetooth Low Energy		LogBLE
	IEEE 802.15.4		LogIEEE802_15_4
	CAN		LogCAN

You can also manually open Wireshark window, before setting up a network with:

```
(monitor) host.wireshark-all<Protocol Name>Traffic Run
```

31.2 Observing a specific interface

Inspecting traffic of a specific switch, bus or a wireless medium can be done with emulation `LogToWireshark` command. You can also limit the observation to a specific interface connected to that switch, bus or medium.

Table 31.2: Protocol specific filtered logging

tabulartyabulary

	Protocol		Interface ty
	Ethernet		Swit
	Bluetooth Low Energy		BLEMediu
	IEEE 802.15.4		IEEE802_15_4Mediu
	CAN		CANH

31.2.1 Ethernet example

To enable logging on a switch object run:

```
(monitor) emulation LogToWireshark switch
```

To observe only the sysbus.ethernet interface connected to switch run:

```
(machine-0) emulation LogToWireshark switch sysbus.ethernet
```

The names of Wireshark objects created depend on the machine name, the switch name and the interface name. In the above case Renode creates an object named host.wireshark-switch-machine-0-sysbus-ethernet.

MACHINE TO MACHINE CONNECTIONS

There are various ways of connecting two machines in Renode. Other than *wired* and *wireless* networking (described in their own chapters), there are several other interfaces available, divided into two categories:

- *Symmetrical, like UART or CAN,*
- *Asymmetrical, like GPIO or USB.*

32.1 Symmetrical connections

In Renode, connections like UART and CAN are always symmetrical, meaning each side can be either the initiator or the recipient of a transfer. A symmetrical connection is represented by a “hub” object to which the communicating machines need to be connected.

32.1.1 UART-based connections

To connect two UART devices, you need to create a UART hub, which can be created in the Monitor by executing:

```
(monitor) emulation CreateUARTHub "uartHub"
```

The UART hub created by this line is named `uartHub`, but you can use any name.

Then, you have to connect your UART devices to the common UART hub using the connector mechanism:

```
(monitor) mach set 0
(machine-0) connector Connect sysbus.uart uartHub
(machine-0) mach set 1
(machine-1) connector Connect sysbus.uart uartHub
```

To disconnect a device from the `uartHub`, execute:

```
(machine-1) connector Disconnect sysbus.uart uartHub
```

The UART hub is created in a paused state, which means it won't transfer any data between devices. To enable communication, you must start your hub.

For simplicity, in the default scenario your `uartHub` will start automatically when starting the whole simulation:

```
(monitor) start
```

In case you need to start your uarthub manually (e.g. when adding it to a running simulation):

```
(monitor) uartHub Start
```

Now, data sent by sysbus.uart on machine-0 will be received by sysbus.uart on machine-1 and vice versa.

Your hub can be paused at any moment by executing:

```
(monitor) uartHub Pause
```

It can then be resumed using:

```
(monitor) uartHub Resume
```

In order to ensure the determinism of the simulation, bytes sent over an uartHub are buffered and delivered to the receiver at set times. Please note that this might lead to slight delays in communication. The maximum delay is controllable by the quantum parameter. For details on how the synchronization works and how to configure it, refer to *the Time framework synchronization section* of the documentation.

Strict Mode

By default Renode only warns if UARTs with incompatible settings communicate.

Optionally you can enable StrictMode with:

```
(monitor) uartHub StrictMode True
```

In Strict Mode the UART Hub will not forward messages between UARTs with mismatched settings (i.e baud rate, parity type, or stop bits), and instead drop those messages. For UARTs with frame info handling (i.e. the model implements the IUARTWithFrameInfo interface) strict mode will instead forward all messages with frame info attached. Based on this information the peripheral model will handle error conditions and set appropriate flags and interrupts.

Currently frame info handling is supported on the following UARTs:

- **NXP LPUART**

32.1.2 CAN-based connections

Connecting two machines with the CAN bus is very similar to connecting them with a UART.

First, you need to create a CAN hub in the Monitor:

```
(monitor) emulation CreateCANHub "canHub"
```

Then, you have to connect both of your devices to the CAN Hub using the connector mechanism:

```
(monitor) mach set 0
(machine-0) connector Connect sysbus.can canHub
(machine-0) mach set 1
(machine-1) connector Connect sysbus.can canHub
```

To disconnect a device from the CANHub, execute:

```
(machine-1) connector Disconnect sysbus.can canHub
```

Note

Similar to the UART Hub, the CAN Hub can be freely started, paused, and resumed using the same commands.

32.2 Asymmetrical connections

In Renode, GPIO and USB connections are always asymmetrical. This requires you to specify which machine is the controller and which is the peripheral.

32.2.1 GPIO connections

GPIO connections in Renode allow you to send a boolean signal (true or false) between GPIO pins of two machines. The `GPIOConnector` object represents a unidirectional GPIO connection with a source and destination, which is why GPIO is listed as an asymmetrical connection method. However, you can create two `GPIOConnector` objects with reversed source and destination in order to effectively get a bidirectional connection.

To create a `GPIOConnector`, use:

```
(monitor) emulation CreateGPIOConnector "gpio-con"
```

You can connect a maximum of two machines to your GPIO Connector, one being INumber GPIO Output and the other being IGPIOReceiver. Trying to connect more will always result in an error.

Next, you need to select GPIO pins available in the machines you want to use in the connection. In the example below, we chose to use pin 7 in the source machine and pin 4 in the destination machine.

```
(monitor) mach set "source"
(source) connector Connect sysbus.gpio gpio-con
(source) gpio-con SelectSourcePin sysbus.gpio 7

(monitor) mach set "destination"
(destination) connector Connect sysbus.gpio gpio-con
(destination) gpio-con SelectDestinationPin sysbus.gpio 4
```

Note

Keep in mind that the signal can only be sent from the `SourcePin` to the `DestinationPin`. To create a connection working the other way around, please create another `GPIOConnector`.

32.2.2 USB connections

To be able to create a USB connection between two machines in Renode, first, you need to create a USBConnector.

```
(monitor) emulation CreateUSBConnector "usb-connector"
```

In this type of connection, your machines are assigned the device or controller role. Like in the other connection types, you need to use a connector mechanism to connect the usb-connector to the USB port on your machine. Your device then has to be registered in the controller device and connected to its USB.

```
(monitor) mach set "device"  
(device) connector Connect sysbus.usb usb-connector  
  
(device) mach set "controller"  
(controller) usb-connector RegisterInController sysbus.usb-controller
```

32.3 Modeling the timing of the events

Some models require an event to happen at a specified moment in time (e.g. some time after the other event). To achieve this, you can use various mechanisms:

- implementing a `LimitTimer` in a peripheral, that can execute actions periodically. This is used mainly, but not only, for implementing timer logic,
- `machine.LocalTimeSource.ExecuteInNearestSyncedState` to delay an event by some amount of time, which is not clearly specified,
- `machine.LocalTimeSource.ExecuteInSyncedState`, which gives you more control over the specific time at which the event should happen,
- `machine.ObtainManagedThread`, which is the easiest way to execute actions at a specified frequency.

All of these mechanisms are handled by the machine and its time source, so that the logic is hidden from the peripheral model.

More detailed information about the Time framework in Renode can be found in the relevant chapter: *Time framework*.

UART INTEGRATION

Renode enables exposing emulated UART devices to the host machine and interacting with them using the standard workflow (tools, scripts, etc.) exactly as if they were actual hardware serial ports. This opens the door to designing hybrid setups where part of the system is simulated in Renode and the rest consists of physical devices or software running in the “real world”.

Renode provides two independent mechanisms for exposing virtual UART devices to the host machine:

- using a pty device (Linux/macOS only),
- over a network socket (available on all platforms).

You can also redirect UART output to a file, but this does not allow for reading user input.

33.1 UART pty terminal

Note

This feature is available on Linux/macOS only.

UART pty terminal integration allows creating a pty device in the host filesystem that acts as a bridge between the real world and the simulation. Data written to/read from the pty device by the software running in the real world is automatically transferred to the virtual UART device (and vice versa).

In order to expose a virtual UART device, create the UART pty terminal with the following command in the Monitor:

```
(monitor) emulation CreateUartPtyTerminal "term" "/tmp/uart"
```

This will create a new file in the host filesystem (/tmp/uart) that can be referenced from within the simulation as term.

Now you need to *load your platform* and connect the newly created UART pty terminal to the simulated UART device:

```
(machine-0) connector Connect sysbus.uart0 term
```

This assumes your UART is called `sysbus.uart0`, but you might need to adjust it to match your platform.

Finally, open the terminal application on your host machine (screen/picocom/PuTTY/etc.) and attach it to the `/tmp/uart` file. You can interact with it just like with the hardware.

33.2 Socket terminal

Note

This feature is available on all supported host platforms (Linux/macOS/Windows).

Socket terminal integration allows exposing a virtual UART device over a network socket and making it available for any device in the network. The data sent to the socket available on a selected port number by the software running in the real world (on the local host machine or over the network) is automatically transferred to the virtual UART device (and vice versa).

In order to expose a virtual UART device, create a Socket terminal with the following command in the Monitor:

```
(monitor) emulation CreateServerSocketTerminal 12345 "term"
```

This will open a tcp network port 12345 on the host machine that can be referenced from within the simulation as `term`.

Now, connect the newly created Socket terminal to the simulated UART device:

```
(machine-0) connector Connect sysbus.uart0 term
```

Finally, open the terminal application on your host machine (netcat/telnet/PuTTY/etc.) and connect it to port 12345. You can interact with it just like with the hardware.

33.2.1 Emitting configuration bytes

By default Server Socket will emit the following initial configuration bytes in order to properly configure a newly connected terminal:

```
0xff, 0xfd, 0x00, // IAC DO    BINARY
0xff, 0xfb, 0x01, // IAC WILL  ECHO
0xff, 0xfb, 0x03, // IAC WILL  SUPPRESS_GO_AHEAD
0xff, 0xfc, 0x22  // IAC WONT  LINEMODE
```

In order to avoid generating them, pass additional `false` argument when creating the Socket terminal:

```
(monitor) emulation CreateServerSocketTerminal 12345 "term" false
```

33.3 Redirecting to a file

Renode can redirect UART output to a file on your host. To enable this feature, call:

```
(machine-0) uart CreateFileBackend @uart_file
```

By default, the UART output will be cached by the host IO system. If you want the output to be flushed immediately after being sent, use:

```
(machine-0) uart CreateFileBackend @uart_file_flush true
```

If you want, you can stop this output with:

```
(machine-0) uart CloseFileBackend @uart_file
```

Keep in mind that subsequent calls to the `CreateFileBackend` method will not overwrite the previous file of the same name, but rather copy, appending a consecutive number to its name.

CAN INTEGRATION

Note

This feature is available on Linux only.

Renode can connect to a virtual CAN interface on the host machine by using an internal SocketCAN bridge. This integration relies on [SocketCAN](#) for the communication between the host and internal CAN networks.

Using this standardized communication method allows Renode to transfer Classical and FD CAN frames. XL CAN frames are handled by the bridge, but they won't be forwarded to the internal network, as this frame type is not yet supported in Renode.

To see CAN host-integration in action, you can try the [SocketCAN bridge demo](#).

34.1 Host requirements

The bridge relies on SocketCAN to connect to a CAN interface, which includes native, virtual and SLCAN based interfaces. The examples below assume use of a virtual CAN interface, for which drivers are implemented in the vcan kernel module. To ensure that the module is present in the host, load it with:

```
$ sudo modprobe vcan
```

To create and set up a virtual network interface named vcan0, run:

```
$ sudo ip link add dev vcan0 type vcan  
$ sudo ip link set up vcan0
```

Note

Depending on the Linux distribution, the vcan module may not be included, or it may not support FD and/or XL CAN frames.

34.2 Creating a SocketCAN bridge

A SocketCAN bridge connects to an already existing CAN interface whose name can be provided as a `canInterfaceName` argument. If not provided, Renode will try to connect to the interface named `vcan0`.

In the simulation the bridge can be connected to internal networks as described in *CAN-based connections*.

By default, Renode will try to enable handling of FD and XL CAN frames, but it won't require them. To force the use of a specified type, set the optional arguments `ensureFdFrames` and `ensureXlFrames` to `true`. By setting those, regardless of the creation method, the command or a platform construction will fail if the ensured frame type cannot be enabled.

Warning

Be careful about creating a cycle in your CAN network topology.

The frames sent from one bridge can be received by another bridge via the common `vcan` interface. If those two bridges are part of the same network, then an infinite loop of packets may be created.

34.2.1 In the Monitor

As mentioned above, there are two ways of creating a `SocketCANBridge`. The first option is to use the `CreateSocketCANBridge` command.

For example, the following command will create a `SocketCANBridge` named `socketcan`, that connects to the `vcan1` interface and will fail if handling FD frames is not possible.

```
(machine-0) machine CreateSocketCANBridge "socketcan" "vcan1" ensureFdFrames=true
```

The bridge can be connected to CAN bus as described in *CAN-based connections*.

34.2.2 In the platform description

Another option is to add a `SocketCAN` bridge declaration to a `repl` file. The following example presents a snippet that sets all the arguments of a `socketcan` instance of the `SocketCANBridge` with default values.

```
socketcan: CAN.SocketCANBridge @ sysbus
  canInterfaceName: "vcan0"
  ensureFdFrames: false
  ensureXlFrames: false
```

The bridge can be connected to CAN bus as described in *CAN-based connections*.

ARDUINO IDE/CLI INTEGRATION

Renode supports integration with [Arduino IDE](#), making it possible to easily upload and run software targeting Arm-based Arduino boards (currently the Arduino Nano 33 BLE is supported) directly from the IDE. The same interface can also be used to upload binaries from the console using [Arduino CLI](#).

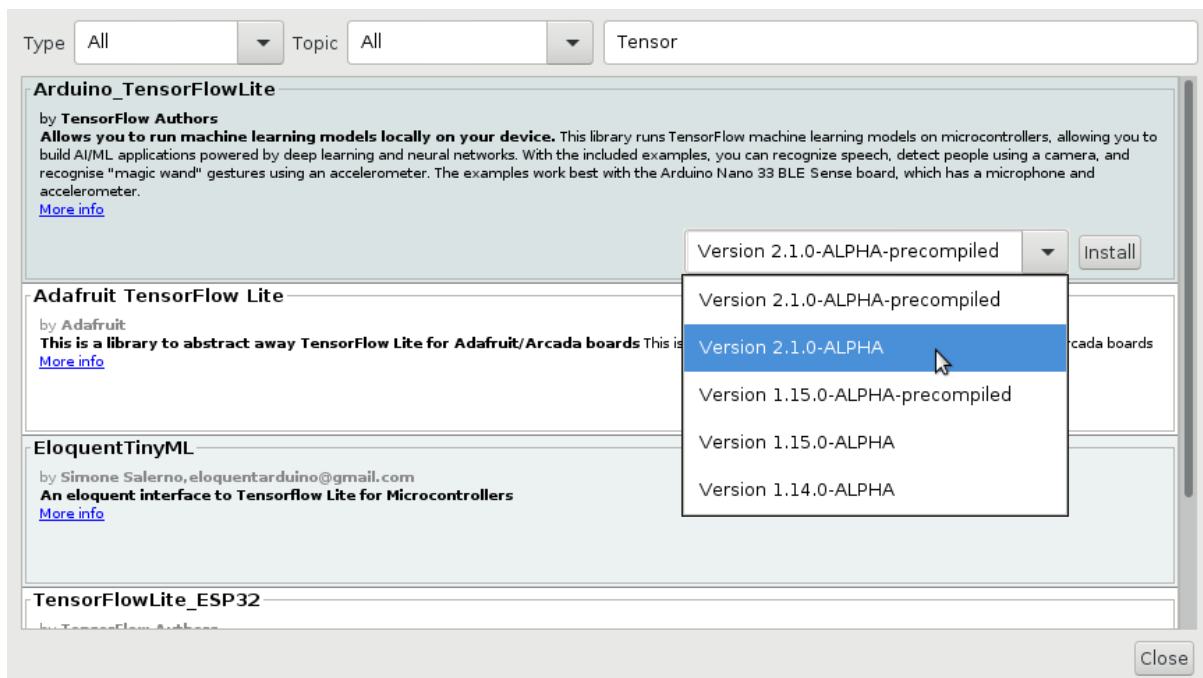
This document describes how to use the integration layer on a TensorFlow Lite's Hello World project example, but this should work for any custom project as well.

Note

The Renode-Arduino integration layer is available on Linux only.

35.1 Installing TensorFlow examples

TensorFlow Lite examples do not come preinstalled with Arduino IDE. In order to add them to your system, select Tools -> Manage Libraries and search for the “Arduino_TensorFlowLite” library. Select the 2.1.0-ALPHA version (**not** the precompiled one) and press the Install button:



At the moment, the support for USBSerial in Renode is still experimental, so you need to patch the library with the following command:

```
sed -i'' '/#define DEBUG_SERIAL_OBJECT/s/(Serial)/(Serial1)/' ~/Arduino/libraries/  
↪ Arduino_TensorFlowLite/src/tensorflow/lite/micro/arduino/debug_log.cpp
```

This will switch serial output from USBSerial to UART.

Note

Please make sure that the path to installed Arduino libraries is correct.

35.2 Configuring Renode

Renode provides the **ArduinoLoader** pseudo-device that is used to integrate with Arduino IDE/CLI. It acts as a USB device (CDC/ACM profile) and implements the arduino bootloader protocol (SAM-BA).

In order to connect Renode to Arduino IDE/CLI, perform the following steps:

1. Enable support for USB/IP in the host machine:

```
$ sudo modprobe vhci_hcd
```

2. Create the Arduino Nano 33 BLE platform in Monitor:

```
(monitor) mach create  
(machine-0) machine LoadPlatformDescription @platforms/boards/arduino_nano_  
↪ 33_ble.repl
```

Note

ArduinoLoader supports any platform based on a Cortex-M CPU.

3. Start the USB/IP server in Renode and attach the loader to it:

```
(machine-0) emulation CreateUSBIPServer  
(machine-0) host.usb CreateArduinoLoader sysbus.cpu 0x10000 0 "arduinoLoader"
```

Note

When creating the loader, you can specify the binary load address ([0x10000]{.title-ref} in the case of Arduino Nano 33 BLE board), the port through which the bootloader is connected to the [host.usb]{.title-ref} controller (should be changed if you have other devices already connected) and the name of the loader (arduinoLoader in this case).

The values below are the default ones, so you can skip all of them leaving just:

```
(machine-0) host.usb CreateArduinoLoader sysbus.cpu
```

3. Once your simulation is fully set-up and you are ready to receive and run the binary, start the loader:

```
(machine-0) arduinoLoader WaitForBinary 120 true
```

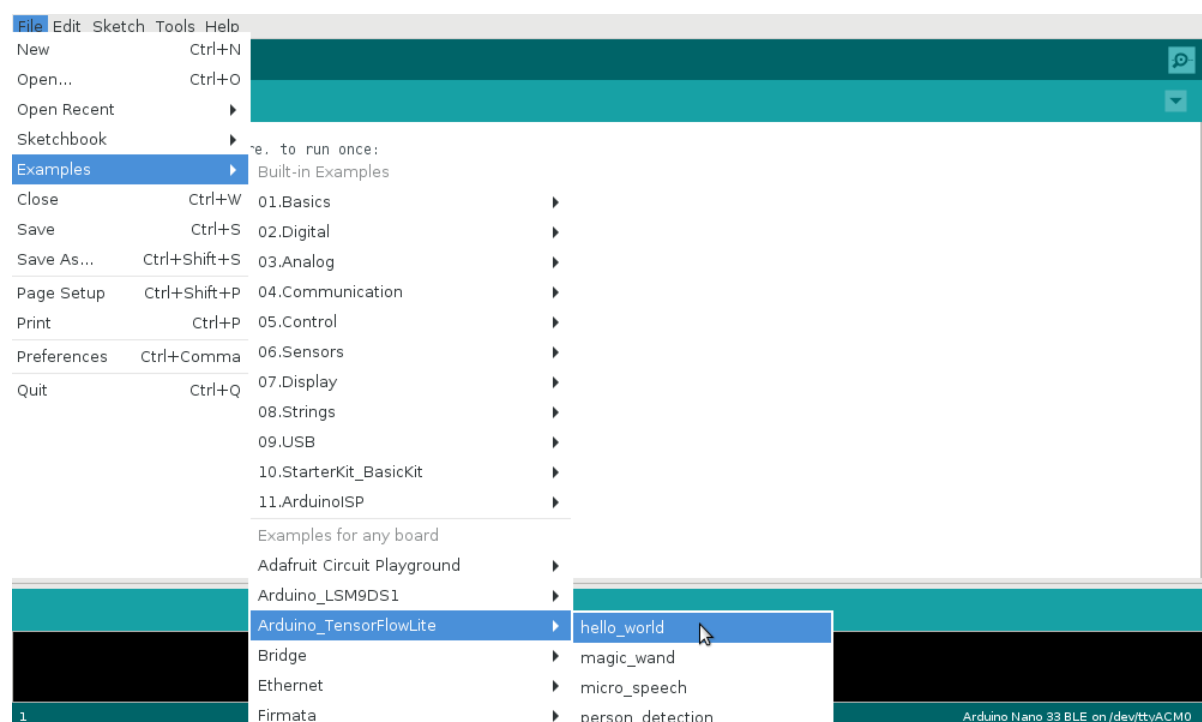
This will automatically connect Renode to the host using the `usbip` command (this uses `sudo` so you might be asked for your password) and wait 120 seconds for the binary to be uploaded by the Arduino IDE/CLI.

Note

If you don't want Renode to automatically connect to your host with the `usbip` command, do not pass the last argument (`true`). Remember that in such case you must do it manually before uploading the binary, as otherwise Arduino IDE/CLI won't be able to detect Renode and the process will fail.

35.3 Loading from Arduino IDE

Start the Arduino IDE and select your sketchbook (in this example we will use the **TensorFlow Lite Hello World** sample).

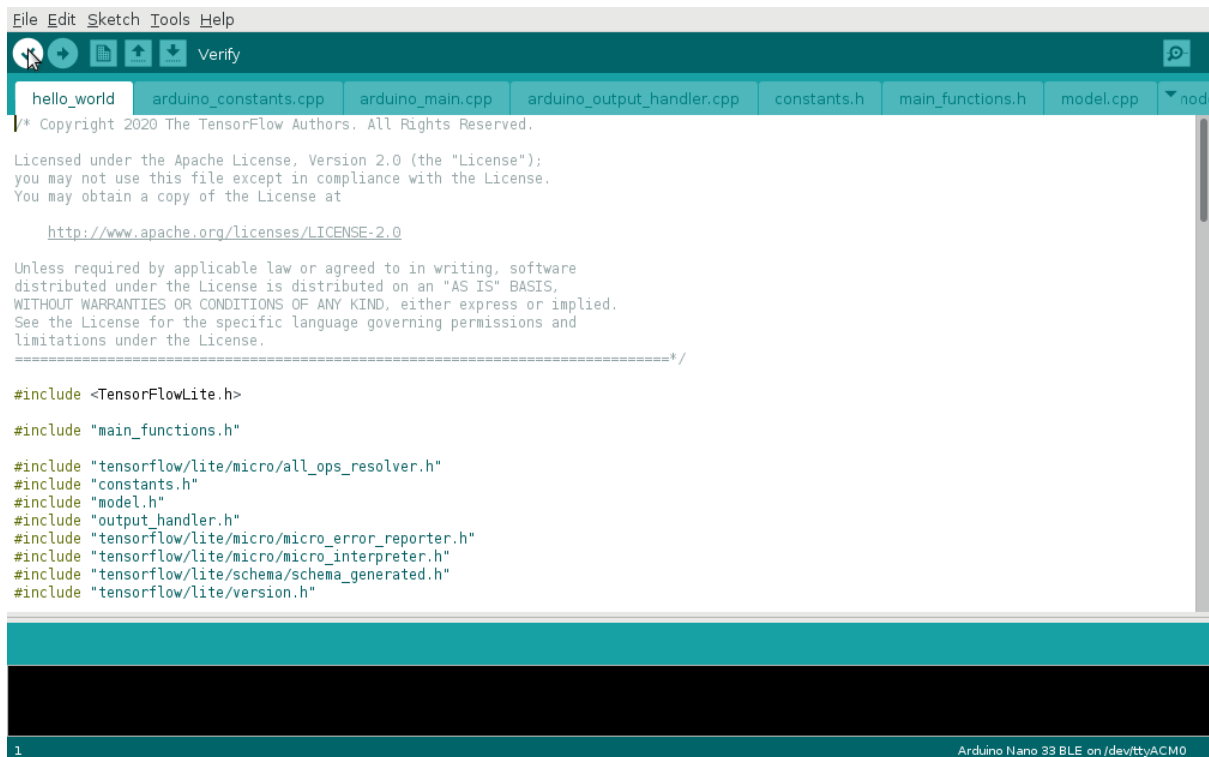


Note

The Arduino Nano 33 BLE board support does not come with Arduino IDE by default. You need to add it in the Tools -> Board -> Boards Manager menu by installing the “Arduino Mbed OS Nano Boards” package.

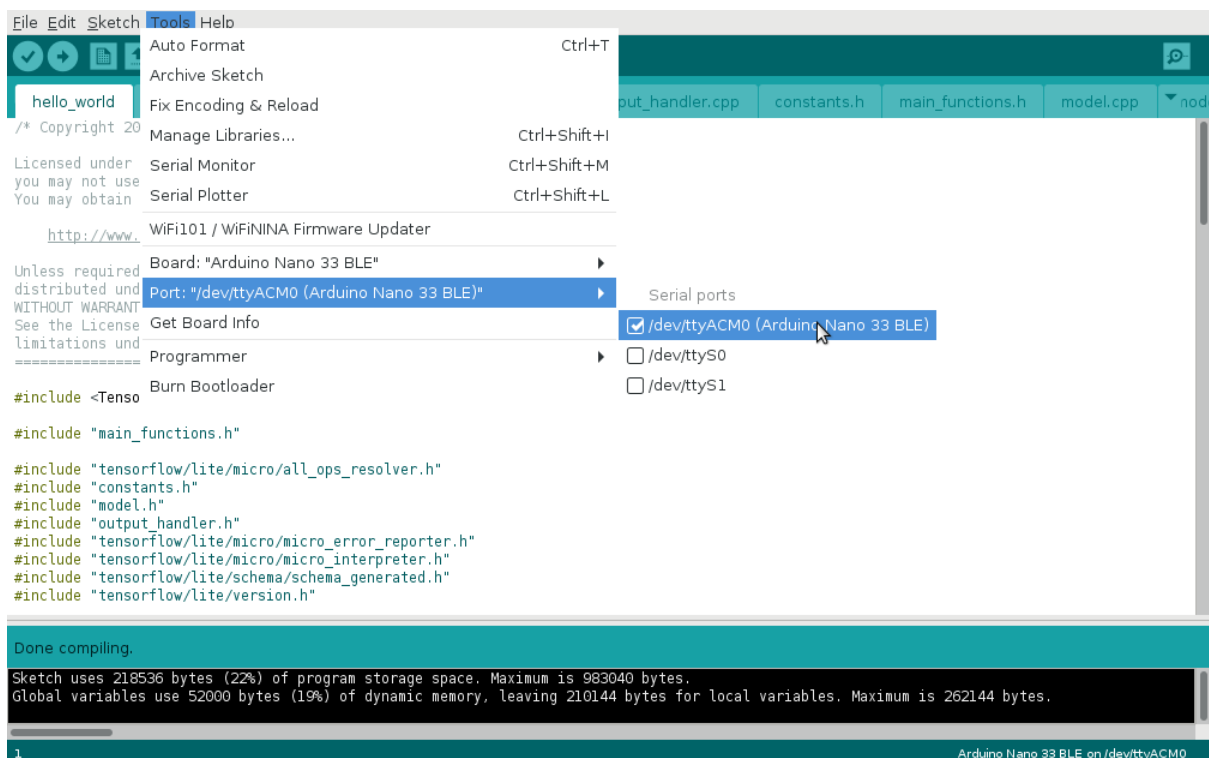
Select the Arduino Nano 33 BLE board from the Tools -> Board -> Arduino Mbed OS Nano Boards menu.

Compile your project by pressing the Verify button.



Once everything compiles correctly, start ArduinoLoader in Renode (as described in *the previous section*).

Select the proper /dev/ttyACMx device as a port from the Tools -> Port menu.



Upload the binary by pressing the Upload button.

```

File Edit Sketch Tools Help
Upload
hello_world arduino_constants.cpp arduino_main.cpp arduino_output_handler.cpp constants.h main_functions.h model.cpp nod
/* Copyright 2020 The TensorFlow Authors. All Rights Reserved.

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
=====*/

#include <TensorFlowLite.h>

#include "main_functions.h"

#include "tensorflow/lite/micro/all_ops_resolver.h"
#include "constants.h"
#include "model.h"
#include "output_handler.h"
#include "tensorflow/lite/micro/micro_error_reporter.h"
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/schema/schema_generated.h"
#include "tensorflow/lite/version.h"

Done compiling.
Sketch uses 218536 bytes (22%) of program storage space. Maximum is 983040 bytes.
Global variables use 52000 bytes (19%) of dynamic memory, leaving 210144 bytes for local variables. Maximum is 262144 bytes.
1 Arduino Nano 33 BLE on /dev/ttyACM0

```

35.4 Loading from Arduino CLI

You don't have to use the IDE in order to upload the binary - there is also an Arduino CLI tool that allows you to compile and upload your project directly from the command line.

Note

Make sure that `arduino-cli` is installed in your system (it does not come with Arduino IDE by default) and available in `PATH`. For details, see the [project's github page](#).

First, compile a project with the following command:

```
arduino-cli compile -b arduino:mbed:nano33ble hello_world.ino
```

Make sure that everything compiles fine and start `ArduinoLoader` in Renode (as described in *the previous section*).

Now, upload the binary with the following command:

```
arduino-cli upload -b arduino:mbed:nano33ble --port /dev/ttyACM0 hello_world.ino
```

Note

Please, make sure to select the proper `/dev/ttyACMx` device.

35.5 Starting the simulation

Once the binary is received, you'll see the following message in Monitor:

```
(machine-0) arduinoLoader WaitForBinary 120 true  
Binary of size 217088 bytes loaded at 0x10000
```

Now you can start the simulation with:

```
(machine-0) showAnalyzer sysbus.uart0  
(machine-0) start
```

On UART you should see the following output:

```
123  
123  
128  
128  
128  
128  
135  
135  
135  
135  
136  
136  
136  
136  
141  
141  
141  
141  
142  
142  
142  
142  
148  
148  
148  
148  
153  
153  
153  
153  
155  
165  
165  
165  
165  
168  
168
```

(continues on next page)

(continued from previous page)

168	
172	
172	
172	
172	
173	
173	
173	
173	
178	
178	
178	
178	
178	
178	
178	
178	
178	
178	
178	
178	
181	
181	
181	
181	
184	
184	

SHARING FILES BETWEEN HOST AND SIMULATED PLATFORM

Renode has 4 main methods of file sharing between the guest and the host:

1. VirtIO block device,
2. directory sharing,
3. built-in TFTP server,
4. *file transfer via the TAP interface.*

Tutorial on TAP-based transfers can be found in *the chapter on host-guest networking*. To achieve the best performance and portability it is recommended to use TFTP or VirtIO.

36.1 Sharing files using VirtIO block device

VirtIO is a widely used standard for virtualized devices supported in modern OSes. Advantages of using VirtIO to share files in Renode:

- ubiquity of drivers available for various guest OSes,
- support for the MMIO-based VirtIO block and filesystem device model,
- no need for having a platform-specific network controller modeled in Renode,
- faster transfer compared to simulated network transfers.

To use VirtIO block device you need to prepare the filesystem image and fill it with the resources of your choice. Start by preparing a directory with files you want to constitute your filesystem.

On Linux you can create a filesystem with:

```
$ truncate drive.img -s 128MB
$ mkfs.ext4 -d your-directory drive.img
```

Then you need to add the VirtIO block device support to the simulated Linux and the virtual platform in Renode.

Note

Since Linux v2.6.25 VirtIO drivers are supported and should be enabled by default (check the CONFIG_VIRTIO, CONFIG_VIRTIO_MMIO and CONFIG_VIRTIO_BLK configuration options).

To enable VirtIO, add a device tree entry describing device bus location and interrupt configuration:

```
virtio@100d0000 {
    compatible = "virtio,mmio";
    reg = <0x100d0000 0x150>;
    interrupt-parent = <&plic>;
    interrupts = <42>;
};
```

Now you need to add a corresponding extension to the Renode platform definition (the .repl file):

```
virtio: Storage.VirtIOBlockDevice @ sysbus 0x100d0000
    IRQ -> plic@42
```

Note

Addresses and interrupt line number must be consistent across .repl and DTS.

You can set up an underlying image for the VirtIO block device by using:

```
virtio LoadImage @drive.img
```

By default, Renode loads the image in a non-persistent mode. If you want to make the VirtIO block device persistent add the true argument at the end of the command:

```
virtio LoadImage @drive.img true
```

You can use standard tools like dd or mount the device to get access to it. By default, the VirtIO device is listed in the emulated Linux as /dev/vda.

36.2 Directory sharing

Directory sharing functionality is available when using VirtIO filesystem device.

This device enables directory sharing between the guest and host. Advantages of using this feature:

- real time host and guest file transfer
- no need to restart the machine to upload new files
- no need to repack images or filesystems

36.2.1 Libfuse

Sharing directory on the host is performed with the use of FUSE filesystem daemon, which handles requests on the directory from Renode. The shared directory must be a FUSE filesystem connected via a Unix domain socket. A passthrough filesystem has been prepared to use with this device with use of libfuse library.

36.2.2 Installation

Requirements:

- Meson
- Ninja

Filesystem daemon installation:

```
$ git clone https://github.com/antmicro/libfuse --branch passthrough-hp-uds
$ cd renode-filesharing-libfuse
$ mkdir build; cd build
$ meson setup ..
$ ninja
```

Filesystem daemon binary will be located at `example/passthrough_hp_uds`. For easier usage, export that binary location to `$PATH`.

36.2.3 Usage

Note

Virtiofs drivers have been supported in Linux since version 5.4 and should be enabled by default (you can check the `CONFIG_VIRTIO_FS` config option during compilation). Remember to include the virtiofs device under soc in the device tree.

Add a device tree entry describing device bus location and interrupt configuration:

```
virtio@100d0000 {
    compatible = "virtio,mmio";
    reg = <0x100d0000 0x150>;
    interrupt-parent = <&plic>;
    interrupts = <0x2>;
};
```

Now you need to add a corresponding extension to the Renode platform definition (the `.repl` file):

```
virtio: Storage.VirtIOFSDDevice @ sysbus 0x100d0000
    IRQ -> plic@2
```

Note

Addresses and interrupt line number must be consistent across `.repl` and DTS.

Start the filesystem daemon:

```
$ passthrough_hp_uds path_to_share
```

By default, this creates a USD socket in `/tmp/libfuse-passthrough-hp.sock`.

Create the virtiofs device in Renode:

```
virtio Create @/tmp/libfuse-passthrough-hp.sock "tag"
```

with tag being a name of your choosing.

In guest you can now mount the shared directory:

```
# mount -t virtiofs tag /mnt
```

36.3 Sharing files using TFTP

TFTP (Trivial File Transfer Protocol) is a protocol that allows file transfer between a client and a remote host. Advantages of using TFTP to share files in Renode:

- simplicity,
- configuration doesn't require any interference in machine structure,
- everything can be done in the Monitor,
- does not require host integration, works on all host platforms.

Having a built-in TFTP server in Renode allows you not only to transfer files, but also to easily verify the correctness of your network stack in a deterministic simulated environment.

36.3.1 Starting the TFTP server

To configure TFTP in Renode you need to create a *switch* and *connect it to your machine*.

Now you can create the TFTP server and connect it to your switch:

```
emulation CreateNetworkServer "server" "192.168.100.100"
connector Connect server switch
server StartTFTP 69
```

Note

Port 69 is a default for the TFTP protocol, but you can provide any other number acceptable for your TFTP client.

After you successfully start the server, you can access it in the Monitor via `server.tftp`

36.3.2 Using the TFTP server

Single files can be shared via the TFTP server using the `ServeFile` command.

`ServeFile` accepts two parameters. The first parameter is a path to your host file and the second parameter is the name under which you expose it via TFTP:

```
server.tftp ServeFile @path/to/file "filename"
```

Note

The second parameter is optional and if it is not specified, the file will be exposed with its original name.

Similarly, you can share directories via TFTP using ServeDirectory:

```
server.tftp ServeDirectory @path/to/directory
```

Note

Keep in mind that the built-in TFTP server does not handle uploading files from the guest to the host.

CO-SIMULATING WITH AN HDL SIMULATOR

Renode includes two integration methods for HDL co-simulation, allowing you to connect HDL peripherals with interrupts and external interfaces, like UART Rx/Tx lines:

- DPI-based SystemVerilog method (for simulators supporting DPI like Verilator, Questa or VCS)
- Custom Verilator-only C++ method

37.1 Adding co-simulated blocks

The Renode side of the integration layer is a plugin called CoSimulationPlugin, consisting of C# classes, which initiate the communication.

Typically an instance of the CoSimulatedPeripheral class corresponds to an HDL model. To add a co-simulated block to your platform, use the following snippet in your REPL file:

```
block: CoSimulated.CoSimulatedPeripheral @ sysbus <0x20000000, +0x100000>
```

37.2 DPI-based integration method

In this case, messages between Renode and an HDL simulator are transmitted over TCP sockets using the standard DPI interface supported in many simulators, including Verilator, VCS or Questa.

As the DPI layer relies on a TCP socket connection, you need to also specify the address parameter for the peripheral:

```
block: CoSimulated.CoSimulatedPeripheral @ sysbus <0x20000000, +0x100000>  
address: "127.0.0.1"
```

The HDL side uses a SystemVerilog interface which connects directly to your HDL simulation using signals specific to the bus you are simulating.

For co-simulation, you can pass the external simulator location via the SimulationPath property to spawn it automatically by Renode. In such case you can pass custom command line parameters using the SimulationContext property, e.g. for Verilator:

```
block: CoSimulated.CoSimulatedPeripheral @ sysbus <0x20000000, +0x100000>  
address: "127.0.0.1"  
SimulationPath: build/verilated
```

(continues on next page)

(continued from previous page)

```
SimulationContext: "--custom_features_enabled +RENODE_RECEIVER_PORT={0} ↵  
↵+RENODE_SENDER_PORT={1} +RENODE_ADDRESS={2}"
```

Otherwise, the HDL simulator has to be running before connecting to it.

37.2.1 Supported buses

- AHB
- APB3
- AXI4
- AXI4-Lite

37.2.2 Examples

You can find examples for every supported bus with instructions on building them in the [renode-dpi-examples repository](#). All examples have been tested both with [Verilator](#) and [Questa](#).

37.3 Custom, direct integration method (Verilator-only)

When using Verilator, you can also compile your HDL simulation as a dynamic library and link it with Renode at runtime, instead of using TCP sockets. Renode can spawn or link a simulation on its own.

Note

You shouldn't specify the address parameter for a peripheral that is connected as a dynamic library.

To interface your HDL simulation with Renode you need to connect signals of your HDL design with a [C++ interface](#).

37.3.1 Supported buses

- APB3
- AXI4
- AXI4-Lite
- Wishbone
- CFU Custom Function Unit interface

37.3.2 Examples

You can find many examples of verilated peripheral models such as [CFU](#), [UART](#), [RAM](#), and even a [CPU](#) in the [renode-verilator-integration repository](#). For detailed instructions on how to build models from the repository or your own, see *Co-simulating your verilated model*.

37.4 Connecting to verilated models

When using Verilator, your HDL models are first translated to C++ (or ‘verilated’) and then compiled.

Renode comes with several .resc files that use HDL models precompiled in this way (we also sometimes call them ‘verilated’ in their compiled form, to explain their origin), like a **verilated UART model** or **verilated Ibex**.

The Monitor command used to load the verilated model library depends on your host OS. On Linux, to add a precompiled FastVDMA HDL model to a peripheral named dma, you would run:

```
dma SimulationFilePathLinux @https://dl.antmicro.com/projects/renode/zynq-
↳ fastvdma_libVfastvdma-Linux-x86_64-1246779523.so-s_2057616-
↳ 93e755f7d67bc4d5ca33cce6c88bbe8ea8b3bd31
```

You can use `SimulationFilePathLinux`, `SimulationFilePathMacOS`, and `SimulationFilePathWindows` in the same script if you want to provide different payloads for different OSes, making the script multi-platform. Renode will select the one that matches your host OS.

If you are interested in just a single operating system, you can simply use `SimulationFilePath` and Renode will interpret it as a model matching your current OS.

By convention, we host two types of precompiled models:

- models prefixed with `V` use the socket based integration
- models prefixed with `libV` communicate with Renode via library calls

Most of the built-in scripts use binaries with the `libV` prefix.

BUILDING RENODE FROM SOURCE

This document provides detailed information on how to prepare the build environment, and then build and test Renode itself.

38.1 Prerequisites

38.1.1 Core prerequisites

The following instructions have been tested on Ubuntu 24.04, however there should not be any major issues preventing you from using other (especially Debian-based) distributions as well.

First, install the .NET SDK package as per the installation instructions, which can be found on [the official .NET site](#).

To install the remaining dependencies, use:

```
sudo apt update
sudo apt install git automake cmake autoconf libtool g++ coreutils policykit-1 \
    libgtk-3-dev uclibc-headers glib2.0 glib2.0-dev python3 python3-pip
```

First, install the .NET SDK package as per the installation instructions, which can be found on [the official .NET site](#). Make sure to install the right architecture, Arm64 for Apple Silicon, x64 for Intel

To install the remaining prerequisites, use:

```
brew install binutils gnu-sed coreutils dialog cmake
xcode-select --install
```

Note

This requires [homebrew](#) to be installed in your system.

Building Renode on Windows uses MinGW and Git Bash, and requires you to properly set up the system environment.

Git

1. Download and install git using the default options You can get it from [the official website](#).
2. Ensure the installation directory (C:\Program Files\Git by default) is in the system PATH variable.

Note

Prior to cloning the repository on *Windows*, git has to be configured appropriately. Run the following commands in Git Bash to set the options correctly:

```
git config --global core.autocrlf false
git config --global core.symlinks true
```

Python 3

1. Download and install the Windows version of the Python 3 framework from [the Python website](#).
2. Add location of the binaries to the system PATH variable. The installer can do this for you.

MinGW**Note**

The download link is from the MinGW-W64-builds project, linked from [the official MinGW-W64 website](#)

1. Download MinGW-w64 15.1.0 with the x86_64 architecture, win32 threads [the download site](#).
2. Extract the downloaded package and add its mingw64\bin directory (for example C:\mingw-w64\mingw64\bin) to the system PATH variable.

CMake

1. Download CMake and install Windows CMake from [the CMake website](#).
2. Ensure that the installation directory is in the system PATH. The installer will offer to do this for you

C# build tools

See [the official .NET site](#) for instructions on how to install .NET SDK.

38.2 Downloading the source code

Renode's source code is available on GitHub:

```
git clone https://github.com/renode/renode.git
cd renode
```

Submodules will be automatically initialized and downloaded during the build process, so you do not need to do it at this point.

38.2.1 Additional prerequisites (for Robot framework testing)

If you followed the instructions above, Python should be installed in your system. Install the pip package manager and some additional modules to enable writing and running test cases with the Robot framework:

```
python3 -m pip install -r tests/requirements.txt
```

38.3 Building Renode

Note

On Windows, the building process described in this section can only be executed in Git Bash.

To build Renode, run:

```
./build.sh
```

There are some optional flags you can use:

<code>-c</code>	clean instead of building
<code>-d</code>	build Debug configuration
<code>-v</code>	verbose output
<code>-p</code>	create packages after building
<code>-t</code>	create a portable package
<code>--source-package</code>	build a source package (dotnet on Linux only)
<code>-n</code>	tag built packages as nightly
<code>-s</code>	update submodules
<code>-b</code>	custom build properties file
<code>--skip-fetch</code>	skip fetching submodules and additional_
<code>↪resources</code>	
<code>--no-gui</code>	build with GUI disabled
<code>-B</code>	bundle target runtime (default value: linux-x64,
<code>↪ requires --net, -t)</code>	
<code>-F</code>	select the target framework for which Renode_
<code>↪should be built (default value: net8.0)</code>	
<code>--profile-build</code>	build optimized for profiling
<code>--tlib-coverage</code>	build tlib with coverage reporting
<code>--external-lib-only</code>	only build external libraries
<code>--external-lib-arch</code>	build only single arch (implies --external-lib-
<code>↪only)</code>	
<code>--tlib-export-compile-commands</code>	build tlibs with 'compile_commands.json'_
<code>↪(requires --external-lib-arch)</code>	
<code>--host-arch</code>	build with a specific tcg host architecture_
<code>↪(default: i386)</code>	
<code>--skip-dotnet-target-generation</code>	don't generate 'Directory.Build.targets' file,_
<code>↪useful when experimenting with different build settings</code>	
<code>--tcg-opcode-backtrace</code>	collect a backtrace for each emitted TCG opcode,
<code>↪ to track internal TCG errors (implies Debug configuration)</code>	
<code>--shared</code>	build the librenode native library
<code>--ui</code>	rebuild the web-based UI

Additionally you can directly specify flags which will be passed to the build system after `--`. For example, if you wanted to override the `CompilerPath` property you could use::

```
./build.sh -- p:CompilerPath=/path/to/gcc
```

You can also build `Renode.sln` from your IDE (like Visual Studio), but the `build.sh` script has to be run at least once.

38.4 Creating packages

The build script can create native packages only, i.e., you must run it on Windows to create an `.exe` installer package, on Linux for `.deb`, `.rpm` and `.pkg.tar.xz` packages or on macOS for the `.dmg` image.

38.4.1 Prerequisites

Depending on the system, there may be some prerequisites for building Renode packages.

Run:

```
sudo apt-get install ruby ruby-dev rpm 'bsdtar|libarchive-tools'
sudo gem install fpm
```

No additional prerequisites for macOS.

Note

The packaging process described in this section can only be executed in Git Bash.

1. Download and install **Inno Setup** and add it to the Windows system PATH variable

38.4.2 Building

To build binary packages, run:

```
./build.sh -p
```

To build portable packages (embedding the dotnet runtime into the binary), run:

```
./build.sh -t
```

The packages will have a version assigned to them, defined by the contents of the `tools/version` file.

You can also build nightly packages with:

```
./build.sh -pn
```

This will append a date and a commit SHA to the output files.

38.4.3 Location of packages

After completing successfully, the script will print the location of the files created:

```
renode/output/packages/renode_<version>.{deb|rpm|tar.gz}
```

```
renode/output/packages/renode_<version>.dmg
```

renode/output/packages/RenodeSetup-<version>.exe

PLATFORM DESCRIPTION FORMAT

To address the need of easily assembling peripheral models into complete platform definitions, a YAML-like platform description format was created for Renode based on common use cases as experienced in daily work with the framework.

Typically, files in this format have the `.repl` (REnode PLaatform) extension.

The format is meant to be human-readable, concise, easy to parse, base upon, extend and modify.

39.1 Indentation

Within Renode's platform description format, meaningful indentation (similar to e.g. Python) is used *alongside* curly braces (`{`, `}`). The rules are as follows:

1. Only spaces are used for indentation and indent has to be a multiple of four spaces.
2. Syntactically one level of indentation corresponds to one brace (opening one if we indent, and closing if dedent).
3. The indentation inside braces is not meaningful, this also applies to new line characters. They are all treated as white characters. When meaningful indentation is used, we refer to it as indent mode (as opposed to non-indent mode). To separate elements in non-indent mode (corresponding to lines in indent mode), a semicolon must be used.

For example these files are equivalent:

```
line1
line2
    line3
    line4
        line5
    line6
```

```
line1
line2 { line3; line4 { line5 }; line 6 }
```

39.2 Comments

There are two types of comments:

- line comments start with `//` and continue to the end of the line;

- multiline comments are delimited by `/*` and `*/` and can span multiple lines.

Both comments can be used in indent and non-indent mode, but there is one special rule. When a multiline comment spans multiple lines, it has to end at the end of the line. Otherwise it would be difficult to establish what indentation should be used for the rest of the line.

In other words this source is legal:

```
line1 /* here a comment starts
      here it continues
      and here ends*/
line2
```

But this one is not:

```
line1 /* here a comment starts
      here it continues
      and here ends*/ line2
```

39.3 Basic structure

Each platform description format consists of *entries*. An entry is a fundamental unit of peripheral description. The basic format of an entry is as follows:

```
variableName: TypeName registrationInfo
    attribute1
    attribute2
    ...
    attributeN
```

All of `TypeName`, `registrationInfo` and `attributes` are optional, but at least one of them must be present. If an entry contains a `TypeName`, then it is a *creating entry* (otherwise it is an *updating entry*).

Each creating entry declares a variable, there can be only one declaration for a given variable. The declaration can be done anywhere in the file hierarchy. All peripherals that are registered in the machine are also imported as variables and can have their updating entries (but not creating entries).

Both of the below are legal:

```
variable1: SomeType
    property: value

variable1:
    property: otherValue
```

```
variable1:
    property: value

variable1: SomeType
    property: otherValue
```

Entries (the ones that do not specify a type) can update and override information provided by the former ones. Eventually all entries corresponding to the given variable are *merged* so that the merge result contains attributes from all entries. Some of these can possibly be invalidated (overridden) by another (newer) entry.

Note

In practice, that means that the “newest” value for an attribute will be selected - that is the one which is provided **last** in the file. This also takes into account all files chained by using keywords. The entry specifying the type is not handled in any special way, nor does it have any priority when selecting properties' values.

TypeName must generally be provided with the full namespace the type is located in. However, if the namespace starts with `Antmicro.Renode.Peripherals`, then this part can be omitted.

A creating entry (not an updating one) can be optionally prepended with `local` - the variable declared in this entry is then called a *local* variable.

For example:

```
local cpu: SomeCPU
    StringProp: "a"

cpu:
    IntProp: 32
```

If the variable is local, then we can reference it only within that file. This will be clearer after reading the next section, but generally if one file depends on another, both can declare same named local variable and they are completely independent, in particular they can have different types.

39.4 Values

A *value* is a notion widely used in the platform description format. There are several kinds of values:

- *simple values* that can be further divided into:
 - *strings* delimited by a double quote, with `\` used as an escaped double quote;
 - *multiline strings* delimited by triple quotes `'''`, with `\'''` used as escaped triple quotes. For example:

```
name: '''this is \'\'
some
multiline
name'''
```

- *boolean values*, either `true` or `false`;
- *numbers*, which can be decimal or hexadecimal with the `0x` prefix;
- *ranges* representing an interval and that can be supplied in two forms:

* <begin, end> where begin and end are decimal or hexadecimal numbers (example: <0, 100>)

* <begin, +size> where begin and size are decimal or hexadecimal numbers (example: <0x10000, +0x200>)

- *reference values*, which point to a variable and are given just as the name of the variable;
- *inline objects* that denote an object described in the value itself and not tied to any variable (described *later*);
- *lists*, which can contain all types of values as described *below*;
- *dictionaries*, described *below*.

39.4.1 Lists

A list is a collection of values, enclosed in square brackets and separated by commas. For example, a list of integers can be defined as:

```
[1, 2, 3]
```

Lists can contain all types of values, including other lists:

```
[["a", "b", "c"], ["d", "e", "f"]]
```

They can also contain reference values:

```
[cpu0, cpu1]
```

as well as inline objects:

```
[new Point { x: 640; y: 480 }, new Point { x: 5120; y: 2160 }]
```

39.4.2 Dictionaries

A dictionary is a collection of key-value pairs. It is enclosed in braces and separated by semicolons.

While keys must be of simple type (string, numbers, enumeration value, boolean, reference or range), the values can be any kind of platform description format *value type*.

Example of string-numbers mapping with a dictionary:

```
{ "left": 0; "middle": 1.65; "right": 3.3; }
```

39.5 Registration info

Registration info tells in which register a given peripheral should be registered and how. A peripheral can be registered in one or more registers. For a single registration the format of registration info is as follows:

```
@ register registrationPoint as "alias"
```

where `registrationPoint` is a value and is optional. The as `"alias"` part is called an *alias* and is also optional. Using `registrationPoint`, the registration point is created or directly used (if the value specified is a registration point): If the registration point is not given, then either a `NullRegistrationPoint` is used or (if `NullRegistrationPoint` is not accepted) a registration point with no constructor parameters or all parameters optional.

If the registration point is a simple value, then a registration point is used with a constructor taking one parameter to which this simple value can be converted and possibly other optional parameters. Note that any ambiguity in the two cases mentioned above will lead to an error.

If the registration point is a reference value or an inline object then they are directly used as a registration point.

During registration, the registered peripheral is normally given the same name as the name of the variable. The user can, however, override this name with a different one using the mentioned alias.

Multiple registrations are also supported; this has the following form:

```
@ {
    register1 registrationPoint1;
    register2 registrationPoint2;
    ...
    registerN registrationPointN;
} as "alias"
```

The meaning and optionality of the elements is the same as it was in the previous case with the only difference that the peripheral is registered multiple times, possibly in different registers. Note that - as was mentioned at the beginning of this document - the indentation within braces does not matter.

Registration info can be given in any entry (creating or updating), also in more than one entry. In such case only the registration from the newest entry takes place. Registration can also be cancelled, i.e. overridden without providing new registration info. This is done using `@ none` notation, for example:

```
variable: @none
```

39.6 Core-specific registrations

Normally, when a peripheral is registered on the bus, it is visible to all cores. However, it is possible to restrict the registration to a specific core. This is done by using inline registration objects instead of a range value.

To specify a core-specific registration, you can use `BusPointRegistration` or `BusRangeRegistration`. Example of a core-specific registration:

```
nvic: IRQControllers.NVIC @ sysbus new Bus.BusPointRegistration {
    address: 0xE000E000;
    cpu: cpu_m33
}
```

In this example, the `nvic` is only visible at address `0xE000E000` to the `cpu_m33`.

If you need to specify the size of the registration explicitly, you can use `BusRangeRegistration`:

```
sysbus new Bus.BusRangeRegistration { address: 0x81000000; size: 0x1000; cpu: ↵
↵cpu0 }
```

This is equivalent to the <address, +size> range syntax, but allows specifying additional parameters like cpu.

39.7 Attributes

There are five kinds of attributes:

- constructor or property attributes;
- interrupt attributes;
- preinit attributes;
- init attributes;
- reset attributes.

39.7.1 Constructor or property attributes

A constructor or property attribute has the following form:

```
name: value
```

name is the name of the property (if the initial letter is uppercase) or constructor parameter (otherwise) and value is a value. When used with a property, if the attribute's value is convertible to this property type, then such converted value will be set (otherwise an error is produced).

Note, however, that another entry may update the property so that only the final (i.e. the last containing an attribute setting this property) entry is effective.

The none keyword can also be used instead of a value. Having it there means that the property is not set using any value and its value *before applying the description* is kept. It can be useful when some entry sets some value and we want to update this entry but not set any value.

The empty keyword can be used to set the default value of property or constructor parameter:

- numerical values are set to 0;
- string values are set to null;
- enum values are set to value corresponding to index 0 in this enum;
- reference types are set to null;

Constructor attributes are merged in a similar way, i.e. attributes from all entries belonging to the given variable are analyzed and for each name we take the last one value with this name. The constructor of the peripheral is chosen based on the set of merged attributes. For each possible constructor of the type specified in the creating entry we check whether:

- each parameter of the constructor has a default value or corresponding attribute, i.e. attribute having same name as the name of the parameter;
- the corresponding attribute has value convertible (for simple types) or assignable (otherwise) to the parameter type;
- all attributes have been used.

If all the conditions are satisfied then the analyzed constructor is marked as usable. If only one constructor is usable, then the object is created using this constructor. If there is no such constructor or there are more than one, an error is produced.

Because it is much easier to debug constructor selection problems if all the data are in one place (i.e. name of the type and constructor attributes), a warning is issued whenever a non creating entry contains constructor parameters (effectively updating a creating one).

Note that it is only possible to provide constructor attributes for an entry whose variable is going to be created, so it is not possible to provide any on variables representing peripherals existing before a given description is processed.

39.7.2 Interrupt attributes

As the name suggests, interrupt attributes are used to specify which interrupts of the variable in which the attribute is defined are connected and where. The simplest format of such attribute is as follows:

```
-> destination@number
```

where `destination` is a variable implementing the `IGPIOReceiver` interface and `number` is the destination interrupt number. Note that there is nothing specified on the left side - this is only possible if there is a single property of type `GPIO`, or there are multiple properties, but one of them is marked with a `DefaultInterrupt` attribute. This is the one that gets connected.

Whenever the user wants to specify which property should be connected, a more general form can be used:

```
propertyName -> destination@number
```

where `propertyName` is the name of the property (of the `GPIO` type) that should be connected. Also, if the type implements `INumberedGPIOOutput`, a number can be used instead of the property name.

If more than one interrupt is to be connected to the same destination peripheral, the following form of the attribute can be used:

```
[irq1, irq2, ..., irqN] -> destination@[irqDest1, irqDest2, ..., irqDestN]
```

Where `irq1` connects to `irqDest1` etc. Again, `irq` s can be names or numbers (if `INumberedGPIOOutput` is implemented) and `irqDest` s have to be numbers. Naturally, the arity of sources and destinations has to match.

There is also a possibility of connecting a single source to multiple destinations with the `|` sign, which can be used with every interrupt attribute format:

```
-> destination@number | another_destination@number
propertyName -> destination@number | another_destination@number
[irq1, irq2, ..., irqN] -> destination@[irqDest1, irqDest2, ..., irqDestN] |
↳another_destination@[irqDest1, irqDest2, ..., irqDestN]
```

Note that every attribute separated by the `|` sign has to be of the same arity as the source.

There is also a notation used in case of local interrupts:

```
source -> destination#index@interrupt
```

destination has to implement `ILocalGPIOReceiver` and index is the index of the local GPIO receiver. This notation can also be used with multiple interrupts:

```
[irq1, irq2, ..., irqN] -> destination#index@[irqDest1, irqDest2, ..., irqDestN]
```

Just as in the case of properties, interrupt attributes can update older ones. This is done basing on the source interrupt, i.e. if two attributes from different entries use the same source interrupt, only the one from the latter is used. Again, as in properties, the user may want to cancel the irq connection without specifying a different one. The keyword `none` can be used for this purpose:

```
source -> none
```

39.7.3 Init attributes

Init attributes are used to execute monitor commands on the variable. They have one of the following forms:

```
init:
    monitorStatement1
    monitorStatement2
    ...
    monitorStatementN
```

```
init add:
    monitorStatement1
    monitorStatement2
    ...
    monitorStatementN
```

The difference between them is that during merge phase the first one overrides the given variable's previous init attribute (if there is one) and the second one concatenates itself to that previous one. The final entry is eventually executed: every statement is prepended with the name of the peripheral the variable is tied to and then directly parsed by the Monitor. Note that this means that the init section is only legal for variables that are registered.

39.7.4 Preinit attributes

Preinit attributes are used to execute monitor commands before the variable is created. This is useful for setting up global state or preparing the environment for the creation of the variable (e.g. compiling a peripheral model from C# source). These commands are executed in the global context, so they are not automatically prepended with the variable name.

They have one of the following forms:

```
preinit:
    monitorStatement1
    monitorStatement2
    ...
    monitorStatementN
```

```
preinit add:
    monitorStatement1
    monitorStatement2
    ...
    monitorStatementN
```

Just like *Init attributes*, `preinit` overrides previous `preinit` attributes for the same variable, while `preinit add` appends to them.

The order of execution for `preinit` blocks is as follows: `sysbus` `preinit` blocks are executed first, then other variables' `preinit` blocks are executed in the order the variables appear in the file.

Each `preinit` block defined in a `repl` file is executed with the file's directory prepended to the Monitor path, so relative paths are resolved starting from the file's containing directory. For example, given a directory structure like

```
.
├── inner
│   ├── MyPeripheral.cs
│   └── myplatform.repl
├── MyPeripheral.cs
└── outer.repl
```

then, if `outer.repl` includes a using of `./inner/myplatform.repl`, which contains

```
peripheral: MyPeripheral @ sysbus 0x0
    preinit:
        include @MyPeripheral.cs
```

then the `include` invocation inside `myplatform.repl` will find `inner/MyPeripheral.cs`.

39.7.5 Reset attributes

Reset attributes are used to execute monitor commands on the variable *when* the corresponding peripheral is being reset. They have one of the following forms:

```
reset:
    monitorStatement1
    monitorStatement2
    ...
    monitorStatementN
```

```
reset add:
    monitorStatement1
    monitorStatement2
    ...
    monitorStatementN
```

Similarly to *Init attributes*, the `reset` overwrites the previous `reset` attribute (if exists), while the `reset add` concatenates itself with the previous one.

Internally, this attribute creates an `<entry>.reset` macro for a given entry, e.g. for the `peripheral` entry, `peripheral.reset` will be defined with contents of the `reset` attribute. This

macro is run when the peripheral is reset by machine, e.g. as a result of machine Reset in the Monitor.

39.8 Inline objects

Inline objects are values similar to reference values, but instead of creating a separate variable and then referencing it, it is defined directly in the place of reference. The form is as follows:

```
new Type
  attribute1
  attribute2
  ...
  attributeN
```

The effect is the same as creating an entry of this type and with those attributes, but it cannot be updated and is only available in the place of reference. So, for example, these codes lead to the same effect:

```
variable: SomeType
  SomeProperty: point

point: Point
  x: 5
  y: 3
```

```
variable: SomeType
  SomeProperty: new Point {x: 5; y: 3}
```

39.9 Depending on other files

One description can depend on another, in which case it can use all (non-local) variables from that file.

The using keyword declares a dependency:

```
using "path"
```

The line above is called a *using entry*. Using entries have to come before any other entries.

Using entries can form arbitrary hierarchies. The only limitation is that there can be no cycles (these are reported as errors).

39.10 Entry override order

As mentioned before, entries can be overridden. Variables are gathered in a certain order, and later ones override the previous. First, the using entries are processed, top to bottom. Then, entries in the current file are gathered from top to bottom. This process is applied recursively when there are more levels of usings.

For example, consider the following platform definition:

```
// myplatform.repl
using "a.repl"
using "b.repl"

variable: SomeType
```

```
// a.repl
variable:
  SomeProperty: "set by a, first"

variable:
  SomeProperty: "set by a, second"
```

```
// b.repl
variable:
  SomeProperty: "set by b, first"

variable:
  SomeProperty: "set by b, second"
```

The final value of `variable.SomeProperty` is “set by b second”. The reason is, that it’s the **last** override of `variable.SomeProperty` available. We can override this further in `myplatform.repl`.

TIME FRAMEWORK

40.1 Time and performance units

Time inside the emulation is referred to as *virtual time* (as opposed to *host time* or *real time*) and is expressed in *virtual seconds*. Currently, the resolution of *virtual time*, i.e., the minimum expressible time quant, is 10^{-9} *virtual second* or 1 *virtual nanosecond*. Performance of the CPU is expressed in *MIPS* units (*millions of instructions per second*) and is an integer value. This means that setting the time quant to one virtual microsecond allows the CPU with performance of 1 *MIPS* to execute exactly one instruction.

Note

For CPU with performance higher than 1 *MIPS*, it is possible to execute instruction-by-instruction using a special *stepping* mode.

Performance of the CPU can be configured with the following command (in this case, set to 120 *MIPS*):

```
cpu PerformanceInMips 120
```

The default value of the performance setting is 100 *MIPS*.

40.2 Time sources and sinks

Objects controlling flow of *virtual time* are called *time sources*. By default, there is one *time source*, called *master time source*, associated with the emulation. Currently, there is no API to create other *master time sources*, but the framework itself is capable of handling multiple *time domains* (with each *master time source* being a root of the domain).

Objects that are aware of the time flow are called *time sinks*. Each *sink* is connected to exactly one *source* (at any time, although it can change *sources*) and awaits information about how much *virtual time* has passed so far.

There are objects that play both roles: *sink* and *source* - they are called *slave time sources*. The reason for having a *slave time source* is explained in the following section.

40.3 Synchronization

Virtual time flows in the form of *quants* generated by *master time sources* and granted to *sinks*. The value of a *quant* is expressed in *virtual seconds* (or a fraction thereof) and is configurable by the user.

Once a *quant* is granted to the *sinks*, the *source* waits for them to finish execution. Once all slaves have reported back, it is time for the *synchronization phase*. As it is guaranteed that no nodes work during this phase, it is safe to process all inter-node communication during this. Only when the *synchronization phase* is finished, can a new *quant* be granted to all *sinks*.

It is possible to introduce *slave time sources* in the *domain* that divides the quantum into smaller fragments and allows for tighter synchronization of selected nodes.

40.4 Time domain

A configuration of *time sources* and *time slaves* connected together is called a *time domain*. All members of a *time domain* observe the flow of the same *virtual time* and are synchronized together. In case of having multiple *time domains* the time values are not comparable between them - in other words, there is no synchronization across different *time domains*.

There is one *time domain* by default in the emulation. The *master time source* is associated with the emulation itself, every machine provides its own *slave time source* that drives the CPU(s) being a *sink*.

40.5 Configuration and monitoring

There are several properties provided by a *time source* that can be used to tune the *virtual time* flow:

Quantum

The amount of *virtual time* between two *synchronization phases*. It is guaranteed that the difference of *virtual time* perceived of two *slaves* is never higher than a *quantum* of its closest common *source*.

The value of *quantum* can be changed between *synchronization phases*, but the new value won't be used until the next grant.

The *slave time source* can have a different *quantum* than its *master time source*. If the slave's quantum is smaller than master's, the slave will split the quantum into smaller pieces and report back to master only when all the pieces are used up. If the slave's quantum is bigger than master's, the slave will report immediately (not executing any CPU instructions) and accumulate the granted interval until it reaches the required quantum.

It is illegal to set *quantum* to zero as it would not allow the *virtual time* to pass at all.

AdvanceImmediately

Boolean value that can be used to force the emulation to execute as fast as the host system allows for. By default (i.e., when *AdvanceImmediately* is not enabled) Renode slows down the execution of instructions whenever it detects *virtual time* passes faster than *real time*.

There are also some read-only properties that can be used to monitor the current state of the *source*:

CurrentLoad

Floating-point value being a ratio of *real time* required to emulate a given interval of *virtual time*, calculated as an average of 10 samples.

CummulativeLoad

The same as Load but calculated as an average over all samples since the beginning of the emulation.

ElapsedVirtualTime

The amount of *virtual time* that passed since the *source* has been started.

NearestSyncPoint

The time stamp of the nearest *synchronization phase*.

NumberOfSyncPoints

The number of *synchronization phases* executed so far.

40.6 CPU pausing vs. halting

When any of the members of *time domain* pauses its execution, it will effectively block *virtual time* from passing. As a result, all other members will execute to the nearest *synchronization phase* and wait for the paused member to continue.

Sometimes it is convenient to have a member in the *domain* that is disabled, but does not block others - e.g., a CPU that is not clocked in multi-CPU architectures. The *CPU* class provides the `IsHalted` property that allows to achieve this goal.

PERIPHERAL MODELING GUIDE

Renode allows the user to “model” HW peripherals in several ways:

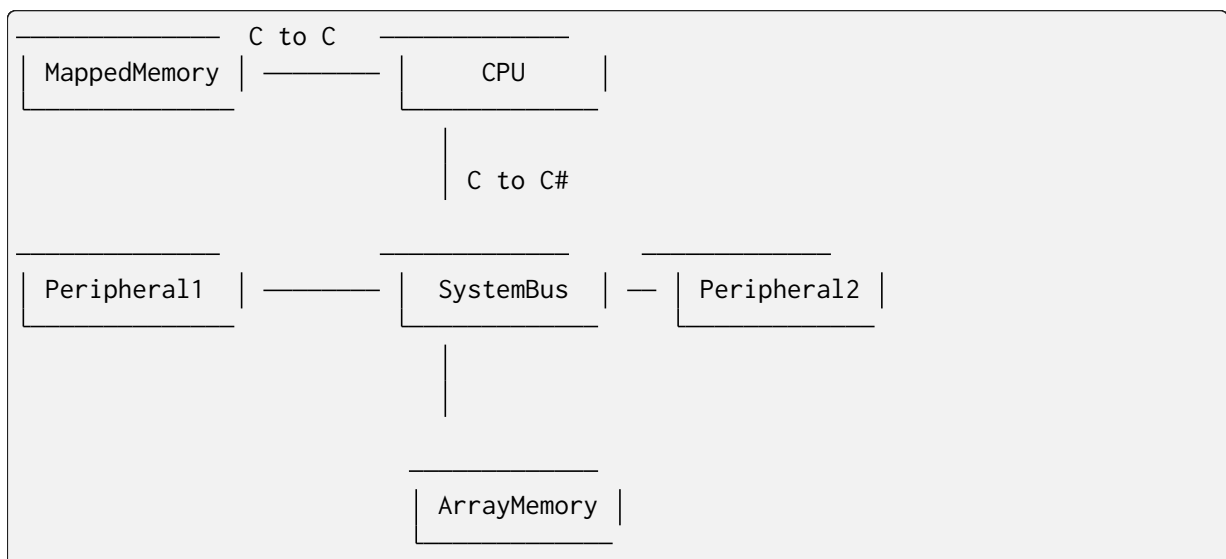
- **automatic tags from the SVD file** used mainly for logging purposes,
- **manual tags with return value** used for logging and trivial flow control,
- **Python peripherals** used for implementing very simple logic,
- C# models, used to describe advanced peripheral logic and interconnect - described in details below.

41.1 How does access to the system bus work?

read/write operations executed by the CPU (usually in the C implementation in the `tlib` sub-module) are either directed to the internal memory or passed to the system bus and handled by the framework at the C# level.

Access to the memory modeled as **MappedMemory** is handled entirely at the C level, all other operations are passed from C to C# via **TranslationCPU.Read{Byte,Word,DoubleWord,QuadWord}FromBus/**
CPU.Write{Byte,Word,DoubleWord,QuadWord}ToBus functions.

NOTE: It is possible to change **MappedMemory** type to **ArrayMemory** in order to handle all memory operations at the C# level. Keep in mind this might cause a significant drop in performance.



41.2 What if there is no peripheral mapped at given offset?

In the case of write, the operation will be ignored and a warning message will be generated in the log.

In the case of read, the default value of 0 will be returned and a warning message will be generated in the log.

41.3 What happens when the peripheral does not implement the given access width?

By default this situation is treated as if there was no peripheral mapped at a given offset.

It is, however, possible to enable automatic translation of access type at the peripheral level using the `AllowedTranslation` attribute - see an [example of usage](#).

Note that automatic translation might generate more accesses on the bus, e.g., 4 byte reads per one double word read or 1 double word read and one double word write per one byte write. This might have unintended side effects for some registers, e.g., automatically incrementing FIFO data register, issuing the “read-to-clear” behavior or others, depending on the registers’ semantics. It is up the developer to verify if the automatic translation is safe in the context of a given peripheral model.

41.4 Writing a peripheral model in C#

A C# class is considered a peripheral model if it implements the `IPeripheral` interface.

In order for the peripheral to be attachable to the system bus, it must implement at least one (but can implement a few) of: `IBytePeripheral`, `IWordPeripheral`, `IDoubleWordPeripheral`, `IQuadWordPeripheral` interfaces, enabling 8, 16, 32 and 64-bit accesses respectively.

Double word bus peripherals must implement at least three methods:

- for reading (e.g., `ReadDoubleWord`) - called by the system bus in order to read a value from the peripheral,
- for writing (e.g., `WriteDoubleWord`) - called by the system bus in order to write a value to the peripheral,
- for resetting (`Reset`) - called by the framework to restore the state of the peripheral to the initial state.

Although it’s technically possible to implement read/write method in any way, the preferred one is to use the Register Framework ([the source code](#)). For an example of usage, see [the LiteX UART](#).

You can even use a base class (`Basic{Byte,Word,DoubleWord}Peripheral`) to simplify the code - see an [example](#).

The following section explains how to design a peripheral using the Register Framework.

41.5 Register modeling guidelines

Create a private enum, preferably named `Registers`, that lists **all** registers supported by the peripheral. Conforming to the enum naming convention (or marking it with the `RegistersDescription` interface) allows the system bus to generate better log messages by including register name in logs generated by `sysbus LogPeripheralAccess`.

Use human-readable, PascalCase encoded names (i.e., `InterruptEnable` instead of `IEN`) even if they are referred to differently in the documentation. As values of the enum fields, use the offset from the beginning of the peripheral's memory space (i.e., offsets relative to the beginning of the peripheral, **not** absolute addresses). Keep in mind that a platform can have multiple peripherals of a given type. Please be reasonable here - there are sometimes peripherals with too many registers or registers forming a **repeatable pattern** - in such case a creative approach is encouraged.

Do not **implement** all registers - only those that are actually used by the software and can be therefore tested.

For each register **list** all fields but **implement** only the necessary ones. Fields that are not implemented should be marked as tags, reserved or ignored - this will help generating better access logs.

There are different type of fields available in Registers Framework:

- flags - single-bit fields (**example**),
- enum fields - single-or-multiple bit fields where bit patterns encode some non-numeric value (**example**),
- value fields - single-or-multiple bit fields encoding a numeric value (**example**).

For each field you can select an access mode (Read&Write by default) that defines which operations are allowed and how they are handled by the framework. Possible basic values (that can be combined together with a bitwise `|` OR operator) are:

- Read,
- Write,
- Set - writing 1 sets the bit, writing 0 has no effect,
- Toggle - writing 1 toggles the current value, writing 0 has no effect [this is most likely usable for flag fields only],
- WriteOneToClear - writing 1 clears the bit, writing 0 has no effect [this is most likely usable for fields flag only],
- WriteZeroToClear - writing 0 clears the bit, writing 1 has no effect [this is most likely usable for fields flag only],
- ReadToClear - the value is set to 0 after read.

Writing a non-zero value to a read-only field will be ignored and generate a warning in the log. Reading from the write-only field will return the default value of 0 (but will not generate a warning in the log, as it's impossible to infer which fields are read).

By default each register provides an automatic backing field. It means that the software will read the previously written value (assuming that fields are writable and readable). It is possible to access the backing field and modify its value from the code. In order to do that use an out parameter - see **an example**.

There are helper methods for generating groups of registers - see the [DefineMany](#) usage example.

It is also possible to attach callbacks for situations when the field is:

- written (with any value) ([example](#)),
- changed (written with a value different than the current one) ([example](#)),
- read (the value is taken from the backing field) ([example](#)),
- read - value provider (the value is generated by the callback itself) ([example](#)).

There are also callbacks for the whole register - [WriteCallback](#) and [ReadCallback](#). They are useful when the value of multiple fields is necessary for the callback logic.

41.6 Bus peripheral size

In most cases the size of the peripheral on the bus is well defined and can be included in the model. In order to do that, the class must implement the [IKnownSize](#) interface. The size encoded in the `Size` property is expressed in bytes.

Note: Peripherals **not** implementing the [IKnownSize](#) interface can be also used in Renode, but it is required to provide the size each time [the device is registered](#) (in the `repl` file).

41.7 Testing guidelines

For a peripheral to be pushed to Renode upstream repository, it is required that a test is provided, executing at least one binary. The preferred way of testing peripherals is to use standard tests/samples if available, e.g. Zephyr samples, driver tests etc. or provide a custom specific binary. All testing binaries should be buildable from sources.

The test case should be described in Robot Framework. See [examples](#) of simple tests.

41.8 Example peripherals

Here is a list of various Renode peripheral models that can be used as an inspiration:

- [UART](#)
- [Timer](#)
- [GPIO controller](#)
- [I2C controller](#)
- [SPI controller](#)
- [I2C sensor](#)
- [SPI sensor](#)

41.9 Automatic generation of peripheral stubs

To avoid writing the register layout by hand, Renode provides a tool to automate the process - to “scaffold” a generic peripheral structure, thus enabling the developer to focus mostly on implementing the behavioral logic. The [peakrdl-renode](#) tool works by parsing a peripheral's

memory map described in the **SystemRDL** file, and outputting an autogenerated file containing a partial class.

The class has to be then supplemented by writing custom logic, describing the behavior of the peripheral. This allows the developers to simplify the manual and error-prone process of codifying the registers, which can be especially useful during the hardware development phase, where the devices' register layout is not yet stable. SystemRDL data can also be used as a source of truth for software and RTL development, ensuring a consistent state of various parts of the project.

Renode provides an example of an autogenerated peripheral in the repository:

- **Caliptra I3C autogenerated part**
- **Caliptra I3C behavioral logic**

Only the logic is written by hand, the autogenerated file shouldn't be modified at all.

This sample demonstrates that for large models containing dozens or hundreds of registers, the tool substantially improves the development process.

To start using the peakrdl-renode tool, execute the following commands:

```
python3 -m venv .venv
source .venv/bin/activate
pip install peakrdl
pip install git+https://github.com/renode/renode/#subdirectory=tools/PeakRDL-
↪renode
```

Run `peakrdl renode --help` to verify that the tool was installed correctly.

You can experiment with a very basic model to see how the tool behaves.

Let's create a sample SystemRDL file, describing a layout of a peripheral.

```
addrmap Peripheral {
    regfile {
        default regwidth = 32;

        reg {
            name = "BitFields";
            field {
                name = "FIRST";
                sw = r;
                hw = w;
            } first [0:0];
            field {
                name = "SECOND";
                sw = rw;
                hw = rw;
                onwrite = wzc;
            } second [1:5];
        } bit_fields @ 0x0;

        reg {
```

(continues on next page)

(continued from previous page)

```

    field {
        name = "FIRST";
        sw = rw;
        hw = rw;
    } first [0:0];
    } also_bit_fields @ 0x10;
} registers @ 0x100;
};

```

To generate the C# scaffolding, assuming that the source file is named `peripheral.rdl`, run:

```

peakrdl renode -n MyPeripheral -N Miscellaneous -o MyPeripheral_generated.cs_
↪peripheral.rdl

```

The result should look similar to this:

```

// Generated by PeakRDL-renode

using Antmicro.Renode.Core.Structure.Registers;
using Antmicro.Renode.Peripherals.Bus;

namespace Antmicro.Renode.Peripherals.Miscellaneous
{
    public partial class MyPeripheral : IProvidesRegisterCollection
    ↪<DoubleWordRegisterCollection>, IPeripheral, IDoubleWordPeripheral
    {
        /// <summary> Register "registers.bit_fields" at 0x100 </summary>
        protected Registers_BitFieldsType Registers_BitFields;
        /// <summary> Register "registers.also_bit_fields" at 0x110 </summary>
        protected Registers_AlsoBitFieldsType Registers_AlsoBitFields;

        public DoubleWordRegisterCollection RegistersCollection { get; }

        public MyPeripheral()
        {
            RegistersCollection = new DoubleWordRegisterCollection(this);
            Registers_BitFields = new Registers_BitFieldsType(this);
            Registers_AlsoBitFields = new Registers_AlsoBitFieldsType(this);
            this.Init();
        }

        partial void Init();

        partial void Reset();

        void IPeripheral.Reset()
        {
            this.Reset();
            RegistersCollection.Reset();
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

uint IDoubleWordPeripheral.ReadDoubleWord(long offset)
{
    return RegistersCollection.Read(offset);
}

void IDoubleWordPeripheral.WriteDoubleWord(long offset, uint value)
{
    RegistersCollection.Write(offset, value);
}

public struct Registers_BitFieldsType
{
    /// <summary> Field "first" at 0x0, width: 1 bits </summary>
    public IFlagRegisterField FIRST;
    /// <summary> Field "second" at 0x1, width: 5 bits </summary>
    public IValueRegisterField SECOND;

    public Registers_BitFieldsType(IProvidesRegisterCollection
↪ <DoubleWordRegisterCollection> parent)
    {
        parent.RegistersCollection.DefineRegister(0x100, 0x0, true)
            .WithFlag(0, out FIRST, mode: FieldMode.Read, name: "FIRST")
            .WithValueField(1, 5, out SECOND, mode: FieldMode.Read |
↪ FieldMode.WriteZeroToClear, name: "SECOND");
    }
}

public struct Registers_AlsoBitFieldsType
{
    /// <summary> Field "first" at 0x0, width: 1 bits </summary>
    public IFlagRegisterField FIRST;

    public Registers_AlsoBitFieldsType(IProvidesRegisterCollection
↪ <DoubleWordRegisterCollection> parent)
    {
        parent.RegistersCollection.DefineRegister(0x110, 0x0, true)
            .WithFlag(0, out FIRST, name: "FIRST");
    }
}
}

```

Having this file generated, you can create a second file, `MyPeripheral.cs`, and start by implementing the `Init` and `Reset` methods. Then, you can refer to the individual registers' fields, e.g. by attaching callbacks, as shown in the snippet below, in the `AttachCallbacks` method.

```

using Antmicro.Renode.Core.Structure.Registers;
using Antmicro.Renode.Peripherals.Bus;

```

(continues on next page)

(continued from previous page)

```

using Antmicro.Renode.Logging;

public partial class MyPeripheral
{
    partial void Init()
    {
        // Add you logic here

        // This is an example of custom logic, achieved by attaching callbacks to
        registers
        AttachCallbacks();
        this.Reset();
    }

    partial void Reset()
    {
        // Add your logic here

        secondVal = 0;
    }

    private void AttachCallbacks()
    {
        // Example of callbacks attached to a register, implementing behavioral
        logic
        this.Registers.BitFields.FIRST.ValueProviderCallback += (val) =>
        {
            this.Log(LogLevel.Debug, "Previous FIRST value: {0}", val);
            // Test low bit of "secondVal"
            return (secondVal & 1) == 1;
        };
        this.Registers.BitFields.SECOND.WriteCallback += (old_val, new_val) =>
        {
            secondVal = new_val + 1;
            this.Log(LogLevel.Debug, "second_val: {0} -> {1}", old_val,
            secondVal);
        };
    }

    private ulong secondVal;
}

```

Note

For more information, refer to the [peakrdl-renode's README](#)

LOADING ASSEMBLY TO MEMORY

Renode allows the user to load assembly code in several ways:

- [LLVMAssembler](#) used mainly for tests,
- [writing hex instructions directly](#) which can be used for custom instructions,
- loading compiled ASM code directly, described below

Note

Before going further, it is recommended to read the [Debugging with GDB](#) and [Debugging with VSCode](#) chapters.

42.1 Prerequisites

You need to have the target architecture's GNU toolchain, e.g [RISC-V](#) or [ARM](#). We'll use `riscv64-unknown-elf-*` utilities (for ARM you'd use `arm-none-eabi-*`).

42.2 Compiling assembly code to an executable

Let's start with simple RISC-V assembly that stores and loads from memory at `0x00064000`:

```
.globl _start
_start:
    li a0, 0x00064000
    li a2, 0x5
    sw zero, (a0)
    lw a4, (a0)
    sw a2, (a0)
    lw a4, (a0)
loop:
    j loop
```

To compile the code use your GNU toolchain's `as` and `ld`:

```
riscv64-unknown-elf-as -march=rv64imac_zba_zbb asm.s -o asm.o
riscv64-unknown-elf-ld asm.o -o asm.elf
```

Let's use this simple `asm.resc` file with platform:

```

using sysbus
machine LoadPlatformDescriptionFromString
"""
cpu: CPU.RiscV64 @ sysbus
    cpuType: "rv64imac_zba_zbb"
    privilegedArchitecture: PrivilegedArchitecture.Priv1_10
    timeProvider: empty

dram: Memory.MappedMemory @ sysbus 0x00
    size: 0x06400000
"""
machine StartGdbServer 3333
sysbus LoadELF @asm.elf

```

Notice, that we need CPU type and model in our platform to match compiled ASM code. Here we have model CPU.RiscV64 and cpuType as rv64imac_zba_zbb. For more details go to [Configuring a RISC-V CPU](#).

Start Renode with this .resc file:

```

$ renode asm.resc
(monitor) i $CWD/test.resc
(machine-0)

```

Now in separate terminal use the target architecture's GDB:

```
$ riscv64-unknown-elf-gdb asm.elf
```

In GDB connect to Renode's GDB server:

```

(gdb) target remote :3333
Remote debugging using :3333
0x000000000000100b0 in _start ()

```

It's best to use GDB's TUI with ASM and register view. To do that:

```

(gdb) tui enable
(gdb) layout asm
(gdb) layout regs

```

Now you can see registers and our assembly source. To step over use si.

```

-Register group: _
↪general
|zero      0x0    0      |ra      0x0    _
↪0x0
|sp        0x0    0x0    |gp      0x0    _
↪0x0
|tp        0x0    0x0    |t0      0x0    0 _
↪
|t1        0x0    0      |t2      0x0    0 _
↪

```

(continues on next page)

(continued from previous page)

fp	0x0	0x0		s1	0x0	0	↳
↳							
a0	0x0	0		a1	0x0	0	↳
↳							
a2	0x0	0		a3	0x0	0	↳
↳							
	>0x100b0 <_start>	lui		a0,0x64			↳
↳							
	0x100b4 <_start+4>	li		a2,5			↳
↳							
	0x100b6 <_start+6>	sw		zero,0(a0)			↳
↳							
	0x100ba <_start+10>	lw		a4,0(a0)			↳
↳							
	0x100bc <_start+12>	sw		a2,0(a0)			↳
↳							
	0x100be <_start+14>	lw		a4,0(a0)			↳
↳							
	0x100c0 <loop>	j		0x100c0 <loop>			↳
↳							
	0x100c2	unimp					↳
↳							
remote Thread 1 (asm) In: _start							
↳	L?? PC: 0x100b0						
(gdb)							

Do note that some instructions are pseudoinstructions and the source you've written may not necessarily be the same as the assembly view in GDB. The generated instructions may also differ between LLVMAsmbl and GNU as.

42.3 Debugging tlib

Note

Remember to compile and start Renode with debug flag (build.sh -d, renode -d) for this to work!

Renode also gives you option to see how the instructions are being translated. To do that we'll use VSCode.

After starting Renode and attaching GDB, move to VSCode and navigate to src/Infrastructure/src/Emulator/Cores/tlib/arch/riscv/translate.c and set the breakpoint right before the case OPC_RISC_LUI:, and use the provided (gdb) Tlib Attach launch script.

Search for the Renode process, it should be dotnet type, attach to it (you may need to elevate your privileges to root, for rootless debugging check [YAMA's ptrace_scope](#)).

Then do si in GDB, which should make your VSCode stop at the previously-set breakpoint.

MICROSEMI MI-V EXAMPLE

In this tutorial you will learn how to setup a fully functional emulation of a single Mi-V board by Microsemi, with a RISC-V CPU. It covers topics ranging from installing Renode, through basic commands, to the process of *integrating with Microsemi's SoftConsole IDE*.

While this tutorial focuses on a single Mi-V platform, Renode supports multiple devices running in the same simulation - and it should be fairly simple to extend this scenario to multiple interconnected nodes.

43.1 Installation

The Renode framework is hosted [on GitHub](#). You can download and install it following the process described in the [README](#) file, but the easiest way to start is by downloading a binary release from <https://github.com/renode/renode/releases>. The precompiled packages are provided as deb and rpm packages for Linux, dmg package for macOS and a zip archive for Windows.

43.2 Starting Renode

One of the many ways to interact with Renode is via its command line interface.

After installing Renode, you should be able to run the `renode` command from your command line. Alternatively, look for the `Renode.exe` binary. Once Renode is started, you will see two windows - one for Renode's smart logger and one for the CLI called "the Monitor".

From this window you will be able to create and control the whole emulation environment.

43.3 Scripts

While you can type all commands interactively, it's a good idea to group them into reusable Renode scripts, which typically have the `.resc` extension. Their purpose is to load binaries, set starting conditions, prepare the environment, connect machines to the network etc.

For the purpose of this article we will use the script available in the Renode package named: [scripts/single-node/miv.resc](#). For details about the particular script you will be running, please inspect the file in the Renode installation directory (e.g. for Linux it is `/opt/renode/scripts`).

43.4 Loading our setup

The provided script creates a single machine and loads a sample LiteOS-based application.

To run a script, use the `include` command (or `i`, for short), with a path to the script to load, prepended with the `@` sign, like this:

```
include @scripts/single-node/miv.resc
```

After the script is loaded, you will see a new terminal - a UART window opened by the `showAnalyzer` command.

In the provided scripts, we're using precompiled binaries hosted online, but you can provide your own binaries by setting the `$bin` variable before loading the script:

```
$bin=@path/to/application.elf
```

The emulation is now loaded, but not started. You can control it using `start` and `pause` - and other commands, as described in the next section.

43.5 Simple commands

43.5.1 Start and pause

To control whether the simulation is running, use `start` and `pause`.

43.5.2 Machines

In the provided script, we use the `mach create` command to create the machine. This switches the context in the Monitor. All subsequent commands are executed with regard to the current machine.

To change the machine use the `mach set` command. Use a number or the name of the machine, e.g. `mach set 1` or `mach set "MI-V"`.

All machines can be listed with the `mach` command. To clear the current selection use `mach clear`.

43.5.3 Accessing peripherals

All peripherals are accessible in the Monitor, with most of their methods and properties exposed to the user. To list all available peripherals use the `peripherals` command.

43.5.4 Peripheral methods and properties

To access a peripheral you have to provide its fully qualified name. All peripherals are registered in `sysbus`, so use `sysbus.uart` to access the UART or `sysbus.gpioOutputs.led0` to access one of the LEDs.

The using `sysbus` command is used in most of the provided demos, allowing you to drop the `sysbus.` prefix.

Typing a peripheral's name gives you a list of available methods, fields and properties. The list is automatically generated, so most of the accessible members are not designed for the end-user.

The list shows examples of correct Monitor syntax for each member type.

43.5.5 Other commands

To find information about built-in Monitor commands, type `help` and refer to the documentation. Running `help builtin_command_name` prints out the help for the given command.

43.6 Debugging and inspection

Renode offers you many ways to verify the behavior of your applications. Thanks to having full control over the environment, you can add logging, hooks on events, interactive code debugging and more in a way that is 100% transparent to the emulated application.

Here we will present just a few of the available debugging options.

43.6.1 Function name logging

When the application is stuck or misbehaves it is always a good idea to inspect the trace of function calls. To enable logging of function names in a selected machine run `cpu LogFunctionNames true` (and `false`, respectively, to disable it).

Since the amount of logged data may be too overwhelming to be useful, you can filter the logged functions to the ones that begin with a specified prefix. For example `cpu LogFunctionNames true "UART_ LOS_"` will only log functions that begin with either `"UART_"` or `"LOS_"` prefix.

43.6.2 Logging of peripheral accesses

If your driver does not behave correctly, it may be a good idea to investigate the communication with the device it controls. To enable logging of each interaction between the CPU and the UART peripheral, run `sysbus LogPeripheralAccess uart`.

This feature is available only for peripherals registered directly on the system bus.

43.6.3 GDB

A popular tool for debugging, GDB, can be used to analyze applications running in Renode. It uses the same remote protocol as OpenOCD, so it can be easily integrated with most GDB-based IDEs, such as SoftConsole or Eclipse. To start a GDB stub in Renode, run `machine StartGdbServer 3333` (where 3333 is a sample port number) and connect from GDB by calling `(gdb) target remote :3333`. To start the emulation you have to run both `start` in Renode and `continue` in GDB.

You can use most of GDB's regular features: breakpoints, watchpoints, stepping, reading/writing to variables, etc. You can also use the `monitor` command in GDB to send commands directly to the Renode CLI (to avoid switching between two console windows).

43.7 SoftConsole integration

One of the Renode's main goals is to easily integrate with tools that are used for a developer's everyday work. A great example of such a tool is [SoftConsole, an Eclipse-based IDE from Microsemi](#).

SoftConsole offers debugging capabilities that can be normally used when attached to hardware. By altering the project settings you can connect it to Renode.

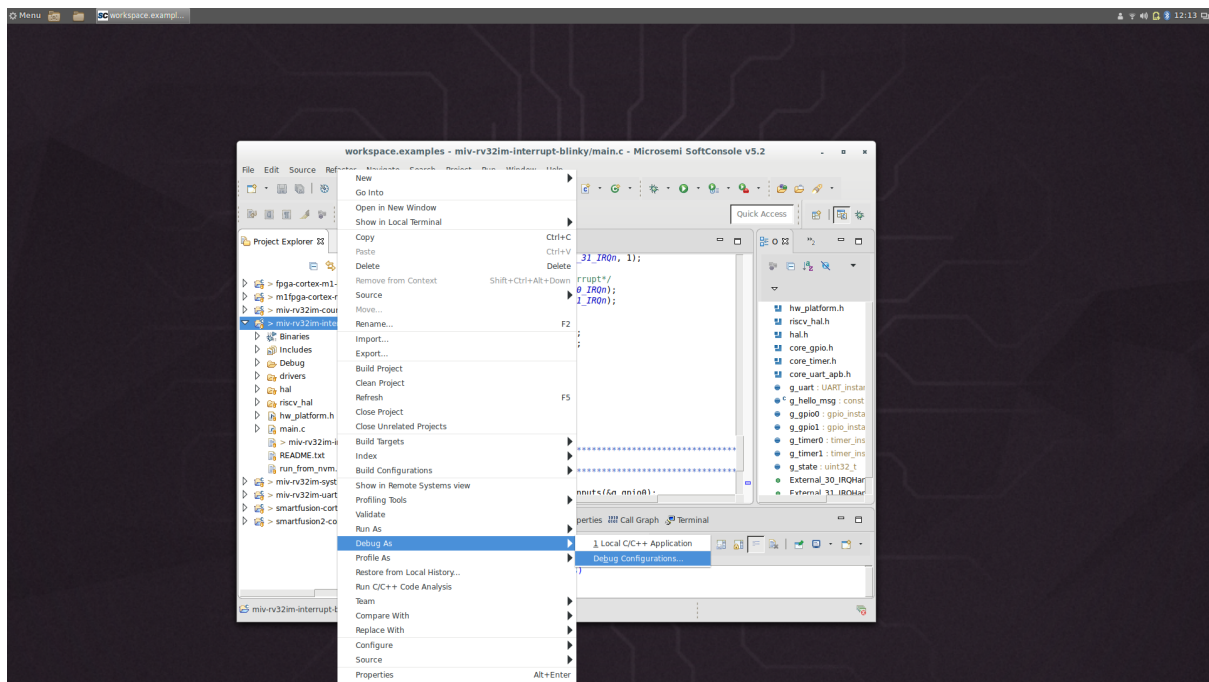
Start by running the GDB server in renode:

```
(monitor) include @scripts/single-node/miv.resc
(MI-V) machine StartGdbServer 3333 true
```

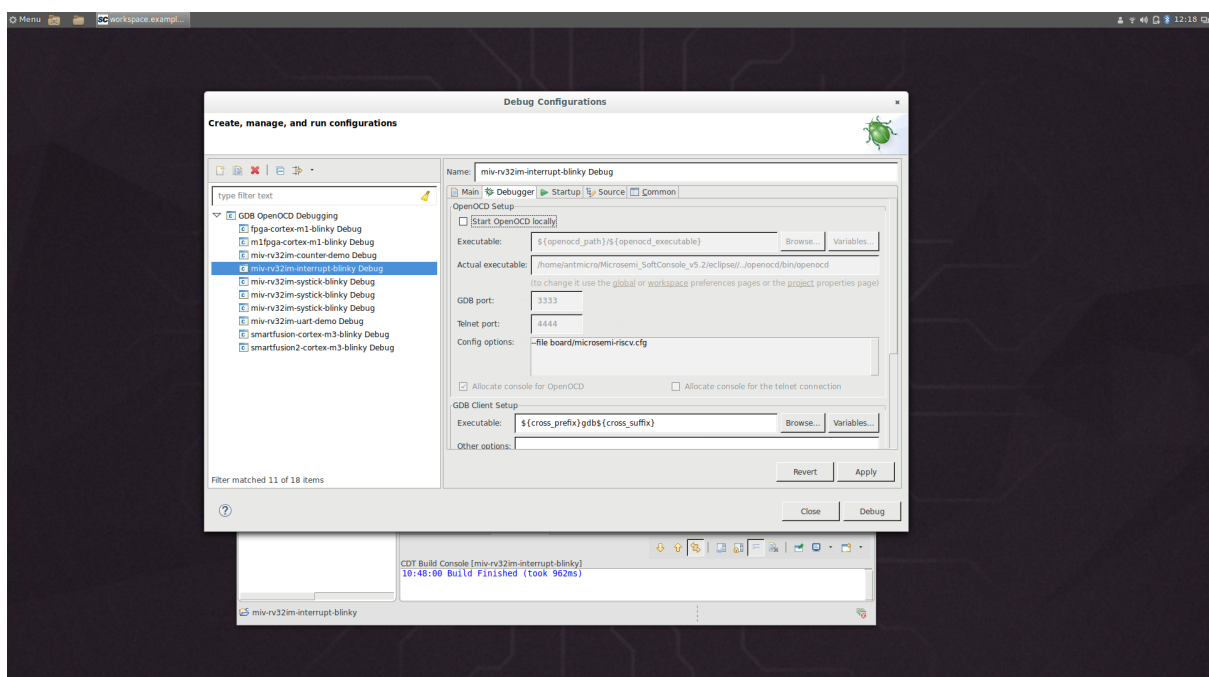
Please note the true parameter - it forces Renode to autostart as soon as the GDB client connects.

Now you need to configure the debug configuration in SoftConsole.

In the Project Explorer, right click on your project's name, select Debug As and Debug Configurations....



This opens a window, where you need to open the Debugger tab. There, uncheck the checkbox Start OpenOCD locally, as Renode will serve the same purpose as OpenOCD usually does.

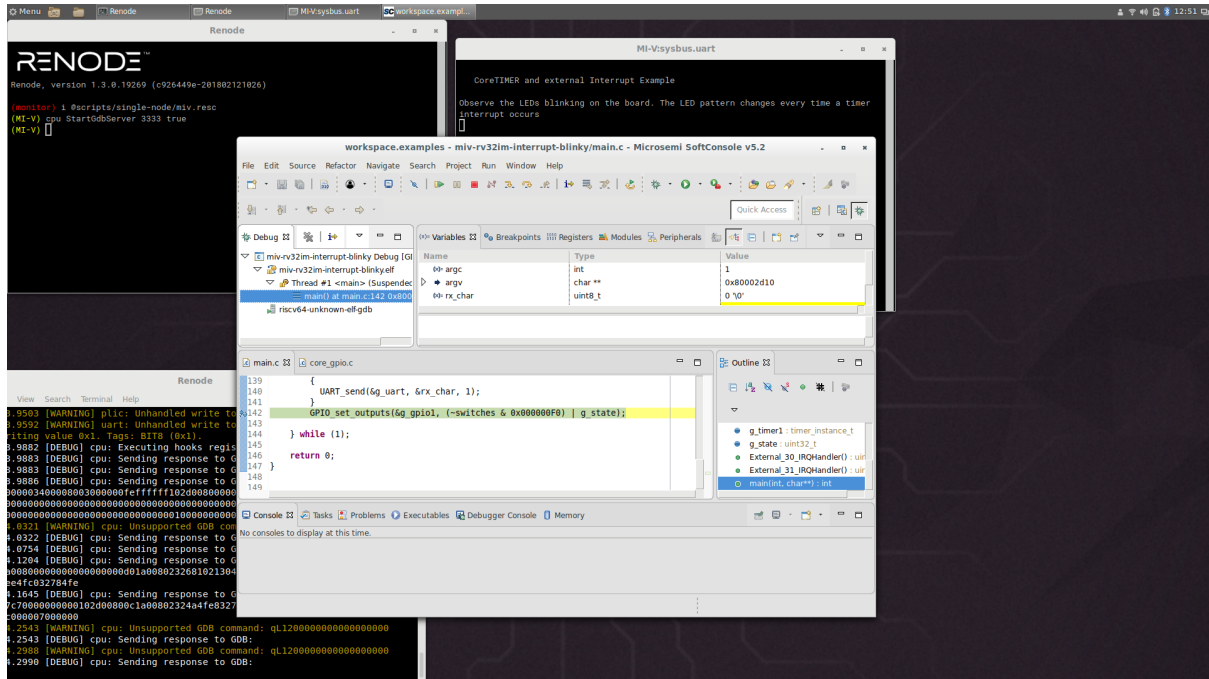


You must verify that the remote port number in the Remote Target section is the same as the one provided in the StartGdbServer command.

With these changes in place, you can now click Debug. SoftConsole will connect to Renode and the emulation will start automatically.

By default, you will observe a breakpoint at the beginning of the main function being hit.

In SoftConsole you can add your own breakpoints, inspect and change variables and step over your code, while still being able to interact with Renode in the usual way, via the Monitor.



TESTING ZEPHYR PTP SUPPORT

This tutorial will guide you on how to use Renode to run a set of tests verifying Zephyr's [TSN/PTP support](#).

44.1 Prerequisites

To start the test you need to download and build Renode according to the *Building Renode from source*.

The test suite is using [Robot Framework](#) and can be run with a single script.

To create your own Zephyr binaries to be tested, you need to follow [Zephyr's Getting Started Guide](#).

The tests require two Zephyr ELF files built from the `zephyr/samples/net/gptp` sample, targeting the `sam_e70_xplained` board.

To build them, create two overlay files, one for the Grand Master node, one for the slave.

Config for the Grand Master (`gm.conf`):

```
CONFIG_NET_GPTP_GM_CAPABLE=y
CONFIG_ETH_SAM_GMAC_RANDOM_MAC=y

CONFIG_NET_CONFIG_MY_IPV4_ADDR="192.0.2.1"
CONFIG_NET_CONFIG_MY_IPV6_ADDR="2001:db8::1"
CONFIG_NET_GPTP_NEIGHBOR_PROP_DELAY_THR=200000
```

Config for the slave node (`slave.conf`):

```
CONFIG_NET_GPTP_GM_CAPABLE=n
CONFIG_ETH_SAM_GMAC_RANDOM_MAC=y

CONFIG_NET_CONFIG_MY_IPV4_ADDR="192.0.2.2"
CONFIG_NET_CONFIG_MY_IPV6_ADDR="2001:db8::2"
CONFIG_NET_GPTP_NEIGHBOR_PROP_DELAY_THR=200000
```

Follow the Zephyr documentation to build these samples. For example, to build the Grand Master application, run:

```
west build --board sam_e70_xplained -- -DOVERLAY_CONFIG=gm.conf
```

44.2 The test suite

The suite executes a set of tests running on two SAM E70 nodes, connected via ethernet. The following tests are implemented:

- node should send a PDelay request packet
- slave node should accept a Grand Master node by calling a specific Zephyr callback
- master node should send Announce packets with expected parameters
- master node should send properly formed Sync and Sync Follow Up packets
- master node should send Sync packets in valid intervals
- slave node should sync its clock to master

44.2.1 Running the test

To start the test, simply run the following command from the Renode root directory:

```
renode-test tests/platforms/SAME70.robot
```

This will run the whole suite of tests. After the test is finished, the result will be stored in `output/tests/report.html`.

To switch the binaries used in these tests, edit the provided `.robot` file. Alternatively, if you don't want to make any changes, you can use the `--variable` switch to specify the files you want to use:

```
renode-test --variable ZEPHYR_MASTER_ELF:path/to/zephyr.elf --variable ZEPHYR_
↪SLAVE_ELF:path/to/another/zephyr.elf tests/platforms/SAME70.robot
```

RENODE, FOMU AND ETHERBONE BRIDGE EXAMPLE

This tutorial shows how to run a hybrid simulation where part of the platform is run in Renode and part on FPGA hardware.

45.1 System architecture

The system architecture consists of three main parts:

- **FOMU** - Lattice iCE40UP5K-based board with RGB LED, connected to the host machine over USB;
- EtherBone bridge, translating wishbone packets between TCP and USB;
- Renode simulating the **LiteX** platform, running Zephyr OS that controls the RGB LED.

45.2 Prerequisites

45.2.1 Simulation

For the simulation part, you need to have the latest version of Renode available in your system. You can install [a prebuilt package for your OS](#) or build a copy from sources as specified in the *Building Renode from source*.

45.2.2 Hardware

For the hardware part, you need to have the **FOMU** board.

FOMU needs to be flashed with the **Foboot** bitstream containing the **ValentyUSB** IP core with a USB-Wishbone bridge. The manufactured FOMU comes preloaded with this bitstream and can be used right away.

If you have assembled your own copy of the board or changed the original bitstream, please remember to load Foboot again. For convenience, the prebuilt version of the bitstream is [hosted by Antmicro](#).

45.2.3 EtherBone bridge

In order to connect Renode to **FOMU** you need to download the EtherBone-to-USB bridge shipped with **LiteX**.

Clone the repository and initialize the environment:

```
git clone https://github.com/enjoy-digital/litex
cd litex
./litex_setup.py init # this will clone the dependencies
export PYTHONPATH=`pwd`:`pwd`/litex:`pwd`/migen
```

litex_server.py additionally requires the pyusb package to be installed in the system:

```
pip3 install pyusb
```

45.3 Verifying the device

Plug **FOMU** into a USB port and verify if it has been recognized by checking the dmesg logs:

```
[65038.250957] usb 2-1: new full-speed USB device number 16 using xhci_hcd
[65038.409283] usb 2-1: New USB device found, idVendor=1209, idProduct=5bf0,
↳bcdDevice= 1.01
[65038.409286] usb 2-1: New USB device strings: Mfr=1, Product=2, SerialNumber=0
[65038.409287] usb 2-1: Product: Fomu DFU Bootloader v1.7.2-3-g9013054
[65038.409288] usb 2-1: Manufacturer: Foosn
```

Note: The version of the product might differ, but it should work correctly with v.1.7.2 upwards.

If the device is not detected, see the section below.

45.4 Loading the bitstream (optional)

If your device is detected as a DFU Bootloader in version 1.7.2 or higher, you can skip this step.

In order to upload a bitstream to a device that is not recognized as a DFU Bootloader you will need an external programming board.

Note

For convenience, you can use [the Fomu Programmer](#) - the Open Hardware programming board for **FOMU** by Antmicro.

Download and make **iceprog** - open source programming software for Lattice iCE40:

Note

Building iceprog requires the ftdi library headers to be available in the system.

```
git clone https://github.com/cliffordwolf/icestorm
cd icestorm/iceprog
make
```

Download the prebuilt bitstream:

```
wget https://antmicro.com/projects/renode/foboot-bitstream.bin-s_104250-  
fc5f419372eb9a3a0baa5556483163bcfccb7d33 -O foboot-bitstream.bin
```

Note

You can also build the bitstream yourself by following the instructions on the [Foboot](#) page.

Attach the board to the programmer and load the bitstream to the FPGA:

```
sudo iceprog foboot-bitstream.bin
```

45.5 Running the demo

Start the EtherBone bridge from the [LiteX](#) repository:

```
cd litex  
sudo python3 litex/tools/litex_server.py --usb --usb-vid 0x1209 --usb-pid 0x5bf0
```

Run the Zephyr OS image in simulation using the script shipped with Renode:

```
(monitor) start @scripts/complex/fomu/renode_etherbone_fomu.resc
```

Now you can control the HW LED from Zephyr's shell using special commands:

```
uart:~$ led_toggle  
uart:~$ led_breathe
```

`led_toggle` toggles the green led

`led_breathe` makes the blue led blink with a fade-in/fade-out effect

USB/IP SUPPORT

Renode provides a built-in USB/IP server and allows you to export virtual USB devices to the external world.

Exported devices can be attached to the host machine and from that moment the interaction with them is the same as with real hardware.

This allows for a hybrid setup where part of the system (i.e., virtual USB device and its environment) is simulated in Renode and the rest comes from the real world.

46.1 USB/IP protocol

The USB/IP protocol allows to share USB devices between a server (exporting party) and a client (importing party) over the IP network.

The protocol is **well supported in Linux**, and there exists a **corresponding project for Windows as well**, the latter however has not been tested.

Renode currently works as a server only - it is able to support exporting devices only, but is *not able* to connect physical USB devices to the emulation.

46.2 Creating a USB/IP server

In order to create a USB/IP server instance in Renode, type the following in the Monitor:

```
(monitor) emulation CreateUSBIPServer
```

As a result a `host.usb` device is created in the emulation and the server is started.

Now you can connect to it from your host machine. First, you need to import the `vhci_hcd` kernel module:

```
$ sudo modprobe vhci_hcd
```

Now, you can list exported devices:

```
$ sudo usbip list -r 127.0.0.1
usbip: info: no exportable devices found on 127.0.0.1
```

The output above informs that Renode is not exporting any devices at the moment.

46.3 Exporting devices

Let's export a simple USB device - e.g. a mouse:

```
(monitor) host.usb AttachUSBMouse
```

Note

Note that there are some helper methods allowing to easily attach simple USB devices like mouse, keyboard or pendrive directly from `host.usb`. For more advanced scenario see `fo-boot_usbip`.

List exported devices again:

```
$ sudo usbip list -r 127.0.0.1
Exportable USB devices
=====
- 127.0.0.1
    1-0: unknown vendor : unknown product (0000:0000)
        : /renode/virtual/1-0
        : (Defined at Interface level) (00/00/00)
        : 0 - Human Interface Device / Boot Interface Subclass / Mouse (03/01/02)
```

46.4 Attaching exported device to host

The next step is to attach the exported virtual USB device to the host machine:

```
$ sudo usbip attach -r 127.0.0.1 -b 1-0
```

Note that the `-b` argument must match the device id returned by the `usb list` command.

A new USB mouse should be now visible in the host. Confirm it by reading the system logs:

```
$ sudo dmesg
...
[1310770.549767] usb 7-1: New USB device found, idVendor=0000, idProduct=0000,
↳bcdDevice= 0.00
[1310770.549771] usb 7-1: New USB device strings: Mfr=0, Product=0, SerialNumber=0
[1310770.560676] input: HID 0000:0000 as /devices/platform/vhci_hcd.0/usb7/7-1/7-
↳1:1.0/0003:0000:0000.002D/input/input66
[1310770.560888] hid-generic 0003:0000:0000.002D: input,hidraw4: USB HID v0.00
↳Mouse [HID 0000:0000] on usb-vhci_hcd.0-1/input0
```

Or with `lsusb`:

```
$ lsusb -v -d 0000:0000

Bus 007 Device 091: ID 0000:0000
Device Descriptor:
bLength          18
```

(continues on next page)

(continued from previous page)

```

bDescriptorType      1
bcdUSB                2.00
bDeviceClass          0
bDeviceSubClass       0
bDeviceProtocol       0
bMaxPacketSize0      64
idVendor              0x0000
idProduct             0x0000
bcdDevice             0.00
iManufacturer         0
iProduct              0
iSerial               0
bNumConfigurations    1
Configuration Descriptor:
  bLength              9
  bDescriptorType      2
  wTotalLength         0x0022
  bNumInterfaces       1
  bConfigurationValue  1
  iConfiguration       0
  bmAttributes         0x00
  (Missing must-be-set bit!)
  (Bus Powered)
  MaxPower             0mA
  Interface Descriptor:
    bLength            9
    bDescriptorType     4
    bInterfaceNumber    0
    bAlternateSetting   0
    bNumEndpoints       1
    bInterfaceClass     3 Human Interface Device
    bInterfaceSubClass  1 Boot Interface Subclass
    bInterfaceProtocol  2 Mouse
    iInterface          0
      HID Device Descriptor:
        bLength        9
        bDescriptorType 33
        bcdHID         0.00
        bCountryCode   0 Not supported
        bNumDescriptors 1
        bDescriptorType 34 Report
        wDescriptorLength 46
      Report Descriptors:
        ** UNAVAILABLE **
  Endpoint Descriptor:
    bLength            7
    bDescriptorType     5
    bEndpointAddress    0x81  EP 1 IN
    bmAttributes        3

```

(continues on next page)

(continued from previous page)

Transfer Type	Interrupt
Synch Type	None
Usage Type	Data
wMaxPacketSize	0x0004 1x 4 bytes
bInterval	10

Now you can control your mouse from Renode. Type:

```
(monitor) host.usb MoveMouse 100 100
```

and observe the cursor moving on the screen.

46.5 Real life scenario: Foboot

In this section we will show how to run a simulation of **Foboot: The Bootloader for Fomu**.

Foboot runs on the Fomu platform that uses the ValentyUSB core to implement a software-driven USB device where the whole logic (including generating USB descriptors) is executed by the CPU.

46.5.1 Create a Fomu platform

Renode comes with a definition of the Fomu platform. To create a new virtual Fomu instance, type in the Monitor:

```
(monitor) mach create
(machine-0) machine LoadPlatformDescription @platforms/cpus/fomu.repl
```

Load the Foboot software:

```
(machine-0) sysbus LoadELF @https://antmicro.com/projects/renode/fomu--foboot.elf-
↪s_112080-70b1181d470646a31ebef7300fc8e6dc5447e282
```

Create a USB/IP server and export Fomu:

```
(machine-0) emulation CreateUSBIPServer
(machine-0) host.usb Register sysbus.valenty
```

Start the emulation:

```
(machine-0) start
```

46.5.2 Use it on your host machine

Import Fomu on the host:

```
$ sudo usbip attach -r 127.0.0.1 -b 1-0
```

Upload software using dfu-util:

```
$ wget https://antmicro.com/projects/renode/fomu--test_binary_flash.bin-s_1016-
↪4a3c37baf69aeb401f834521b0ac4bc6d157ecdf -O fomu--test_binary_flash.bin
$ sudo dfu-util -D fomu--test_binary_flash.bin
dfu-util 0.9

Copyright 2005-2009 Weston Schmidt, Harald Welte and OpenMoko Inc.
Copyright 2010-2016 Tormod Volden and Stefan Schmidt
This program is Free Software and has ABSOLUTELY NO WARRANTY
Please report bugs to http://sourceforge.net/p/dfu-util/tickets/

dfu-util: Invalid DFU suffix signature
dfu-util: A valid DFU suffix will be required in a future dfu-util release!!!
Opening DFU capable USB device...
ID 1209:5bf0
Run-time device DFU version 0101
Claiming USB DFU Interface...
Setting Alternate Setting #0 ...
Determining device status: state = dfuIDLE, status = 0
dfuIDLE, continuing
DFU mode device DFU version 0101
Device returned transfer size 1024
Copying data from PC to DFU device
Download [=====] 100% 1016 bytes
Download done.
state(7) = dfuMANIFEST, status(0) = No error condition is present
state(8) = dfuMANIFEST-WAIT-RESET, status(0) = No error condition is present
Done!
```

Disconnect dfu-util to reboot into the uploaded software:

```
$ sudo dfu-util -e
```

As Fomu does not have many interfaces we can observe, the uploaded binary is quite trivial. Looking through the log you will see repeated writes of consecutive values to 0x40000000.

To analyze the loaded binary in more detail you can use Renode's *GDB debugging capabilities* or extensive *logging support*.

CO-SIMULATING YOUR VERILATED MODEL

Creating HDL models and co-simulating them in Renode using Verilator is an advanced topic, so be sure to visit the *Co-simulating with an HDL simulator* chapter beforehand. If you want to use example verilated models as inspiration to build your own ones, visit [Antmicro's GitHub](#).

Note

You can run this tutorial on Linux and Windows operating systems.

47.1 Creating a verilated peripheral

To make your own verilated peripheral, in the main cpp file of your verilated model, you need to include C++ headers applicable to the bus you are connecting to.

You also need to specify the type of external interfaces you want to integrate with Renode - e.g., UART's rx/tx signals.

```
// uart.h and axilite.h can be found in Renode's CoSimulationPlugin
#include "src/peripherals/uart.h"
#include "src/buses/axilite.h"
```

Next, you will need to define a function that will call your model's eval function and provide it as a callback to the integration library structure, along with bus and peripheral signals.

```
void eval() {
    top->eval();
    #if VM_TRACE
    main_time++;
    tfp->dump(main_time);
    tfp->flush(); // optional
    #endif
}

void Init() {
    AxiLite* bus = new AxiLite();

    //=====
    // Init bus signals
    //=====
```

(continues on next page)

(continued from previous page)

```

bus->clk = &top->clk;
bus->rst = &top->rst;
bus->awaddr = (unsigned long *)&top->awaddr;
bus->awvalid = &top->awvalid;
bus->awready = &top->awready;
bus->wdata = (unsigned long *)&top->wdata;
bus->wstrb = &top->wstrb;
bus->wvalid = &top->wvalid;
bus->wready = &top->wready;
bus->bresp = &top->bresp;
bus->bvalid = &top->bvalid;
bus->bready = &top->bready;
bus->araddr = (unsigned long *)&top->araddr;
bus->arvalid = &top->arvalid;
bus->arready = &top->arready;
bus->rdata = (unsigned long *)&top->rdata;
bus->rresp = &top->rresp;
bus->rvalid = &top->rvalid;
bus->rready = &top->rready;

//=====
// Init eval function
//=====
bus->evaluateModel = &eval;

//=====
// Init peripheral
//=====
uart = new UART(bus, &top->txd, &top->rx,
    prescaler);
}

```

As part of the last step, in the main function, you have to call `simulate` with two port numbers on which Renode awaits communication.

These port numbers are passed as command-line arguments when Renode starts the verilated peripheral.

```

Init();
uart->simulate(atoi(argv[1]), atoi(argv[2]));

```

47.2 Building a verilated peripheral

There are a few prerequisites:

- a local copy of [the renode-verilator-integration repository](#),
- a local copy of [the Renode repository](#) because of its `IntegrationLibrary`,
- Verilator $\geq$ v4.024.

`$RVI_PATH`, `$RENODE_PATH`, and `$VERILATOR_PATH`, respectively, will be used to refer to the paths

of these three prerequisites. The path to Verilator won't be needed if it's properly installed, i.e., the executable is available in PATH and the `verilator-config.cmake` can be found according to the [CMake's find_package search procedure](#).

The path to the directory containing Verilog and C/C++ source files will be referred to as `$SRC_PATH`. It's best to place that directory directly in the `renode-verilator-integration` repository root which ensures that the default path to the [configure-and-verilate.cmake](#) file that contains the CMake logic common to all peripherals is correct.

Note

To run shell commands without any modifications, set all `*_PATH` shell variables before running the commands.

47.2.1 Preparing the peripheral directory

First, put all Verilog and C/C++ source files in `$SRC_PATH`. Then, copy the `$RVI_PATH/cmake/CMakeLists.txt.template` as `CMakeLists.txt` to the `$SRC_PATH` directory:

```
# Execute from a directory containing peripheral's source files
mkdir "$SRC_PATH"
cp *.v *.c *.cpp "$SRC_PATH"
cp "$RVI_PATH/cmake/CMakeLists.txt.template" "$SRC_PATH/CMakeLists.txt"
```

The project's `$SRC_PATH/CMakeLists.txt` file needs minor adjustments to work well with a specific peripheral (only the first two are required):

- replace `<PROJECT_NAME>` with a chosen name,
- replace `<MODULE_FILES>` and `<C_SRC_FILES>` with paths to the peripheral files relative to the `$SRC_PATH`,
- add chosen arguments to be always used during a certain phase of the build by removing `#` and replacing `<ARGS>` with actual arguments in lines that set the `COMP_ARGS`, `LINK_ARGS` and `VERI_ARGS` variables,
- if the peripheral's source directory isn't placed directly in `<RVI_PATH>`, then adjust the path to `configure-and-verilate.cmake` in the include command.

Note

Use a space to separate multiple files or arguments replacing `<*_FILES>` and `<ARGS>` placeholders.

47.2.2 Build commands

Having the CMake source directory prepared, the verilated peripheral can now be built. When using CMake, it's best to keep the build files in a separate build directory:

```
mkdir build && cd build
cmake -DCMAKE_BUILD_TYPE=Release -DUSER_RENODE_DIR="$RENODE_PATH" ${VERILATOR_
↪PATH:+"-DUSER_VERILATOR_DIR=$VERILATOR_PATH"} "$SRC_PATH"
make
```

If the build succeeds, libVtop is the built verilated peripheral.

Note

RENODE_ROOT and VERILATOR_ROOT environment variables can be used instead of CMake USER_RENODE_DIR and USER_VERILATOR_DIR variables (respectively). In case both environment and CMake variables are specified for Renode or Verilator, CMake variables have higher priority.

Note

Use `make -j $(nproc)` (`make -j $(sysctl -n hw.logicalcpu)` on macOS) to optimize build speed by creating as many jobs as the number of available CPU cores.

47.2.3 Linux-specific build information

On Linux, it is advised to use OpenLibm to enable running verilated peripherals with older GNU libc versions. This is because a few common mathematical functions have recently been updated in the GNU libm (a part of GNU libc). If the peripheral is linked against them, you will need these updated functions to run the peripheral.

Pass an additional `-DLIBOPENLIBM=$RVI_PATH/lib/libopenlibm-Linux-x86_64.a` argument to the `cmake` command to use the OpenLibm library that is currently also used by Renode.

47.2.4 Windows-specific build information

The steps above, with a few minor changes, were tested to successfully build peripherals on Windows with **Cygwin** and **MSYS2**. MSYS2 has a well-supported **mingw-w64-verilator** package, so it doesn't require building Verilator.

Note

On Windows, it's even more important to use absolute paths. These could be Cygwin/MSYS2 absolute paths, i.e. `/home/<...>`. Both Windows and Unix path styles are supported.

The CMake command from the *Build commands* section requires adding these arguments to work on Windows with Cygwin/MSYS2 and MinGW:

- `-G "MinGW Makefiles"` – to generate a Makefile for MinGW make,
- `-DCMAKE_SH=CMAKE_SH-NOTFOUND` – required for CMake to work on Windows despite having `sh.exe` in PATH.

Additionally, in the most common toolchain setups, the `mingw32-make` command should be used instead of `make` even if both are available.

Therefore, on Windows, a verilated peripheral can be built with:

```
cmake -G "MinGW Makefiles" -DCMAKE_SH=CMAKE_SH-NOTFOUND -DCMAKE_BUILD_
  ↳TYPE=Release -DUSER_RENODE_DIR="$RENODE_PATH" ${VERILATOR_PATH:+"-DUSER_
```

(continues on next page)

(continued from previous page)

```
↪ VERILATOR_DIR=$VERILATOR_PATH"} "$SRC_PATH"
mingw32-make
```

47.3 Running a verilated peripheral

After building a verilated executable, it's time to attach it to a *Renode machine*, so it is actually used as a peripheral.

First, a dedicated peripheral has to be added to a *Renode platform description (.repl)* file that is going to be used to configure the machine. For a co-simulated UART peripheral called, e.g., `myCoSimulatedPeripheral`, add these lines into your `.repl` file:

```
myCoSimulatedPeripheral: CoSimulated.CoSimulatedUART @ sysbus <0x70000000, +0x100>
    frequency: 100000000
```

Note

The `CoSimulated.CoSimulatedPeripheral` type should be used instead of `CoSimulated.CoSimulatedUART` for co-simulated peripherals other than UART.

Note

The example uses communication based on the library calls integration. To use the socket based intergration you should add the address parameter to the REPL file and use the `Vtop` binary instead.

In Renode, after loading such a platform description with a command either directly in the *Renode monitor* or with an appropriate *Renode script (.resc)* file, the verilated executable needs to be attached. Assuming the `libVtop` binary is located in the Renode root directory, it can be attached with:

```
(machine-0) myCoSimulatedPeripheral SimulationFilePath @libVtop
```

Otherwise, an absolute path or a path relative to the Renode root directory can be used instead of `libVtop`.

Note

Paths must start with the `@` symbol or be surrounded by double quotes `"`.

47.4 Using Renode Verilator sample peripherals

To build and use one of the verilated peripherals models from the [renode-verilator-integration repository](#), simply substitute your custom HDL with one of the examples and *follow the same steps as above*.

Note

You can skip **Preparing the peripheral directory** as their directories are prepared already.

47.5 Performance of the simulation

You can control the performance of the verilated peripheral in two aspects: its virtual time performance in relation to the main CPU and the real-time performance of execution.

As the example above shows, you need to define the clock frequency for each peripheral. The frequency parameter expects a value in Hz.

This value is used to drive the clock signal of the verilated design and is defined in the virtual time domain. It means that each instruction executed by the CPU, configured with a specific PerformanceInMips value, leads to a constant number of clock ticks in the design. For more details, please see *the chapter on Time Framework*.

Since it would be impractical to trigger clock signals after every instruction executed by the CPU, you can buffer these events and send them when you reach a certain threshold. This can be easily configured with the optional limitBuffer constructor parameter:

```
myCoSimulatedPeripheral: CoSimulated.CoSimulatedUART @ sysbus <0x70000000, +0x100>
    frequency: 100000000
    limitBuffer: 10000
```

The default value for the limitBuffer is 1000000, meaning that Renode will not trigger clock signals until it accumulates 1 million ticks.

47.6 Verilator Trace

You can also enable signal trace dumping by setting the --trace or --trace-fst Verilator option in CMakeList.txt corresponding to your verilated model.

Follow the directions below to ensure correct initialization and use of the verilated dump object.

Firstly, include the following definitions. These will enable tracing and allow you to switch between fst and vcd file types with the aforementioned Verilator options.

```
#if VM_TRACE_VCD
# include <verilated_vcd_c.h>
# define VERILATED_DUMP VerilatedVcdC
# define DEF_TRACE_FILEPATH "simx.vcd"
#elif VM_TRACE_FST
# include <verilated_fst_c.h>
# define VERILATED_DUMP VerilatedFstC
# define DEF_TRACE_FILEPATH "simx.fst"
#endif
```

Next, declare the verilated dump object and include collecting signal data with each model evaluation.

```

#if VM_TRACE
    VERILATED_DUMP *tfp;
#endif
vuint64_t main_time = 0;

void eval() {
    top->eval();
    #if VM_TRACE
        main_time++;
        tfp->dump(main_time);
        tfp->flush();
    #endif
}

```

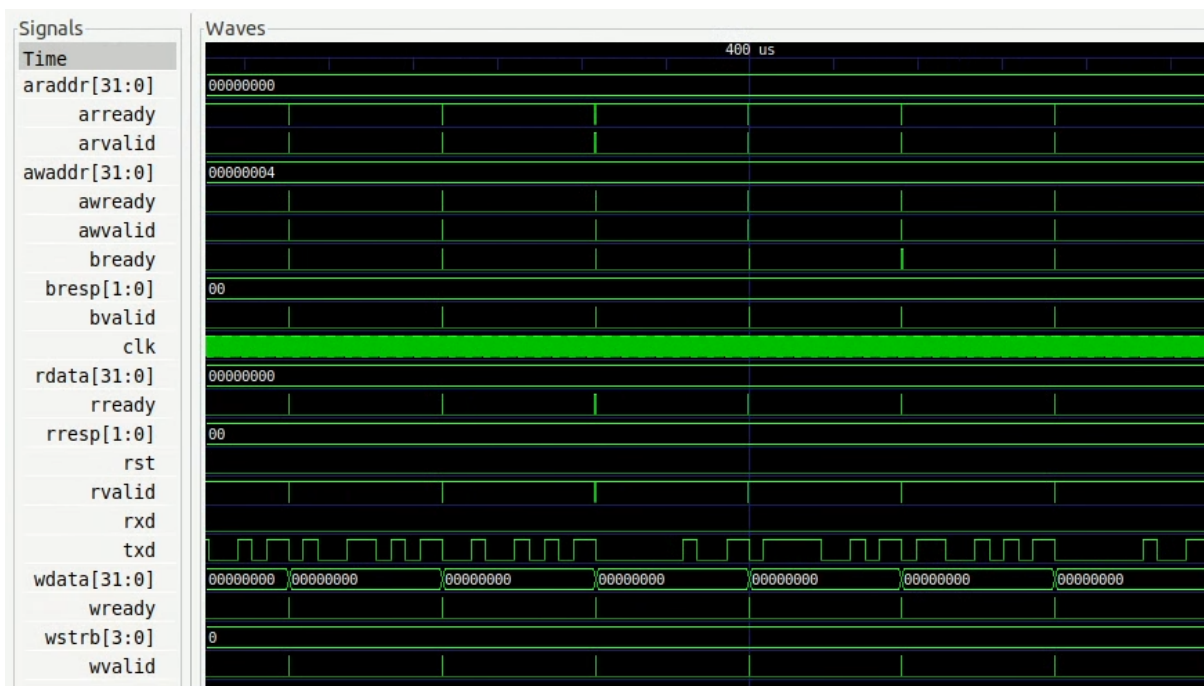
Finally, initialize the verilated dump and run the trace. If you would like to acquire the verilated dump from your co-simulation ran on sockets, please include this part within the main() function; otherwise place it within the Init() function.

```

#if VM_TRACE
    Verilated::traceEverOn(true);
    tfp = new VERILATED_DUMP;
    top->trace(tfp, 99);
    tfp->open(DEF_TRACE_FILEPATH);
#endif

```

The resulting trace is written into a vcd or fst file depending on the specified option and can be viewed in, e.g. [GTKWave viewer](#).



47.7 Core-v-mcu “Hello World” example with verilated UART

47.7.1 Prepare the binary

Instructions on how to set up the SDK are available on the [pulp-builder repository](#). After configuration, set the PULPRT_HOME environment variable with a path to the pulp-rules directory.

You also need to edit the SDK source code. To write a character to the tx UART register, add in `__rt_putc_uart` function in [io.c](#) file:

```
*((volatile uint32_t*)(0x50000004)) = c;
```

The “Hello World” code source can be found at [pulp-rt-examples](#). To compile, run:

```
make all io=uart
```

The resulting binary should be created in the `pulp-rt-examples/hello/build/arnold/test` directory.

47.7.2 Run in Renode simulation

To enable a verilated UART peripheral in the core-v-mcu hello world example, you need to register `CoSimulatedUART` in [core-v-mcu.repl](#), e.g.:

```
verilated_uart: CoSimulated.CoSimulatedUART @ sysbus <0x50000000, +0x100>
    frequency: 100000000
```

Then, you have to provide a binary to the Renode simulation in the Renode monitor type:

```
(monitor) using sysbus
(monitor) mach create
(machine-0) machine LoadPlatformDescription @platforms/cpus/core-v-mcu.repl
```

Attach your binary to the simulation:

```
(machine-0) sysbus LoadELF @path_to_your_binary
```

You can use your verilated UART model:

```
(machine-0) verilated_uart SimulationFilePath @path_to_verilated_uart_model
```

Or you can use the prebuilt one provided by us:

```
(machine-0) $uart?=@https://dl.antmicro.com/projects/renode/verilator--uartlite_
↪trace_off-s_252704-c703fe4dec057a9cbc391a0a750fe9f5777d8a74
(machine-0) verilated_uart SimulationFilePath $uart
```

To enable the UART analyzer window and start simulation, type:

```
(machine-0) showAnalyzer verilated_uart
(machine-0) s
```

BLUETOOTH LOW ENERGY SIMULATION IN RENODE

Bluetooth Low Energy is a widespread wireless protocol most commonly used in consumer devices such as blood pressure monitors, wearables, and smart home appliances. Thanks to its multi-node networking capabilities, Renode can be used to develop and test complete products in those and other areas using BLE.

Renode support for BLE has primarily been developed in the context of the [Zephyr RTOS](#) and Nordic's [nRF52840 SoC](#).

48.1 Running a precompiled demo

You can start with a precompiled demo which will just run two nodes communicating over BLE. Use the following command in the Monitor to do that:

```
(monitor) include @scripts/multi-node/nrf52840-ble-zephyr.resc
```

You should see two serial ports open and packets being sent.

48.2 Building your own Zephyr samples

You can see the full list of Zephyr samples and demos in the [Zephyr documentation](#) which describes in more detail how to install and use the RTOS.

Here we will focus on some essentials which are specific to this demo.

To build the relevant samples, after [setting up Zephyr as usual](#) you can use the following commands.

```
cd ~/zephyrproject/zephyr
west build -b nrf52840dk_nrf52840 -d central samples/bluetooth/central_hr
cp central/zephyr/zephyr.elf ./zephyr-ble-central_hr.elf

west build -b nrf52840dk_nrf52840 -d peripheral samples/bluetooth/peripheral_hr
cp peripheral/zephyr/zephyr.elf ./zephyr-ble-peripheral_hr.elf
```

Note

If you want to run a new build and remove byproducts of the previous build, you should add `-p auto` parameter to your command.

To use these binaries in the demo shipped with Renode, you can override the relevant variables before you load the script:

```
(monitor) $central_bin=@zephyr-ble-central_hr.elf
(monitor) $peripheral_bin=@zephyr-ble-central_hr.elf
(monitor) include scripts/multi-node/nrf52840-ble-zephyr.resc
```

48.3 Looking into the script

Below you can find a full example which creates 2 devices that generate and read heart-rate monitor data over BLE:

```
using sysbus

$central_bin?=@zephyr-ble-central_hr.elf
$peripheral_bin?=@zephyr-ble-central_hr.elf

emulation CreateBLEMedium "wireless"

mach create "central"
machine LoadPlatformDescription @platforms/cpus/nrf52840.repl
connector Connect sysbus.radio wireless

showAnalyzer uart0

mach create "peripheral"
machine LoadPlatformDescription @platforms/cpus/nrf52840.repl
connector Connect sysbus.radio wireless

showAnalyzer uart0

emulation SetGlobalQuantum "0.00001"

macro reset
"""
    mach set "central"
    sysbus LoadELF $central_bin

    mach set "peripheral"
    sysbus LoadELF $peripheral_bin
"""
runMacro $reset

echo "Script loaded. Now start with the 'start' command."
echo ""
```

The script is responsible for creating two machines, opening up their UART analyzers, connecting them in a single network, and loading the provided binaries.

The first Renode “machine” - dubbed `central` - runs the `central_hr sample` which looks for active heart-rate monitors using BLE and connects to the device with the strongest signal. The second

one - peripheral - runs the `peripheral_hr` sample, which creates a machine that functions as a heart-rate monitor and generates dummy heart-rate values.

When the connection is established, central will report data reception from peripheral.

To learn more about specific commands, see the *Working with machines* chapter and [the comments in the script on the Renode repository](#).

48.4 Packet interception hooks with BLE

You can make your simulated machine react to a wireless networking (e.g. BLE) packet appearing on the radio medium by means of a Python hook. Such a hook will execute Python code either directly from the Monitor or from a designated file. To set up a packet interception hook on the machine from the example above, you would run:

```
(peripheral) wireless SetPacketHookFromScript radio "self.DebugLog('Received a_
↪ packet of {} bytes'.format(len(packet)))"
```

BLE HCI INTEGRATION WITH RENODE

The Bluetooth Low Energy standard allows the use of different transports (e.g., UART, USB, SPI) for the HCI protocol. Integration with an external BLE controller via [HCI UART](#) is supported in Renode and the integration can be used for testing the [BLE host stack in Zephyr RTOS](#).

Note

This tutorial was tested with [Zephyr 3.4.0](#) and accompanying [Zephyr SDK 0.16.1](#).

49.1 Integration with Android Emulator

[Android Emulator](#) (a part of the [Android Studio IDE](#) that can be also installed as a standalone tool) supports Bluetooth communication through the [netsim tool](#). It is possible to connect Zephyr's BLE host stack emulated in Renode to the channel of netsim to test communication between an Android device and an emulated board.

49.1.1 Building Zephyr samples for a BLE HCI demo

You can find a complete list of Zephyr samples and demos in the [Zephyr Documentation](#), which describes in detail how to install and use the RTOS.

We will pick [nRF52840 DK](#) board as a sample target. To be able to generate some Zephyr binaries on your system, complete the [Zephyr Getting Started Guide](#) first. Next, create a `nrf52840dk_nrf52840_ble_hci_uart.overlay` file in the `zephyr` directory. This file is used to assign the selected UART to the BLE HCI transport. This file should contain the following content:

```
/ {
    chosen {
        zephyr,bt-uart = &arduino_serial;
    };
};

&arduino_serial {
    status = "okay";
};
```

In most cases, it is possible to create similar overlays for other boards that don't support BLE natively.

Now, you can use the following command to build the `peripheral_hr` sample for the nRF52840 DK. The sample is built with the HCI UART transport enabled:

```
west build -p auto -b nrf52840dk_nrf52840 -d hci_peripheral_hr samples/bluetooth/
↪peripheral_hr -- \
-DCONFIG_BT_HCI=y -DCONFIG_BT_CTLR=n -DCONFIG_BT_H4=y \
-DCONFIG_BT_EXT_ADV=n -DCONFIG_BT_HCI_ACL_FLOW_CONTROL=n \
-DDTC_OVERLAY_FILE=$PWD/nrf52840dk_nrf52840_ble_hci_uart.overlay
```

The built binary should be located in `hci_peripheral_hr/zephyr/zephyr.elf`.

The configuration options used during build may vary depending on the emulated target board and any external BLE controller used during integration. See [the Zephyr RTOS documentation](#) for possible options.

To use these binaries in Renode, load your platform and expose the selected UART to the host via the socket terminal for integration with an external BLE controller:

```
(machine-0) emulation CreateServerSocketTerminal 3456 "ble_hci_uart" false
(machine-0) connector Connect sysbus.uart1 ble_hci_uart
```

You can also use the generic script for the `nrf52840dk_nrf52840` board that is distributed with Renode, you will just need to set some variables for port number (`$port`) and binary path (`$bin`) before loading the script. You can run this command from the command line:

```
renode -e "$port=3456; $bin=@/home/user/zephyrproject/zephyr/hci_peripheral_hr/
↪zephyr/zephyr.elf; i @scripts/complex/hci_uart/hci_uart.resc"
```

If you try to run the emulation at this point, the Zephyr RTOS will assert a timeout due to a missing connection to the BLE controller. In the next step, you will learn how to connect to the BLE controller to run the full example.

Note

On Linux or macOS, you can also use a *pty terminal*.

49.1.2 Setting up Android Emulator

Note

Android Emulator version 33.1.14 or later is required to test the Bluetooth integration.

Before you can set up the Android emulator from the command line, you need to have the Java Runtime Environment installed on your system. To install the prerequisites on a Debian-based system, use the following command:

```
sudo apt install default-jre unzip wget
```

Now, you can run the following commands to set up Android Emulator:

Note

You can set `BASE_PATH` to the directory, where you want to download Android SDK.

```
export BASE_PATH=$HOME

export ANDROID_HOME=$BASE_PATH/android-sdk

export CMDLINE_TOOLS_DIR=$ANDROID_HOME/cmdline-tools
export SDK_LATEST=$CMDLINE_TOOLS_DIR/latest
export EMULATOR_DIR=$ANDROID_HOME/emulator
export PLATFORM_TOOLS_DIR=$ANDROID_HOME/platform-tools

export PATH=$PATH:$EMULATOR_DIR:$PLATFORM_TOOLS_DIR:$SDK_LATEST/bin/
export ANDROID_AVD_HOME=$ANDROID_HOME/system-images

export TEMP_CMDLINE_ZIP=$CMDLINE_TOOLS_DIR/commandlinetools.zip
export TEMP_EMULATOR_ZIP=$ANDROID_HOME/emulator.zip

mkdir -p $CMDLINE_TOOLS_DIR

# https://developer.android.com/studio#command-tools
CMD_DOWNLOAD_URL=https://dl.google.com/android/repository/commandlinetools-linux-
↪9477386_latest.zip

wget -O $TEMP_CMDLINE_ZIP $CMD_DOWNLOAD_URL
unzip $TEMP_CMDLINE_ZIP -d $CMDLINE_TOOLS_DIR
mv $CMDLINE_TOOLS_DIR/cmdline-tools $SDK_LATEST
rm $TEMP_CMDLINE_ZIP

yes | sdkmanager --licenses
sdkmanager --install "platform-tools"
sdkmanager --install "platforms;android-34"

# https://developer.android.com/studio/emulator_archive
EMULATOR_VERSION=33.1.14
EMULATOR_DOWNLOAD_URL=https://redirector.gvt1.com/edgedl/android/repository/
↪emulator-linux_x64-9997245.zip

wget -O $TEMP_EMULATOR_ZIP $EMULATOR_DOWNLOAD_URL
unzip $TEMP_EMULATOR_ZIP -d $ANDROID_HOME
rm $TEMP_EMULATOR_ZIP

sdkmanager --install "system-images;android-34;google_apis;x86_64"

export AVD_NAME_0=Android0
echo "no" | avdmanager --verbose create avd -n $AVD_NAME_0 -k "system-images;
↪android-34;google_apis;x86_64"
```

This script downloads and installs the latest version of the Android SDK command-line tools

and the Android Emulator. It also creates an Android Virtual Device (AVD) named Android0 based on the Android API level 34 system image.

49.1.3 Running Renode and Android Emulator together

To transfer HCI packets from the socket terminal created in Renode to the virtual controller in Android, you need to use **bumble** to decode the gRPC protocol (used by Android Emulator for communication) messages to HCI commands.

Note

Don't install bumble inside a conda environment, because socket module distributed with Anaconda doesn't support Bluetooth sockets. It will cause an exception while trying to bind to HCI socket:

```
AttributeError: module 'socket' has no attribute 'AF_BLUETOOTH'
Exception: Bluetooth HCI sockets not supported on this platform
```

You can use:

```
pipx install git+https://github.com/google/bumble.git@8eeb58e467
```

to install it in an isolated environment.

Install the bumble module:

```
python -m pip install git+https://github.com/google/bumble.git@8eeb58e467
```

To see, if bumble-hci-bridge is available as a global tool, run:

```
bumble-hci-bridge --help
```

Run Android Emulator:

```
emulator -avd Android0 -accel auto -gpu auto
```

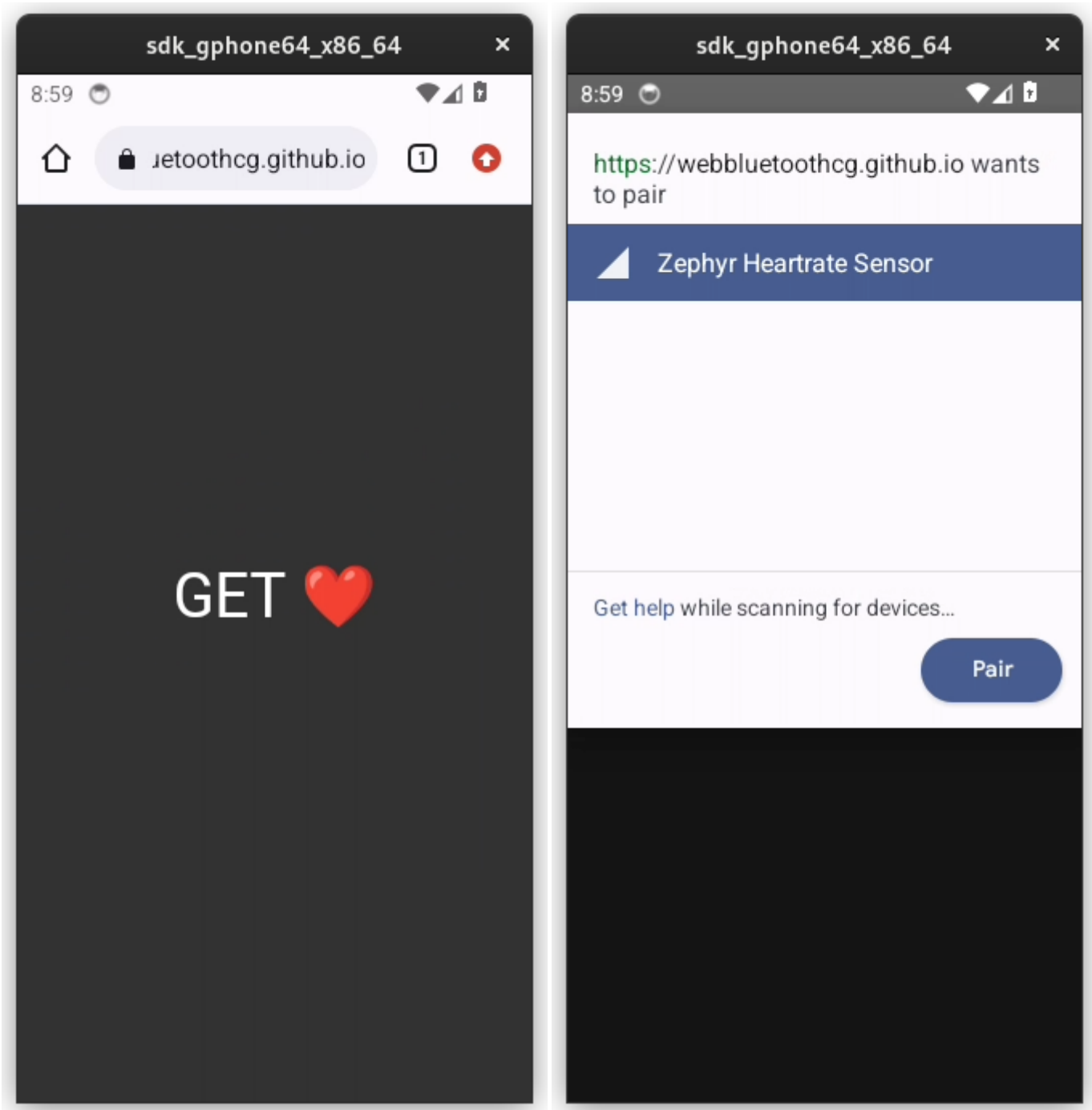
Now, you can establish a connection between the Renode device configured in the previous step and the Android emulator using bumble-hci-bridge:

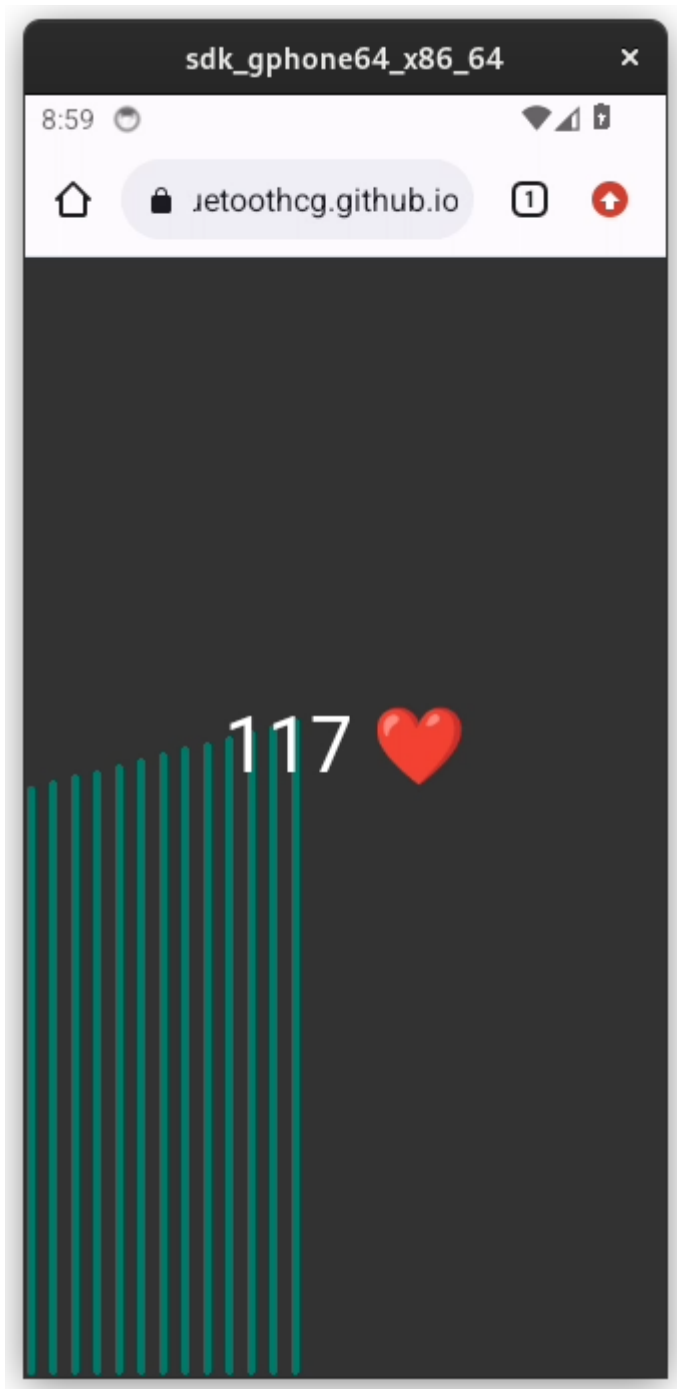
```
bumble-hci-bridge tcp-client:127.0.0.1:3456 android-netsim 0x03:0x0031,0x08:0x013,  
↪0x08:0x032,0x08:0x016,0x03:0x035
```

To connect Renode with Android Emulator:

1. Start Renode simulation (your platform should connect to the HCI bridge created earlier).
2. On an emulated Android device, open the Chrome browser and go to <https://webbluetoothcg.github.io/demos/heart-rate-sensor/>.
3. Click on the page and grant permission for Bluetooth access.

You should be able to see and connect to a BLE peripheral emulated in Renode as shown below.





49.2 Bluetooth Mesh networking

The Zephyr BLE stack provides support for the BLE Mesh protocol. You can create a virtual Mesh network with multiple Renode instances emulating separate BLE devices.

49.2.1 Building the Zephyr BLE Mesh sample

Build the `mesh sample` that will be loaded in Renode for the `nrf52840dk_nrf52840` platform as follows:

```
west build -p auto -b nrf52840dk_nrf52840 -d hci_mesh samples/bluetooth/mesh -- \
-DCONFIG_BT_HCI=y -DCONFIG_BT_CTLR=n -DCONFIG_BT_H4=y \
-DCONFIG_BT_EXT_ADV=n -DCONFIG_BT_HCI_ACL_FLOW_CONTROL=n \
-DCONFIG_BT_SETTINGS=n -DCONFIG_NVS=n -DCONFIG_SETTINGS=n -DCONFIG_HWINFO=n \
-DDTC_OVERLAY_FILE=$PWD/nrf52840dk_nrf52840_ble_hci_uart.overlay
```

The built binary should be located in `hci_mesh/zephyr/zephyr.elf`.

49.2.2 Integration with an external BLE controller

On the host side, you can use tools that provide support for connecting to either physical or virtual BLE controllers: `btvirt` and `btproxy` from [BlueZ](#) or `bumble-hci-bridge` and `bumble-link-relay` from [bumble](#).

The BlueZ use case is described in [Using BlueZ with Zephyr](#) and various aspects of BLE integration are well documented in the [bumble project](#), which provides solutions to many [common problems](#) with permissions.

49.2.3 Creating virtual controllers

Note

This part depends on BlueZ and the `/dev/vhci` device on the host system, so it is limited to Linux hosts. On other systems, you can test it in a Linux virtual machine. If you want to run these steps in a Docker container, it should be started with the following flags `--device=/dev/vhci --net=host --cap-add=CAP_NET_ADMIN`.

`btvirt` tool is part of [bluez-tests on Ubuntu](#) and [bluez-test-tools on Debian](#). It can be installed using `apt` package manager.

To build bluez from source on a Debian-based system:

```
sudo apt update

sudo apt install -y wget xz-utils git bc libusb-dev libdbus-1-dev libglib2.0-dev \
↳ libudev-dev libical-dev libreadline-dev autoconf bison flex libssl-dev \
↳ libncurses-dev libdbus-1-dev python3-docutils cmake udev systemd

wget https://github.com/json-c/json-c/archive/refs/tags/json-c-0.16-20220414.tar.
↳ gz
tar xvf json-c-0.16-20220414.tar.gz
cd json-c-json-c-0.16-20220414
mkdir build && cd build
cmake -DCMAKE_INSTALL_PREFIX=/usr -DCMAKE_BUILD_TYPE=Release -DBUILD_STATIC_
↳ LIBS=OFF ..
make
sudo make install

wget http://www.kernel.org/pub/linux/bluetooth/bluez-5.68.tar.xz
tar -xvf bluez-5.68.tar.xz
cd bluez-5.68
```

(continues on next page)

(continued from previous page)

```
./configure --enable-mesh --enable-testing --enable-tools --enable-deprecated --
↪enable-experimental --prefix=/usr --mandir=/usr/share/man --sysconfdir=/etc --
↪localstatedir=/var
make -j$(nproc)
```

When built from source, btvirt is located in the emulator directory.

The number of virtual Bluetooth controllers that can be created with btvirt is limited to 16 devices by default. If you'd like to use Renode to simulate a BLE Mesh network consisting of more devices, you can increase the compile time constant `MAX_BTDEV_ENTRIES` and rebuild BlueZ. If you want to exceed 128 virtual devices, you should also increase another constant `MAX_MAINLOOP_ENTRIES`.

Note

Commands used in this tutorial assume that you don't have any HCI interfaces available on your system. You can verify it with the `hciconfig` command. If an HCI interface is already registered, use the appropriate index in `hci-socket:<i>` to account for it. The `btvirt` command creates HCI for virtual controller. You can check the created interfaces with `hciconfig`.

To create a BLE Mesh network consisting of three devices simulated in Renode communicating each-to-each symmetrically, run the following commands in separate terminals:

```
$ sudo btvirt -d -l3
$ renode -e "$port=3456; $bin=@/home/user/zephyrproject/zephyr/hci_mesh/zephyr/
↪zephyr.elf; i @scripts/complex/hci_uart/hci_uart.resc"
$ renode -e "$port=3457; $bin=@/home/user/zephyrproject/zephyr/hci_mesh/zephyr/
↪zephyr.elf; i @scripts/complex/hci_uart/hci_uart.resc"
$ renode -e "$port=3458; $bin=@/home/user/zephyrproject/zephyr/hci_mesh/zephyr/
↪zephyr.elf; i @scripts/complex/hci_uart/hci_uart.resc"
$ sudo capsh --caps="cap_net_admin+eip cap_setpcap,cap_setuid,cap_setgid+ep" --
↪keep=1 --user=\$USER --addamb=cap_net_admin -- -c "\$(which bumble-hci-bridge)
↪tcp-client:127.0.0.1:3456 hci-socket:0"
$ sudo capsh --caps="cap_net_admin+eip cap_setpcap,cap_setuid,cap_setgid+ep" --
↪keep=1 --user=\$USER --addamb=cap_net_admin -- -c "\$(which bumble-hci-bridge)
↪tcp-client:127.0.0.1:3457 hci-socket:1"
$ sudo capsh --caps="cap_net_admin+eip cap_setpcap,cap_setuid,cap_setgid+ep" --
↪keep=1 --user=\$USER --addamb=cap_net_admin -- -c "\$(which bumble-hci-bridge)
↪tcp-client:127.0.0.1:3458 hci-socket:2"
```

You can call `hciconfig` to make sure that virtual HCIs were created by `btvirt`. You should see HCIs that belong to virtual bus (Bus: Virtual). They disappear after you kill the `btvirt` process.

```
hci2:      Type: Primary  Bus: Virtual
          BD Address: 00:AA:01:02:00:02  ACL MTU: 192:1  SCO MTU: 0:0
          DOWN
          RX bytes:0 acl:0 sco:0 events:46 errors:0
          TX bytes:523 acl:0 sco:0 commands:46 errors:0
```

(continues on next page)

(continued from previous page)

```

hci1:      Type: Primary  Bus: Virtual
BD Address: 00:AA:01:01:00:01  ACL MTU: 192:1  SCO MTU: 0:0
DOWN
RX bytes:0 acl:0 sco:0 events:46 errors:0
TX bytes:523 acl:0 sco:0 commands:46 errors:0

hci0:      Type: Primary  Bus: Virtual
BD Address: 00:AA:01:00:00:00  ACL MTU: 192:1  SCO MTU: 0:0
DOWN
RX bytes:0 acl:0 sco:0 events:46 errors:0
TX bytes:523 acl:0 sco:0 commands:46 errors:0

```

Note

To be able to test the alternative Zephyr BLE stack running in Renode, make sure to **disable the Bluetooth service**. To see the status of the Bluetooth service on a Debian-based system, run `sudo systemctl status bluetooth`.

In some cases, you may need to use `sudo rfkill unblock all` to unblock your Bluetooth wireless device.

Alternatively, you can connect virtual HCI to Renode using the socat relay tool and btproxy, which is available in the tools directory when BlueZ is built from source. You can use these commands if you do not want to assign temporary extended capabilities to the bumble-hci-bridge Python program.

```

$ sudo btvirt -d -l3
$ renode -e "$port=3456; $bin=@/home/user/zephyrproject/zephyr/hci_mesh/zephyr/
↪zephyr.elf; i @scripts/complex/hci_uart/hci_uart.resc"
$ renode -e "$port=3457; $bin=@/home/user/zephyrproject/zephyr/hci_mesh/zephyr/
↪zephyr.elf; i @scripts/complex/hci_uart/hci_uart.resc"
$ renode -e "$port=3458; $bin=@/home/user/zephyrproject/zephyr/hci_mesh/zephyr/
↪zephyr.elf; i @scripts/complex/hci_uart/hci_uart.resc"
$ sudo ./tools/btproxy -u"/tmp/bt-server-bredr0" -i 0
$ sudo ./tools/btproxy -u"/tmp/bt-server-bredr1" -i 1
$ sudo ./tools/btproxy -u"/tmp/bt-server-bredr2" -i 2
$ socat TCP-CONNECT:127.0.0.1:3456 UNIX-CONNECT:/tmp/bt-server-bredr0
$ socat TCP-CONNECT:127.0.0.1:3457 UNIX-CONNECT:/tmp/bt-server-bredr1
$ socat TCP-CONNECT:127.0.0.1:3458 UNIX-CONNECT:/tmp/bt-server-bredr2

```

Note

You may use TCP server instead of Unix server in btproxy with slightly modified commands:

```

sudo ./tools/btproxy -l127.0.0.1 -p1000 -i 0
socat TCP-CONNECT:127.0.0.1:3456 TCP-CONNECT:127.0.0.1:1000

```

Next:

1. Start the Renode simulation (your platform should connect to the HCI bridge you created earlier).
2. Enter `gpio0.sw0 PressAndRelease` in Renode's monitor for each Renode instance to provision BLE devices to Mesh network.
3. All successive button presses (`gpio0.sw0 PressAndRelease`) will cause the message to be sent to other nodes in the network and `led0` blinking on all boards.
4. You can watch `led0` state in Renode's monitor with command `watch "gpio0.led0 State" 200`, which prints the led state every 200 milliseconds.

The following messages should be printed on the `uart0` console:

```
*** Booting Zephyr OS build zephyr-v3.3.0 ***
Initializing...
[00:00:00.041,351] <inf> bt_hci_core: bt_dev_show_info: Identity:
↪C8:7F:54:3D:8E:49 (public)
[00:00:00.041,442] <inf> bt_hci_core: bt_dev_show_info: HCI: version 5.1 (0x0a)
↪revision 0x09a9, manufacturer 0x005d
[00:00:00.041,442] <inf> bt_hci_core: bt_dev_show_info: LMP: version 5.1 (0x0a)
↪subver 0x8a6b
Bluetooth initialized
[00:00:00.041,534] <inf> bt_mesh_prov_device: bt_mesh_prov_enable: Device UUID:
↪00000000-0000-0000-0000-00000000dddd
Mesh initialized
Self-provisioning with address 0x1fd8
[00:00:08.152,130] <inf> bt_mesh_main: bt_mesh_provision: Primary Element: 0x1fd8
[00:00:08.152,130] <dbg> bt_mesh_main: bt_mesh_provision: net_idx 0x0000 flags
↪0x00 iv_index 0x0000
Provisioned and configured!
Sending OnOff Set: on
set: on delay: 0 ms time: 0 ms
set: off delay: 0 ms time: 0 ms
set: on delay: 0 ms time: 0 ms
```

BLE devices are emulated in separate Renode instances and communicate in a virtual network through the virtual HCI interfaces owned by the Linux operating system. You can observe traffic on these interfaces in Wireshark using HCI protocol dissector.

If you have multiple computers with physical BLE controller (either USB BLE adapter or built-in BLE module), you can use Renode to emulate BLE devices on different computers and make them communicate in BLE Mesh network to achieve real physical arrangement.

49.2.4 Integration with a physical BLE controller

An external USB BLE adapter (you can build one from [Zephyr HCI USB sample](#)) or a built-in BLE module can be used for integration.

To establish a connection between Renode and the USB BLE adapter using `bumble-hci-bridge`:

```
bumble-hci-bridge tcp-client:127.0.0.1:3456 usb:0
```

Note

You may need to change the permissions for the USB device to **access it as a regular user**.

You can connect directly to the HCI socket if the kernel has already registered an interface:

```
sudo capsh --caps="cap_net_admin+eip cap_setpcap,cap_setuid,cap_setgid+ep" --
↪keep=1 --user=\$USER --addamb=cap_net_admin -- -c "$(which bumble-hci-bridge)
↪tcp-client:127.0.0.1:3456 hci-socket:0"
```

Note

Do not start the emulation until the bridge is configured, otherwise the BLE host stack may timeout due to lack of connection to the BLE controller.